

Solver Specification and Verification Report:

`highOrderManufacturedFDAImplicitPETSc`

Manufactured monodomain-like PDE–ODE solver with high-order finite volumes, Picard, diagonal- I_{ion} , and JFNK nonlinear coupling on a PETSc Krylov backend

Pablo

May 27, 2026

Abstract

This report documents the numerical formulation implemented in:

`highOrderManufacturedFDAImplicitPETSc`.

The solver is a method of manufactured solutions (MMS) analogue of the electro-activation solver: it solves a monodomain-like reaction–diffusion PDE for the transmembrane potential V_m , evolves local state ODEs for (u_1, u_2, u_3) , evaluates a nonlinear ionic-current surrogate $I_{\text{ion}}(V_m, u_1, u_2, u_3)$, and allows the same nonlinear coupling strategies used in the physiological solver: `Picard`, `diagonalIion`, and `JFNK`. The PDE can be discretised with the standard cell-centred finite-volume operator or with a high-order Local Regression Estimators (LRE) reconstruction. The nonlinear source can be evaluated at cell centres or integrated from cell quadrature points, where the state ODEs are also advanced. Time integration is a θ method with `backwardEuler` ($\theta = 1$) or `crankNicolson` ($\theta = 1/2$), and the mass matrix can be `lumped` or high-order `consistent`. Linear voltage systems are solved through a thin PETSc wrapper that caches the KSP, matrix and ILU(T) symbolic factorisation across nonlinear iterations and time steps; an Eigen fallback (`SparseLU/BiCGSTAB`) is also available. `JFNK` uses a persistent matrix-free `MatShell`+KSP pair with restarted GMRES and an optional preconditioner derived from $\mathbf{A}$ with a local I_{ion} linearisation (`diagonalIion`). The sparse-matrix assemblies of $\mathbf{M}$ and $\mathbf{K}$ are chunked over cells/faces to bound the transient triplet-vector peak on large 3D unstructured meshes. The current implementation also exposes an automatic memory-optimisation mode with adaptive stiffness triplet reservation, compact consistent-mass assembly, conditional allocation of ionic quadrature-point fields, and allocator trimming around large setup phases. The results section is data driven: tables are generated from the current tutorial outputs and the plotted figures can be regenerated with the supplied MATLAB scripts.

Contents

1	Purpose and notation	2
2	Governing equations	3
2.1	Electrophysiology motivation	3
2.2	Manufactured PDE–ODE system	3
2.3	Exact solution and the role of β	4
3	Spatial discretisation	4
3.1	Finite-volume integral form	4
3.2	Standard cell-centred operator	4
3.3	High-order LRE operator	5
3.4	Lumped and consistent mass matrices	5
3.5	High-order integration of I_{ion} and states	6

4	Time integration	6
4.1	The θ method	6
4.2	ODE states inside one PDE time step	7
5	Nonlinear coupled solve	7
5.1	Residual and diagnostics	7
5.2	Picard fixed-point method	8
5.3	Diagonal- I_{ion} linearisation	8
5.4	Jacobian-free Newton–Krylov	9
5.5	Armijo line search and rollback	9
6	Linear-algebra backend and implementation notes	10
6.1	JFNK preconditioner: <code>diagonalIion ILU(T)</code>	11
6.2	Chunked assembly of $\mathbf{M}$ and $\mathbf{K}$	11
6.3	Memory layout in the time loop	12
7	Theoretical accuracy and practical error floors	13
8	Solver controls	13
9	Output diagnostics	15
10	Error Measures and Convergence Rates	15
11	Results	16
11.1	Two-dimensional manufactured problem	17
11.1.1	Picard, <code>crankNicolson</code> , <code>consistent</code>	17
11.1.2	<code>diagonalIion</code> , <code>crankNicolson</code> , <code>consistent</code>	27
11.1.3	JFNK, <code>crankNicolson</code> , <code>consistent</code>	37
11.2	Two-dimensional results: comments	47
11.3	Three-dimensional manufactured problem	47
11.3.1	Picard, <code>crankNicolson</code> , <code>consistent</code>	47
11.3.2	<code>diagonalIion</code> , <code>crankNicolson</code> , <code>consistent</code>	58
11.3.3	JFNK, <code>crankNicolson</code> , <code>consistent</code>	68
12	Notes	78

1 Purpose and notation

The MMS problem is not a physiological ionic model. Its role is to provide an exact solution for a PDE–ODE–source structure that is deliberately close to the electro-activation solver. If the same finite-volume operator, same source quadrature, same state ODE integration, same θ time discretisation, and same nonlinear coupling are used here and in the electro solver, then the observed MMS convergence gives evidence for the numerical order of those shared building blocks.

Symbol	Meaning	Location in solver
$\Omega \subset \mathbb{R}^d$	computational domain, $d = 1, 2, 3$	OpenFOAM mesh
K_i	finite-volume cell	mesh.cells()
$ K_i $	cell volume	mesh.V()
$V_m(\mathbf{x}, t)$	manufactured transmembrane potential	Vm
$\mathbf{u} = (u_1, u_2, u_3)$	manufactured state variables	u1,u2,u3
$I_{\text{ion}}(V_m, \mathbf{u})$	nonlinear source surrogate	Iion
χ and C_m	monodomain scaling constants	cardiacProperties
$\mathbf{D}$	conductivity tensor	conductivity
$\mathbf{M}$	mass matrix	M (moved into BImplicit after $\mathbf{A}/\mathbf{E}$)
$\mathbf{K}$	diffusion/stiffness matrix	K
$\mathbf{L} = (\chi C_m)^{-1} \mathbf{K}$	scaled diffusion operator	absorbed into $\mathbf{A}/\mathbf{B}$ (no separate st)
$\mathbf{A}, \mathbf{B}$	implicit θ -scheme matrices	AImplicit, BImplicit
$\mathbf{F}(\mathbf{x})$	nonlinear residual for voltage candidate $\mathbf{x}$	residualFor
$\mathbf{P}$	persistent JFNK preconditioner matrix $\approx \mathbf{A} - \theta \text{diag}(\mathbf{d})$	jfnkP

The cell-centred vector of voltages at time t^n is denoted $\mathbf{V}^n \in \mathbb{R}^{N_c}$. A nonlinear iteration inside one time step uses the superscript k , e.g. $\mathbf{V}^k$ is a candidate for $\mathbf{V}^{n+1}$. Quadrature point q in cell K_i is denoted $\mathbf{x}_{iq}$ with positive weight ω_{iq} .

2 Governing equations

2.1 Electrophysiology motivation

The physiological monodomain equation can be written, in one common convention, as

$$\chi C_m \frac{\partial V_m}{\partial t} = \nabla \cdot (\mathbf{D} \nabla V_m) - \chi C_m I_{\text{ion}}(V_m, \mathbf{s}) + I_{\text{ext}}, \quad (1)$$

where $\mathbf{s}$ are ionic states and I_{ion} is often stored as a voltage rate with units V s^{-1} . Dividing by χC_m gives

$$\frac{\partial V_m}{\partial t} = \frac{1}{\chi C_m} \nabla \cdot (\mathbf{D} \nabla V_m) - I_{\text{ion}}(V_m, \mathbf{s}) + \frac{I_{\text{ext}}}{\chi C_m}. \quad (2)$$

The manufactured solver replaces the physiological states by three smooth MMS states and replaces the cell-model ionic current by a constructed nonlinear function. The purpose is not to reproduce cellular electrophysiology, but to exercise the same numerical responsibilities: diffusion, local ODEs, nonlinear source, quadrature, implicit time stepping, and nonlinear iteration.

2.2 Manufactured PDE–ODE system

The solver advances

$$\frac{\partial V_m}{\partial t} = \frac{1}{\chi C_m} \nabla \cdot (\mathbf{D} \nabla V_m) - I_{\text{ion}}(V_m, u_1, u_2, u_3), \quad (3)$$

$$\dot{u}_1 = (u_1 + u_3 - V_m)^2 u_2^2 + \frac{1}{2} (u_1 + u_3 - V_m) u_2^2 (V_m - u_3), \quad (4)$$

$$\dot{u}_2 = -(u_1 + u_3 - V_m) u_2^3, \quad (5)$$

$$\dot{u}_3 = 0. \quad (6)$$

The nonlinear ionic-current surrogate is

$$I_{\text{ion}}(V_m, u_1, u_2, u_3) = -\frac{1}{2} (u_1 + u_3 - V_m) u_2^2 (V_m - u_3) + \frac{\beta}{\chi C_m} (V_m - u_3). \quad (7)$$

The PDE source used in the matrix right-hand side is

$$S(V_m, \mathbf{u}) = -I_{\text{ion}}(V_m, \mathbf{u}). \quad (8)$$

Thus the term called `Iion` in the output is the nonlinear manufactured current, while the term passed into the voltage equation is its negative.

2.3 Exact solution and the role of β

The exact fields are built from two smooth spatial functions F and G :

$$V_m^*(\mathbf{x}, t) = \sqrt{1+t} F(\mathbf{x}), \quad (9)$$

$$u_1^*(\mathbf{x}, t) = (1+t)G(\mathbf{x}) + V_m^*(\mathbf{x}, t), \quad (10)$$

$$u_2^*(\mathbf{x}, t) = \frac{1}{(1+t)\sqrt{G(\mathbf{x})}}, \quad (11)$$

$$u_3^*(\mathbf{x}, t) = 0. \quad (12)$$

The solver uses

$$F = \begin{cases} \cos(\pi x), & d = 1, \\ \cos(\pi x) \cos(2\pi y), & d = 2, \\ \cos(\pi x) \cos(2\pi y) \cos(3\pi z), & d = 3, \end{cases} \quad G = \begin{cases} 1+x, & d = 1, \\ 1+xy^2, & d = 2, \\ 1+xy^2z^3, & d = 3. \end{cases} \quad (13)$$

The constant β closes the analytical balance between the Laplacian of F and the conductivity tensor. For a diagonal constant conductivity sampled from `conductivity[0]`,

$$\beta = \begin{cases} -\pi^2 D_{xx}, & d = 1, \\ -\pi^2 (D_{xx} + 4D_{yy}), & d = 2, \\ -\pi^2 (D_{xx} + 4D_{yy} + 9D_{zz}), & d = 3. \end{cases} \quad (14)$$

This choice makes the manufactured source compatible with the exact trigonometric mode under the selected conductivity tensor. Boundary values for V_m , u_1 , u_2 , and u_3 are overwritten with the exact solution where appropriate, so the verification error is a discretisation error rather than a boundary-data error.

3 Spatial discretisation

3.1 Finite-volume integral form

Integrating (3) over a control volume K_i gives

$$\int_{K_i} \frac{\partial V_m}{\partial t} dV = \frac{1}{\chi C_m} \int_{\partial K_i} \mathbf{n} \cdot \mathbf{D} \nabla V_m dA + \int_{K_i} S(V_m, \mathbf{u}) dV. \quad (15)$$

After spatial discretisation the semi-discrete problem has the matrix form

$$\mathbf{M} \frac{dV}{dt} = \mathbf{L}V + \mathbf{S}(V, \mathbf{u}), \quad \mathbf{L} = \frac{1}{\chi C_m} \mathbf{K}. \quad (16)$$

3.2 Standard cell-centred operator

When `useHighOrderVm=false`, the solver assembles the standard cell-centred finite-volume diffusion operator. For an internal face f between cells P and N , the orthogonal part is of the form

$$\int_f \mathbf{n} \cdot \mathbf{D} \nabla V_m dA \approx a_f (V_N - V_P), \quad (17)$$

where a_f contains the face area, face-normal direction, conductivity tensor, and distance between cell centres. Boundary faces use the OpenFOAM patch type. `zeroGradient` and `empty` patches give no-flux contributions; fixed value patches contribute known boundary terms.

For smooth solutions on regular orthogonal meshes, this standard finite-volume operator is second-order accurate in space for diffusion. On skewed or non-orthogonal meshes the effective order depends on the mesh quality and the non-orthogonal treatment.

3.3 High-order LRE operator

When `useHighOrder_Vm=true`, the solver uses Local Regression Estimators (LRE) stencils. Around each cell centre $\mathbf{x}_i$, a polynomial reconstruction of degree N is fitted from neighbouring cell values:

$$V_m(\mathbf{x}_i + \mathbf{r}) \approx V_i + \nabla V_i \cdot \mathbf{r} + \frac{1}{2} \mathbf{r}^T H_i \mathbf{r} + \frac{1}{6} T_i[\mathbf{r}, \mathbf{r}, \mathbf{r}] + \dots \quad (18)$$

The dictionary entries `N`, `Nn`, `k`, and `maxStencilSize` control the polynomial degree, target number of stencil neighbours, radial weight exponent, and maximum stencil size. The solver can evaluate reconstructed values, gradients, Hessians, and third-order tensors, depending on the selected polynomial order.

Boundary patches are classified automatically before the LRE objects are constructed. `fixedValue` and `fixedVoltage` patches are inserted into the LRE stencils as boundary ghost points so that the manufactured boundary value can be used. `zeroGradient` patches are passed to LRE as `mirrorPatches`, which doubles the local stencil by reflecting neighbour locations across the boundary plane and preserves the full reconstruction order at homogeneous Neumann boundaries. `empty`, `processor`, and `unsupported` patches are left out of the explicit include/mirror lists. A patch cannot be both included as a Dirichlet ghost patch and mirrored; the LRE constructor checks this and aborts if the dictionary is inconsistent.

The high-order dictionaries are read from `highOrderCoeffs/LRECoeffs_Vm` for the diffusion/mass reconstruction and from `highOrderCoeffs/LRECoeffs_Iion` for source/state quadrature. The legacy name `LRECoeffs_states` is still accepted for the latter, but the solver prints a warning and prefers `LRECoeffs_Iion`.

The high-order stiffness matrix is assembled by evaluating reconstructed gradients at face quadrature points:

$$\int_f \mathbf{n} \cdot \mathbf{D} \nabla V_m dA \approx \sum_{q \in f} \omega_{fq} \mathbf{n}_f \cdot \mathbf{D}_f \nabla_h V_m(\mathbf{x}_{fq}). \quad (19)$$

The resulting operator is sparse but has a wider stencil than the standard finite-volume Laplacian. For smooth fields, good stencils, sufficiently accurate quadrature, and non-degenerate meshes, the expected spatial consistency improves with the LRE polynomial degree. In practice the observed order is the minimum of: reconstruction order, quadrature order, time-integration order, ODE tolerance floor, nonlinear residual floor, and mesh/stencil conditioning. If the error is controlled only by the interpolation/reconstruction of V_m , then the expected behaviour is the usual polynomial one: linear reconstruction (p1) gives second-order accuracy, quadratic reconstruction (p2) gives third-order accuracy, and cubic reconstruction (p3) gives fourth-order accuracy.

3.4 Lumped and consistent mass matrices

The lumped mass matrix is diagonal:

$$\mathbf{M}_{ij}^{\text{lump}} = |K_i| \delta_{ij}. \quad (20)$$

It is cheap, robust, positive, and coincides with the usual finite-volume mass for one unknown per cell. If high-order V_m is disabled, the solver uses this diagonal mass even if the input asks for `consistent`, because there is no high-order reconstruction basis to integrate.

The `consistent` mass matrix is available when `useHighOrder_Vm=true` and is assembled with cell quadrature and the LRE reconstruction weights:

$$\mathbf{M}_{ij}^{\text{cons}} \approx \sum_K \sum_{q \in K} \omega_{Kq} \phi_i(\mathbf{x}_{Kq}) \phi_j(\mathbf{x}_{Kq}), \quad (21)$$

where ϕ_i are the reconstruction-induced basis functions. This better matches the high-order spatial approximation, but it is sparse rather than diagonal and increases the cost and conditioning sensitivity of the implicit solve.

3.5 High-order integration of I_{ion} and states

The source integral can be approximated either by a cell-centred rule or by quadrature. Since $S = -I_{\text{ion}}$, when `useHighOrder_Iion=false`,

$$S_i = \int_{K_i} S dV = - \int_{K_i} I_{\text{ion}} dV \approx -I_{\text{ion}}(V_i, u_{1,i}, u_{2,i}, u_{3,i}) |K_i|. \quad (22)$$

When `useHighOrder_Iion=true`, states are stored and advanced at dedicated cell quadrature points and the source integral is approximated as

$$S_i = \int_{K_i} S dV = - \int_{K_i} I_{\text{ion}} dV \approx - \sum_{q \in K_i} \omega_{iq} I_{\text{ion}}(V_{mh}(\mathbf{x}_{iq}), u_{1,iq}, u_{2,iq}, u_{3,iq}). \quad (23)$$

These quadrature-point scalar fields are allocated only for multi-dimensional high-order ionic integration, i.e. when `useHighOrder_Iion=true` and $d > 1$. In all other cases the solver still reports the nominal number of ionic integration points for auditability, but the allocated quadrature-point storage is zero and the source/state update stays cell-centred.

4 Time integration

4.1 The θ method

Let $\Delta t = t^{n+1} - t^n$. The solver maps `backwardEuler` to $\theta = 1$ and `crankNicolson` to $\theta = 1/2$. Applying the θ method to (16) gives

$$\mathbf{M} \frac{\mathbf{V}^{n+1} - \mathbf{V}^n}{\Delta t} = \mathbf{L} (\theta \mathbf{V}^{n+1} + (1 - \theta) \mathbf{V}^n) + \theta \mathbf{S}^{n+1} + (1 - \theta) \mathbf{S}^n. \quad (24)$$

Equivalently,

$$\mathbf{A} \mathbf{V}^{n+1} = \mathbf{B} \mathbf{V}^n + \theta \mathbf{S}^{n+1} + (1 - \theta) \mathbf{S}^n, \quad (25)$$

$$\mathbf{A} = \frac{1}{\Delta t} \mathbf{M} - \theta \mathbf{L}, \quad \mathbf{B} = \frac{1}{\Delta t} \mathbf{M} + (1 - \theta) \mathbf{L}. \quad (26)$$

In the implementation, $\mathbf{L}$ is never stored as a separate sparse matrix. The solver builds $\mathbf{M}$ and $\mathbf{K}$ once (with the chunked assemblies described in §6.2) and constructs $\mathbf{A}$, $\mathbf{B}$ directly from the relation $\mathbf{L} = (\chi C_m)^{-1} \mathbf{K}$ via expression-template arithmetic on the sparse matrices: $\mathbf{A} \leftarrow \mathbf{M}/\Delta t - (\theta/\chi C_m) \mathbf{K}$ and $\mathbf{B} \leftarrow \mathbf{M}/\Delta t + ((1 - \theta)/\chi C_m) \mathbf{K}$. After $\mathbf{A}$ is copied from $\mathbf{M}$, the storage of $\mathbf{M}$ is moved into $\mathbf{B}$ to avoid keeping a third copy of the mass matrix alive throughout the run.

`backwardEuler` is first order in time and strongly damping. It is usually the most robust choice for stiff or poorly converged nonlinear problems. `crankNicolson` is second order in time for smooth solutions and converged nonlinear/source terms, but is less dissipative. Its observed order can drop if the nonlinear iterations, state ODE tolerance, or spatial error dominate.

4.2 ODE states inside one PDE time step

For a voltage candidate $V_m^{n+1,k}$, the state ODEs are advanced from the same time-step-start states $\mathbf{u}^n$. Inside the ODE integrator, voltage is interpolated linearly between the old and candidate values:

$$V_m(\tau) = (1 - \alpha)V_m^n + \alpha V_m^{n+1,k}, \quad \alpha = \frac{\tau}{\Delta t}, \quad 0 \leq \tau \leq \Delta t. \quad (27)$$

The available state solvers are:

<code>stateODESolver</code>	Method	Nominal order
<code>Euler</code> or <code>forwardEuler</code>	explicit Euler	1
<code>RK4</code>	classical Runge–Kutta	4
<code>RKF45</code> or <code>RKT45</code>	adaptive embedded Fehlberg pair	4/5

For `RKF45`, the fifth-order candidate is accepted and the fourth-order embedded estimate supplies the local error indicator

$$e = \max_j \frac{|u_j^{(5)} - u_j^{(4)}|}{\text{stateODEAbsTol} + \text{stateODERelTol} \max(|u_j^{(5)}|, |u_j^{(4)}|)}. \quad (28)$$

A sub-step is accepted if $e \leq 1$ or if the minimum step has been reached; the next sub-step is scaled by a safety factor proportional to $e^{-1/4}$. In convergence studies, the ODE tolerances must be tight enough that the state integration error does not mask the spatial or PDE time-integration order.

5 Nonlinear coupled solve

5.1 Residual and diagnostics

For any voltage candidate $\mathbf{x}$, the solver evaluates the corresponding state fields and source by calling `evaluateNonlinearFields`. This means: reconstruct V_m to I_{ion} quadrature points if required, reset/evolve states from t^n to t^{n+1} using $\mathbf{x}$, compute I_{ion} , and assemble the source. The coupled voltage residual is

$$\mathbf{F}(\mathbf{x}) = \mathbf{A}\mathbf{x} - [\mathbf{B}\mathbf{V}^n + \theta (\mathbf{S}(\mathbf{x}) + \mathbf{b}^{n+1}) + (1 - \theta) (\mathbf{S}^n + \mathbf{b}^n)], \quad (29)$$

where $\mathbf{b}^n$ and $\mathbf{b}^{n+1}$ are known boundary/source contributions from the manufactured boundary treatment.

The solver records several relative diagnostics:

$$R_{\text{cpl}} = \frac{\|\mathbf{F}(\mathbf{x})\|_2}{\|\mathbf{r}(\mathbf{x})\|_2 + \epsilon}, \quad R_{V_m} = \frac{\|\mathbf{V}^{k+1} - \mathbf{V}^k\|_2}{\|\mathbf{V}^k\|_2 + \epsilon}, \quad (30)$$

$$R_{I_{\text{ion}}} = \frac{\|\mathbf{I}_{\text{ion}}^{k+1} - \mathbf{I}_{\text{ion}}^k\|_2}{\|\mathbf{I}_{\text{ion}}^k\|_2 + \epsilon}, \quad R_{u_j} = \frac{\|\mathbf{u}_j^{k+1} - \mathbf{u}_j^k\|_2}{\|\mathbf{u}_j^k\|_2 + \epsilon}. \quad (31)$$

Here ϵ is the OpenFOAM `SMALL` safeguard. When high-order I_{ion} quadrature is active, state residuals are measured at the quadrature points; otherwise they are measured at cell centres. The maximum state residual is

$$R_{\mathbf{u}} = \max(R_{u_1}, R_{u_2}, R_{u_3}). \quad (32)$$

The residual history is written to `postProcessing/highOrderManufacturedFDAImplicitPETSc/nonlinearResiduals.dat` when `writeNonlinearResiduals=true`.

5.2 Picard fixed-point method

Picard treats the nonlinear source as lagged during each linear voltage solve. Given $\mathbf{V}^k$, the solver computes $\mathbf{S}^k = \mathbf{S}(\mathbf{V}^k, \mathbf{u}(\mathbf{V}^k))$ and then solves

$$\mathbf{A}\hat{\mathbf{V}}^{k+1} = \mathbf{B}\mathbf{V}^n + \theta(\mathbf{S}^k + \mathbf{b}^{n+1}) + (1 - \theta)(\mathbf{S}^n + \mathbf{b}^n). \quad (33)$$

The relaxed update is

$$\mathbf{V}^{k+1} = \mathbf{V}^k + \omega \left(\hat{\mathbf{V}}^{k+1} - \mathbf{V}^k \right), \quad \omega = \text{nonlinearRelaxation}. \quad (34)$$

Convergence requires at least `minNonlinearIterations`; in the Picard branch the active criteria are $R_{V_m} \leq \text{nonlinearVmTolerance}$ and all state residuals below `nonlinearStatesTolerance`. If Picard exhausts the iteration budget, the current implementation accepts the last Picard iterate and prints a warning, matching the legacy `highOrderManufacturedFDAImplicit` behaviour.

Algorithm 1 Picard nonlinear coupling

Input: Old solution $\mathbf{V}^n, \mathbf{u}^n$, source $\mathbf{S}^n$

- 1: Build $\mathbf{A}$ and $\mathbf{B}$ from $\mathbf{M}, \mathbf{L}, \theta$, and Δt
- 2: Choose initial guess $\mathbf{V}^0$ from $\mathbf{V}^n$ or extrapolation
- 3: **for** $k = 0, \dots, N_{\text{nl}} - 1$ **do**
- 4: Evaluate states and source $\mathbf{S}^k$ from $\mathbf{V}^k$
- 5: Solve $\mathbf{A}\hat{\mathbf{V}}^{k+1} = \mathbf{r}(\mathbf{S}^k)$
- 6: Relax $\mathbf{V}^{k+1} = \mathbf{V}^k + \omega(\hat{\mathbf{V}}^{k+1} - \mathbf{V}^k)$
- 7: Re-evaluate/update states and source for diagnostics
- 8: Compute $R_{V_m}, R_{u_1}, R_{u_2}, R_{u_3}, R_{I_{\text{ion}}}$
- 9: **if** minimum iterations reached and Picard tolerances hold **then**
- 10: accept and exit loop
- 11: **end if**
- 12: **end for**

Output: Last Picard iterate is accepted; a warning is printed if not converged

5.3 Diagonal- I_{ion} linearisation

`diagonalIion` keeps the same scalar voltage unknowns but adds an approximate local derivative of the nonlinear source to the matrix. A forward finite difference is used:

$$d_i \approx \frac{S_i(\mathbf{V}^k + \varepsilon \mathbf{1}) - S_i(\mathbf{V}^k)}{\varepsilon}, \quad \varepsilon = \text{diagonalIionEpsilon}. \quad (35)$$

The linearised source is

$$S_i(\mathbf{V}^{k+1}) \approx S_i(\mathbf{V}^k) + d_i \left(V_i^{k+1} - V_i^k \right). \quad (36)$$

Moving the unknown part to the left gives

$$(\mathbf{A} - \theta \text{diag}(\mathbf{d})) \hat{\mathbf{V}}^{k+1} = \mathbf{r}(\mathbf{S}^k) - \theta \mathbf{d} \odot \mathbf{V}^k, \quad (37)$$

where $\odot$ is componentwise multiplication and

$$\mathbf{r}(\mathbf{S}^k) = \mathbf{B}\mathbf{V}^n + \theta \left(\mathbf{S}^k + \mathbf{b}^{n+1} \right) + (1 - \theta) (\mathbf{S}^n + \mathbf{b}^n). \quad (38)$$

This is the same right-hand side used by the Picard voltage solve; the `diagonalIion` branch only moves the local linearised source contribution to the left-hand side. This is not a full Newton Jacobian because it ignores off-diagonal source coupling introduced by reconstruction and ignores explicit state sensitivities beyond what the finite-difference source evaluation captures locally. It is often more implicit than Picard while remaining cheaper and simpler than JFNK.

Algorithm 2 diagonalIion nonlinear coupling

Input: Old solution $V^n, \mathbf{u}^n$, source S^n

```
1: Choose initial guess  $V^0$ 
2: for  $k = 0, \dots, N_{\text{nl}} - 1$  do
3:   Evaluate  $S^k$  and  $I_{\text{ion}}^k$  from  $V^k$ 
4:   Perturb  $V_m$  by diagonalIionEpsilon and re-evaluate source
5:   Form  $d_i \approx \partial S_i / \partial V_i$  by finite difference
6:   Solve  $(\mathbf{A} - \theta \text{diag}(\mathbf{d})) \hat{V}^{k+1} = \mathbf{r}(S^k) - \theta \mathbf{d} \odot V^k$ 
7:   Relax the voltage update
8:   Re-evaluate source/states at the relaxed voltage
9:   Check coupled, voltage,  $I_{\text{ion}}$ , and optional state criteria
10:  if minimum iterations reached and diagonal tolerances hold then
11:    accept and exit loop
12:  end if
13: end for
14: if not converged and nonlinearAcceptUnconverged=false then
15:   roll back  $V_m, I_{\text{ion}}$ , and states to the beginning of the time step
16: end if
```

5.4 Jacobian-free Newton–Krylov

JFNK seeks a zero of (29):

$$\mathbf{F}(\mathbf{x}) = 0. \quad (39)$$

Newton’s method would solve

$$\mathbf{J}(\mathbf{x}_k) \delta_k = -\mathbf{F}(\mathbf{x}_k), \quad \mathbf{x}_{k+1} = \mathbf{x}_k + \alpha \delta_k, \quad (40)$$

but the full Jacobian is not assembled. Instead, GMRES only needs products $\mathbf{J}\mathbf{v}$, approximated by the finite difference

$$\mathbf{J}(\mathbf{x})\mathbf{v} \approx \frac{\mathbf{F}(\mathbf{x} + \epsilon \mathbf{v}) - \mathbf{F}(\mathbf{x})}{\epsilon}, \quad \epsilon = \text{JfnkEpsilon} \frac{1 + \|\mathbf{x}\|_2}{\|\mathbf{v}\|_2 + \epsilon_{\text{mach}}}. \quad (41)$$

Each GMRES matrix-vector product therefore performs a full nonlinear residual evaluation: ODE solve, source evaluation, source quadrature if active, and matrix-vector application. This is the main cost of JFNK.

The inner linear solve uses restarted GMRES(m), where $m = \text{JfnkMaxKrylovIterations}$ and the number of restarts is `JfnkMaxRestarts`. The Arnoldi basis is orthogonalised with modified Gram–Schmidt, and the least-squares projected problem is solved with Eigen’s column-pivoted Householder QR. GMRES stops when its relative residual is below `JfnkLinearTolerance` or when the restart budget is exhausted.

5.5 Armijo line search and rollback

After GMRES proposes a Newton correction δ_k , the solver optionally uses Armijo backtracking. Starting from $\alpha = \text{nonlinearRelaxation}$, it accepts the trial step if

$$\|\mathbf{F}(\mathbf{x}_k + \alpha \delta_k)\|_2^2 \leq (1 - 2c\alpha) \|\mathbf{F}(\mathbf{x}_k)\|_2^2, \quad c = \text{JfnkArmijoC}. \quad (42)$$

If the condition fails, α is halved until either the condition is met, `JfnkLineSearchMaxIter` is reached, or `JfnkLineSearchAlphaMin` is reached. This globalises Newton by discouraging steps that increase the nonlinear residual.

For JFNK and `diagonalIion`, if the nonlinear solver reaches the maximum number of iterations without convergence, the default behaviour is a rollback: V_m, I_{ion} , and the state fields are restored to the beginning of the time step. Setting `nonlinearAcceptUnconverged=true` disables this rollback and accepts the last iterate. Picard is the exception: to preserve the legacy manufactured implicit behaviour, an unconverged Picard iteration accepts the last Picard iterate with a warning.

Algorithm 3 Jacobian-free Newton–Krylov nonlinear coupling

Input: Old solution V^n , u^n , old source S^n

```
1: Build initial guess  $x_0$  from  $V^n$  or extrapolation
2: for  $k = 0, \dots, N_{nl} - 1$  do
3:   Evaluate  $F(x_k)$ , states, source, and  $I_{ion}$ 
4:   if minimum iterations reached and nonlinear tolerances hold then
5:     accept  $x_k$  and exit loop
6:   end if
7:   Define matrix-free product  $v \mapsto [F(x_k + \epsilon v) - F(x_k)]/\epsilon$ 
8:   Use restarted GMRES to approximately solve  $J\delta_k = -F(x_k)$ 
9:   if jfnkLineSearch=true then
10:    Set  $\alpha \leftarrow \text{nonlinearRelaxation}$ 
11:    while Armijo condition (42) fails and budget remains do
12:       $\alpha \leftarrow \max(\alpha/2, \text{jfnkLineSearchAlphaMin})$ 
13:    end while
14:  else
15:     $\alpha \leftarrow \text{nonlinearRelaxation}$ 
16:  end if
17:  Update  $x_{k+1} = x_k + \alpha\delta_k$ 
18:  Re-evaluate states/source at  $x_{k+1}$  and write residual diagnostics
19:  if minimum iterations reached and nonlinear tolerances hold then
20:    accept  $x_{k+1}$  and exit loop
21:  end if
22: end for
23: if not converged and nonlinearAcceptUnconverged=false then
24:   roll back  $V_m$ ,  $I_{ion}$ , and states to their time-step-start values
25: end if
```

6 Linear-algebra backend and implementation notes

The PDE-level operators $\mathbf{A}$, $\mathbf{B}$, $\mathbf{K}$, and the JFNK preconditioner $\mathbf{P}$ are stored as Eigen `SparseMatrix<scalar, RowMajor>` while the heavy linear solves run through PETSc by default. The solver wraps the PETSc API in two small classes that exist in the anonymous namespace of the translation unit:

- `PetscKspMatrixSolver` owns a (matrix, KSP, preconditioner) triple. The first call to `reset()` translates an Eigen `SparseMatrix` into a PETSc `MatSeqAIJ`, builds the KSP, and triggers the symbolic ILU(T) factorisation. Subsequent updates call `updateValues()`: this zeros the PETSc `Mat` in place (`MatZeroEntries`), refreshes its numerical entries (`MatSetValue/MatAssemblyEnd`), and asks the PETSc preconditioner to recompute its numerical factor while reusing the existing symbolic structure (`KSPSetReusePreconditioner(PETSC_FALSE)` followed by `KSPSetUp`). This means $\mathbf{P}$ can be refreshed many times per time step without freeing and re-allocating the ILU(T) factor.
- `PetscShellKspSolver` wraps a matrix-free shell-matrix used by JFNK. A persistent `Mat` of type `MatShell`, a KSP, the right-hand side and solution vectors, and the two callback contexts (`matVec`, `applyPreconditioner`) are allocated once outside the time loop. Each Newton iteration only refreshes the `std::function` payloads of those contexts and calls `KSPSolve`; the underlying GMRES Krylov subspace is reused. Numerically this is identical to constructing a fresh shell matrix per Newton iteration, but it avoids allocating and freeing roughly $40 m N_c$ scalars per JFNK call (with $m = \text{jfnkPetscRestart}$ and N_c the number of cells).

For users who prefer to stay in pure Eigen, `linearSolverBackend` can be set to `Eigen`. In that mode the voltage solve uses `Eigen::SparseLU` or `Eigen::BiCGSTAB` and the JFNK inner GMRES uses the in-file implementations `solveGMRES` (no preconditioner) or `solveLeftPreconditionedGMRES` with

the Eigen IncompleteLUT. The two backends are numerically equivalent for the same matrix and tolerances: they differ only in the linear solver implementation, not in the PDE residual.

PETSc type names are normalised before the KSP/PC objects are created. KSP aliases such as PETSc, GMRES, KSPGMRES, and JFNK select PETSc gmres; BiCGSTAB/BCGS select bcgs; and SparseLU/LU select preonly. PC aliases off, false, none, and noPC select PETSc none; ILUT selects ilu; SparseLU/LU select lu; and the common misspelling jakobi is accepted as jacobi. If PetscUseOptions=true, PETSc’s options database can still override solver details through the configured options prefix.

6.1 JFNK preconditioner: diagonalIion ILU(T)

When jfnkPreconditioner=diagonalIion, the inner GMRES applies a left preconditioner $\mathbf{P}^{-1}$ defined implicitly by an ILU(T) factorisation of

$$\mathbf{P} = \mathbf{A} - \theta \text{diag}(\mathbf{d}), \quad d_i = \frac{S_i(\mathbf{x} + \varepsilon \mathbf{1}) - S_i(\mathbf{x})}{\varepsilon}, \quad (43)$$

which is exactly the matrix that the diagonalIion nonlinear method would solve. The preconditioner solver type (jfnkPreconditionerKspType, jfnkPreconditionerPcType) defaults to a direct application of the ILU(T) factor (preonly+ilu). The two ILU(T) controls jfnkPreconditionerDropTolerance and jfnkPreconditionerFillFactor are passed verbatim to PETSc’s PCFactorSetDropTolerance and PCFactorSetFill. The preconditioner is refreshed every jfnkPreconditionerUpdateFrequency Newton iterations inside a time step. Across time steps it stays alive through the cached PetscKspMatrixSolver above. Selecting jfnkPreconditioner=none disables the left preconditioner and runs unpreconditioned GMRES on the matrix-free Jacobian.

The persistent matrix $\mathbf{P} = \text{jfnkP}$ is allocated once. The first preconditioner update copies $\mathbf{A}$ into $\mathbf{P}$ (full sparse copy); all subsequent updates only overwrite the value buffer $\mathbf{P}.\text{valuePtr}()$ from $\mathbf{A}.\text{Implicit.valuePtr}()$ via `std::copy` and then add the diagonal correction in place. This is correct because the sparsity pattern of $\mathbf{A}$ is invariant in time, and the `addDiagonalToMatrix` helper only touches diagonal entries that already exist in $\mathbf{A}$ (both $\mathbf{M}$ and $\mathbf{K}$ have non-zero diagonals).

6.2 Chunked assembly of M and K

The high-order stiffness matrix is the dominant transient memory cost of the setup phase. On a 3D unstructured tetrahedral mesh with $\sim 290\text{k}$ cells, p_3 LRE reconstruction, ~ 60 -cell stencils, and stabilisation active, the monolithic assembly accumulates of order 4×10^8 Eigen triplets in a single `std::vector<Triplet>`. Because the initial reservation cannot match this exactly without an extra pre-pass, the vector would grow by capacity-doubling, briefly holding both the old and new buffer in memory.

To avoid that transient peak, the solver assembles both $\mathbf{M}$ and $\mathbf{K}$ in fixed-size chunks. For $\mathbf{K}$ the chunks are blocks of `faceChunk = 50 000` internal faces; for $\mathbf{M}$ they are blocks of `cellChunk = 50 000` cells. Each chunk fills a triplet buffer of bounded size, builds a chunk-local sparse matrix $\mathbf{K}_{\text{local}}$ (resp. $\mathbf{M}_{\text{local}}$) via `setFromTriplets`, and folds it into the accumulated matrix:

$$\mathbf{K} \leftarrow \mathbf{K} + \mathbf{K}_{\text{local}}, \quad \mathbf{M} \leftarrow \mathbf{M} + \mathbf{M}_{\text{local}}, \quad (44)$$

where the first chunk uses assignment instead of accumulation. After the chunked sweep over internal faces, the boundary-face contributions are appended through the same flush mechanism. The triplet buffer is cleared between chunks and explicitly released (`std::vector<Triplet>().swap(triplets)`) at the end of the routine.

For the stiffness matrix, the chunk triplet reservation is controlled by `stiffnessTripletsPerFaceReserve`. With `memoryOptimization=off` this keeps the conservative historical value of 800 triplets per face. With `memoryOptimization=on`, or with `memoryOptimization=auto` on large 3D high-order cases, the reserve is selected from `memoryOptimizationFluxTripletsPerFace` for flux-only assemblies and from `memoryOptimizationStabilisedTripletsPerFace` when stabilisation is active. This changes only the

capacity of the temporary `std::vector`; the set of triplets that pass `addTripletIfNeeded(...)` is unchanged.

For the consistent high-order mass matrix, `compactMassAssembly` can accumulate all quadrature-point contributions to a cell row in a small local buffer before emitting triplets. The compact path therefore emits one triplet per final row/stencil entry instead of one triplet per quadrature contribution. Mathematically it assembles the same mass matrix; only the local floating-point summation order is changed. The first non-empty mass and stiffness chunks are moved into the global Eigen sparse matrix with `swap()`, avoiding an extra full sparse copy during setup.

Numerically the chunked assembly is identical to the monolithic one: sparse matrix addition is associative and commutative, and the per-triplet filter `addTripletIfNeeded(...)` is independent of the chunking because it inspects each triplet value before insertion.

6.3 Memory layout in the time loop

Several sparse matrices and PETSc objects live across the time loop, so the solver is careful about which copies are kept:

- **M** is moved (not copied) into **B** once the implicit matrices are assembled. After this point **M** is empty and only **A** and **B** hold the time-discrete operators.
- In the Picard/diagonalIon branches, the modified system matrix $\mathbf{A}_{\text{Current}} = \mathbf{A} - \theta \text{diag}(\mathbf{d})$ is allocated once outside the corrector loop; subsequent corrector iterations refresh its value buffer from **A** via `std::copy` and re-apply the diagonal correction in place. The branch is also skipped entirely for pure Picard when the PETSc cached solver is used, since that branch never needs a separately stored linearised matrix.
- The Picard solve uses a single `PetscKspMatrixSolver` cached across the run via `reset()` on the first call. Because **A** is time-constant in the Picard branch, no further numerical updates are needed.
- The JFNK preconditioner solver uses the `updateValues()` fast path described above, so the ILU(T) factor is built once symbolically and only its numerical entries are recomputed per refresh.
- The seven ionic quadrature-point scalar fields (`VmOld`, `VmGuess`, `source`, `Iion`, and three states) are sized with `allocatedIionIntegrationPoints`. This is zero unless high-order ionic integration is actually active in $d > 1$, so pure cell-centred source runs do not pay for unused quadrature-point storage.
- The diagnostic high-order flux/laplacian reconstruction is not evaluated on every time step. The solver always forms the numerical laplacian used in the RHS diagnostic from the already assembled matrix expression $\mathbf{KV} + \mathbf{b}$. The expensive `computeHighOrderLaplacian` path that fills `fluxVm.H0` is only run when an output time needs the flux field, and once more during final post-processing.
- On glibc systems the solver tightens the allocator before any large allocation: `M_ARENA_MAX` is set to 2 to prevent the OpenMP ODE thread pool from forcing dozens of arenas, `M_MMAP_THRESHOLD` and `M_TRIM_THRESHOLD` are lowered to 64 KiB so that mid-sized Eigen temporaries during the LRE QR factorisations are placed in `mmap`'d regions that are returned to the OS on free. When `memoryOptimizationTrimHeap=true` and the memory optimisation mode is effective, `malloc_trim(0)` is called after the mass-matrix assembly, after the stiffness-matrix assembly, and after **A/B** are built; this releases the heap padding left behind by the very large number of small Eigen allocations performed during LRE setup.
- When `profileTimings=true`, setup also prints peak-RSS checkpoints before and after mass assembly, before and after stiffness assembly, and after implicit matrix construction. These

checkpoints report the monotone `getrusage(RUSAGE_SELF).ru_maxrss` peak, not the instantaneous RSS.

These choices do not change any numerical output; they only change peak resident memory and the cost of repeated linear solves.

7 Theoretical accuracy and practical error floors

For a smooth manufactured solution and exact nonlinear solves, the intended orders are:

Component	Expected order	Main caveat
standard FV diffusion	second order in space on regular orthogonal meshes	mesh quality
LRE high-order reconstruction	increases with polynomial degree N	stencil quality and quadrature
backwardEuler	first order in time	numerical damping
crankNicolson	second order in time	requires converged nonlinear/source terms
Euler states	first order in time	rarely appropriate for high-order studies
RK4 states	fourth order for smooth ODEs	fixed step over PDE Δt
RKF45 states	adaptive fourth/fifth-order pair	tolerance sets error floor
Picard/JFNK/diagonal	nonlinear iteration, not spatial order by itself	residual tolerance must be small enough

The measured MMS order is the order of the entire chain. It can be limited by the largest of the following errors: spatial reconstruction, mass matrix approximation, source quadrature, PDE time stepping, state ODE integration, linear solver tolerance, nonlinear residual tolerance, and floating-point or conditioning effects. In particular, a high-order spatial method will not show its nominal order unless Δt , ODE tolerances, and nonlinear tolerances are sufficiently tight.

8 Solver controls

The numerical controls are read from `constant/spatialIntegrationProperties` and `constant/timeIntegrationProperties`. The tutorial `run_convergence.py` writes these dictionaries.

Control	Meaning
<code>useHighOrder_Vm</code>	Selects standard FV or LRE high-order diffusion/mass reconstruction for V_m .
<code>useHighOrder_Iion</code>	Selects cell-centred source/state update or quadrature-point source/state update.
<code>useHighOrder_states</code>	Legacy alias used only if <code>useHighOrder_Iion</code> is absent.
<code>implicitScheme</code>	backwardEuler or crankNicolson ; sets θ .
<code>massMatrix</code>	lumped diagonal FV mass or LRE consistent mass.
<code>highOrderCoeffs/stabilisationAlpha</code>	Optional positive face-stabilisation strength for the high-order diffusion operator; defaults to zero.
<code>highOrderCoeffs/LRECoeffs_Vm</code>	LRE settings N , N_n , k , and <code>maxStencilSize</code> for V_m diffusion and the consistent mass matrix.
<code>highOrderCoeffs/LRECoeffs_Iion</code>	LRE settings N , N_n , k , and <code>maxStencilSize</code> for ionic quadrature. The legacy <code>LRECoeffs_states</code> name is accepted with a warning.
<code>linearSolverBackend</code>	PETSc (default) routes the voltage solve through PETSc; Eigen keeps the original in-process Eigen path.
<code>linearSolver</code>	Eigen sparse solver name when the Eigen backend is selected: BiCGSTAB or SparseLU .
<code>tolerance</code> , <code>maxIterations</code>	Linear PDE solver stopping tolerance and iteration cap.
<code>petscLinearKspType</code> , <code>petscLinearPcType</code> , <code>petscLinearRestart</code>	PETSc KSP type (gmres , bcgs , preonly , ...), preconditioner type (ilu , lu , none , ...) and GMRES restart length for the Picard voltage solve.

Control	Meaning
<code>petscLinearOptionsPrefix,</code> <code>petscUseOptions</code>	Options-database prefix and switch that lets the user override KSP/PC settings from the PETSc options database.
<code>nonlinearMethod</code>	Picard, <code>diagonalIion</code> , or JFNK.
<code>nonlinearIterations</code>	Maximum nonlinear iterations per PDE time step.
<code>minNonlinearIterations</code>	Minimum nonlinear iterations before convergence can be accepted.
<code>nonlinearTolerance</code>	Coupled residual tolerance for JFNK/diagonal branches.
<code>nonlinearVmTolerance</code>	Relative L_2 tolerance for the voltage update.
<code>nonlinearStatesTolerance</code>	Relative L_2 tolerance for state updates.
<code>nonlinearRequireStatesConvergence</code>	If true, state residuals are part of the JFNK/diagonal stopping test.
<code>nonlinearAcceptUnconverged</code>	If false, JFNK/diagonal roll back on nonlinear failure.
<code>nonlinearIionTolerance</code>	Relative L_2 tolerance for I_{ion} changes.
<code>nonlinearRelaxation</code>	Picard relaxation or initial Newton step length.
<code>jfnkLinearSolverBackend</code>	PETSc (default) for the persistent shell-Krylov solver or Eigen for the in-file GMRES implementations.
<code>jfnkMaxKrylovIterations</code>	GMRES restart length m (also the Krylov budget for the matrix-free solve).
<code>jfnkMaxRestarts</code>	Number of restarted GMRES cycles.
<code>jfnkLinearTolerance</code>	GMRES relative residual tolerance.
<code>jfnkPetscKspType,</code> <code>jfnkPetscPcType,</code> <code>jfnkPetscRestart,</code> <code>jfnkPetscOptionsPrefix</code> <code>jfnkEpsilon</code>	PETSc KSP/PC settings and options-database prefix used by the persistent shell-Krylov solver.
<code>jfnkInitGuessOrder</code>	Scaling of the finite-difference Jacobian-vector perturbation.
<code>jfnkInitGuessVmMin/Max</code>	Constant or linear extrapolation initial guess.
<code>jfnkClampODEInput</code>	Clamp for extrapolated guess and optionally for ODE input.
<code>jfnkLineSearch,</code> <code>jfnkLineSearchMaxIter,</code> <code>jfnkLineSearchAlphaMin,</code> <code>jfnkArmijoC</code> <code>jfnkPreconditioner</code>	If true, clamps voltage passed to state ODE evaluations. Armijo backtracking controls.
<code>jfnkPreconditionerKspType,</code> <code>jfnkPreconditionerPcType</code>	Left preconditioner for the JFNK Krylov solve: <code>none</code> , <code>diffusion</code> (alias <code>linear</code> , <code>AImplicit</code>) or <code>diagonalIion</code> (alias <code>diagonal</code> , <code>localDiagonal</code>).
<code>jfnkPreconditionerDropTolerance,</code> <code>jfnkPreconditionerFillFactor</code>	KSP and PC types for the preconditioner solve, typically <code>preonly+ilu</code> to apply the cached ILU(T) factor directly.
<code>jfnkPreconditionerTolerance,</code> <code>jfnkPreconditionerMaxIterations</code>	PETSc ILU(T) drop tolerance and fill factor for the JFNK preconditioner factorisation.
<code>jfnkPreconditionerUpdateFrequency</code>	Inner KSP tolerance and iteration cap for the preconditioner solve (used by iterative inner solvers; the default <code>preonly</code> ignores them).
<code>jfnkPreconditionerOptionsPrefix</code>	How often the preconditioner matrix and factor are refreshed (every n Newton iterations) and the PETSc options-database prefix.
<code>diagonalIionEpsilon</code>	Finite-difference perturbation for diagonal source derivative.
<code>stateODESolver</code>	RKF45, RKT45, RK4, or Euler.

Control	Meaning
stateODEInitialStep, stateODEAbsTol, stateODERelTol, stateODEMaxSteps stateODEUseOpenMP, stateODENumThreads, stateODEOpenMPThreshold	Adaptive ODE step and tolerance controls. Enables OpenMP parallelisation of the per-cell/per-quadrature-point state ODE integration, sets the thread count and the cell threshold above which the parallel region is entered.
memoryOptimization	off, on, or auto. In auto, memory optimisations are effective only for 3D high-order V_m runs with at least <code>memoryOptimizationCellThreshold</code> cells.
memoryOptimizationCellThreshold	Cell-count threshold used by <code>memoryOptimization=auto</code> ; default 8000.
memoryOptimizationAdaptiveTripletsPerFace	Effective, selects a smaller or stabilised stiffness triplet reserve instead of the conservative 800 triplets per face.
memoryOptimizationFluxTripletsPerFace	Triplets-per-face reserve used for high-order stiffness assembly on fluxes only and stabilised cases. Values are clamped to at least 16.
memoryOptimizationStabilisedTripletsPerFace	Triplets-per-face reserve used for high-order stiffness assembly on fluxes only and stabilised cases. Values are clamped to at least 16.
memoryOptimizationTrimHeap	Enables gated <code>malloc_trim(0)</code> calls after large setup phases when memory optimisation is effective.
memoryOptimizationCompactMassAssembly	Enables compact row-wise triplet emission for the consistent high-order mass matrix when memory optimisation is effective.
profileTimings	Records per-phase wall-clock timings (assembly, GMRES, preconditioner setup/apply, sparse solves) for the final summary.
writeNonlinearResiduals	Writes per-iteration nonlinear residual diagnostics.

9 Output diagnostics

At the final time, the solver writes cell-centred fields, manufactured exact fields, error fields, reconstructed/quadrature diagnostics when available, and a text summary. The summary includes mesh size, dimension, Δt , time scheme, mass matrix, high-order settings for V_m and I_{ion} , selected nonlinear method, relevant tolerances, ODE controls, nonlinear convergence statistics, effective memory-optimisation settings, allocated ionic quadrature-point storage, wall-clock timings, and peak resident memory. These diagnostics are intended to make each convergence point auditable: the numerical method used to generate a data point is recorded alongside the error data.

10 Error Measures and Convergence Rates

The solver writes cell-centred volume-weighted L_1 , L_2 , and L_∞ errors for V_m , u_1 , and u_2 . For a generic field $q \in \{V_m, u_1, u_2\}$, let q_i be the numerical value stored at the centre of cell K_i , $q^*(\mathbf{x}, t)$ the manufactured exact solution, and

$$e_i(q) = q_i - q^*(\mathbf{C}_i, t), \quad |\Omega_h| = \sum_i |K_i|. \quad (45)$$

The cell-centred errors written by the solver are

$$E_1^C(q) = \frac{1}{|\Omega_h|} \sum_i |K_i| |e_i(q)|, \quad (46)$$

$$E_2^C(q) = \left[\frac{1}{|\Omega_h|} \sum_i |K_i| e_i(q)^2 \right]^{1/2}, \quad (47)$$

$$E_\infty^C(q) = \max_i |e_i(q)|. \quad (48)$$

These are volume-weighted mean/RMS errors, not unnormalised integral norms.

The solver also writes reconstructed quadrature errors, denoted by the superscript G . Let $\mathbf{x}_{iq}$ be a quadrature point in cell K_i and let ω_{iq} be its normalised quadrature weight, so that $m_{iq} = |K_i| \omega_{iq}$ is the physical measure associated with that point. For voltage, q_{iq}^G is the LRE/Taylor reconstructed voltage $V_{m,h}(\mathbf{x}_{iq})$; for the states, when high-order ionic integration is active, q_{iq}^G is the state advanced directly at the ionic quadrature point. The quadrature-point error is

$$e_{iq}^G(q) = q_{iq}^G - q^*(\mathbf{x}_{iq}, t), \quad |\Omega_h|_G = \sum_i \sum_q m_{iq}. \quad (49)$$

The corresponding quadrature errors are

$$E_1^G(q) = \frac{1}{|\Omega_h|_G} \sum_i \sum_q m_{iq} |e_{iq}^G(q)|, \quad (50)$$

$$E_2^G(q) = \left[\frac{1}{|\Omega_h|_G} \sum_i \sum_q m_{iq} (e_{iq}^G(q))^2 \right]^{1/2}, \quad (51)$$

$$E_\infty^G(q) = \max_{i,q} |e_{iq}^G(q)|. \quad (52)$$

If the corresponding high-order diagnostic is inactive, the report maps the cell-centred errors into the same output columns.

For the quadrature diagnostics, the exact-solution RMS norm and the relative percentage error are

$$\|q^*\|_{2,G} = \left[\frac{1}{|\Omega_h|_G} \sum_i \sum_q m_{iq} (q^*(\mathbf{x}_{iq}, t))^2 \right]^{1/2}, \quad E_{\text{rel},2}^G(q) = 100 \frac{E_2^G(q)}{\max(\|q^*\|_{2,G}, \text{SMALL})}. \quad (53)$$

Pairwise rates use consecutive mesh sizes,

$$p_{N_1, N_2} = \frac{\log(E_{N_1}/E_{N_2})}{\log(h_{N_1}/h_{N_2})}, \quad h_N = 1/N. \quad (54)$$

The convergence tables below recompute log-log fits directly from the `errors\_vs\_N` files, using both the full $N = 10$ – 100 mesh range and the finer $N = 70$ – 100 range.

11 Results

This section collects the current convergence results, tables, and figure references. The 2D data are generated from `/home/pablo/OpenFOAM/pablo-v2312/run/tutorials_electro/highOrderManufacturedFDAImplicit/results/example0/2D_run_20260522_034403/`, and the 3D data from `/home/pablo/OpenFOAM/pablo-v2312/run/tutorials_electro/highOrderManufacturedFDAImplicitPETSc/results/example0/3D_run_20260522_235518/`. All result blocks in this section use `crankNicolson` time integration and the `consistent` mass matrix, and are split by nonlinear method, mesh family, stabilisation parameter, and time step. The 2D orders are fitted over $N = 10$ – 100 and $N = 70$ – 100 ; the 3D orders are fitted over $N = 10$ – 30 and $N = 20$ – 30 . The figures can be regenerated with `plot_convergence_2D.m` and `plot_convergence_3D.m`; missing figure boxes indicate plots that have not yet been generated from the latest data.

11.1 Two-dimensional manufactured problem

The 2D tables and figures in this subsection are generated from `/home/pablo/OpenFOAM/pablo-v2312/run/tutorials_electro/highOrderManufacturedFDImplicitPETSc/results/example0/2D_run_20260522_034403/`. All entries use `crankNicolson` time integration and the consistent mass matrix. Figures are split by nonlinear method, mesh family, stabilisation parameter, and time step; the model pair is also written directly on the curves.

11.1.1 Picard, crankNicolson, consistent

Table 2: 2D manufactured results for Picard, crankNicolson, and the consistent mass matrix. The first six convergence columns are log-log fits over meshes $N = 10$ –100; the next six columns are fits over $N = 70$ –100. The time column is the sum of the reported wall-clock total over all mesh runs in the row, and the memory column is the sum of the reported peak resident memory in MB over the same mesh runs.

$V_m - I_{\text{ion}}$	Mesh type	α	Δt [s]	Fit $N = 10$ –100						Fit $N = 70$ –100						t_{Σ} [s]	RSS $_{\Sigma}$ [MB]
				V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G		
NO–NO	hexa	0.0	10^{-2}	1.992	1.992	2.135	2.135	2.134	2.134	1.985	1.985	2.500	2.500	2.497	2.497	5.143	16406.2
			10^{-3}	1.997	1.997	1.998	1.998	1.998	1.998	2.000	2.000	2.004	2.004	2.004	2.004	37.411	16434.0
		0.1	10^{-2}	1.849	1.849	1.612	1.612	1.618	1.618	2.278	2.278	2.159	2.159	2.163	2.163	18.689	18355.6
			10^{-3}	1.851	1.851	1.607	1.607	1.613	1.613	2.294	2.294	2.140	2.140	2.142	2.142	60.356	18355.3
	triangular unstr	0.0	10^{-2}	0.730	0.730	0.778	0.778	0.784	0.784	0.862	0.862	1.037	1.037	1.107	1.107	66.651	17158.0
			10^{-3}	0.798	0.798	0.777	0.777	0.783	0.783	0.963	0.963	1.038	1.038	1.107	1.107	356.55	17094.1
NO–p1	hexa	0.0	10^{-2}	1.992	1.992	2.000	2.057	2.005	2.061	1.985	1.985	2.002	2.031	2.023	2.031	6.823	16540.8
			10^{-3}	1.997	1.997	1.999	2.050	1.998	2.053	2.000	2.000	2.000	2.007	2.000	2.007	50.315	16582.5
		0.1	10^{-2}	1.849	1.849	2.000	2.049	1.997	2.053	2.278	2.278	2.002	2.039	2.030	2.039	21.520	18420.0
			10^{-3}	1.851	1.851	1.999	2.042	1.990	2.046	2.294	2.294	2.000	2.015	2.008	2.016	73.314	18444.8
	triangular unstr	0.0	10^{-2}	0.730	0.730	0.778	0.778	0.784	0.784	0.862	0.862	1.037	1.037	1.107	1.107	72.359	17303.3
			10^{-3}	0.798	0.798	0.777	0.777	0.783	0.783	0.963	0.963	1.038	1.038	1.107	1.107	405.08	17260.0
NO–p2	hexa	0.0	10^{-2}	1.992	1.992	1.999	2.015	2.002	2.017	1.986	1.986	2.001	2.010	2.011	2.010	14.302	16794.5
			10^{-3}	1.997	1.997	1.999	2.013	1.998	2.015	1.999	1.999	2.000	2.002	2.000	2.002	110.15	16824.2
		0.1	10^{-2}	1.851	1.851	1.999	2.015	2.001	2.017	2.274	2.274	2.001	2.011	2.013	2.011	26.791	18563.5
			10^{-3}	1.853	1.853	1.999	2.013	1.998	2.014	2.290	2.290	2.000	2.003	2.001	2.003	107.80	18552.9
	triangular unstr	0.0	10^{-2}	0.730	0.730	1.878	1.418	1.102	1.411	0.862	0.862	1.678	1.219	1.112	1.269	71.948	17559.7
			10^{-3}	0.799	0.799	1.878	1.417	1.101	1.410	0.963	0.963	1.677	1.218	1.112	1.269	425.12	17491.6
NO–p3	hexa	0.0	10^{-2}	1.992	1.992	1.999	2.019	2.002	2.021	1.986	1.986	2.001	2.011	2.011	2.011	34.842	16938.5
			10^{-3}	1.997	1.997	1.999	2.016	1.999	2.018	1.999	1.999	2.000	2.002	2.000	2.002	113.09	16926.6
		0.1	10^{-2}	1.851	1.851	1.999	2.019	2.001	2.021	2.274	2.274	2.001	2.013	2.013	2.013	36.836	18613.1
			10^{-3}	1.853	1.853	1.999	2.016	1.998	2.018	2.290	2.290	2.000	2.004	2.002	2.004	139.45	18620.6
	triangular unstr	0.0	10^{-2}	0.730	0.730	1.878	1.389	1.102	1.384	0.862	0.862	1.678	1.206	1.112	1.258	78.186	17682.8
			10^{-3}	0.799	0.799	1.878	1.388	1.101	1.383	0.963	0.963	1.677	1.205	1.112	1.258	469.91	17633.3
p1–NO	hexa	0.0	10^{-2}	1.911	2.146	1.616	1.616	1.626	1.626	2.612	2.168	2.381	2.381	2.387	2.387	116.43	17750.5
			10^{-3}	1.912	2.147	1.616	1.616	1.626	1.626	2.611	2.168	2.380	2.380	2.385	2.385	119.56	17760.4
		0.1	10^{-2}	1.971	2.163	1.677	1.677	1.687	1.687	2.614	2.141	2.416	2.416	2.422	2.422	93.581	19067.1
			10^{-3}	1.971	2.163	1.677	1.677	1.687	1.687	2.615	2.141	2.414	2.414	2.419	2.419	138.64	19053.5
	triangular unstr	0.0	10^{-2}	2.271	2.126	2.004	2.004	2.002	2.002	2.384	2.115	2.552	2.552	2.549	2.549	566.18	19578.9
			10^{-3}	2.292	2.131	2.004	2.004	2.003	2.003	2.600	2.164	2.545	2.545	2.542	2.542	1039.5	19596.0
p1–p1	hexa	0.0	10^{-2}	2.301	2.106	2.092	2.092	2.090	2.090	2.222	2.054	2.524	2.524	2.524	2.524	636.61	21518.4
			10^{-3}	2.345	2.112	2.092	2.092	2.089	2.089	2.552	2.095	2.516	2.516	2.515	2.515	884.21	21530.4
		0.1	10^{-2}	1.911	2.146	1.998	2.154	1.838	2.160	2.612	2.168	2.007	2.316	2.258	2.316	81.360	17778.0
			10^{-3}	1.912	2.147	1.997	2.152	1.836	2.158	2.611	2.168	2.005	2.309	2.249	2.308	123.49	17807.9
	triangular unstr	0.0	10^{-2}	1.971	2.163	1.999	2.221	1.898	2.227	2.614	2.141	2.005	2.319	2.234	2.318	86.674	19080.5
			10^{-3}	1.971	2.163	1.999	2.219	1.895	2.225	2.615	2.141	2.003	2.309	2.222	2.308	138.59	19090.8
p1–p2	hexa	0.0	10^{-2}	2.271	2.126	2.004	2.004	2.002	2.002	2.384	2.115	2.552	2.552	2.549	2.549	713.46	19611.5
			10^{-3}	2.292	2.131	2.004	2.004	2.003	2.003	2.600	2.164	2.545	2.545	2.542	2.542	1204.8	19634.4
		0.1	10^{-2}	2.301	2.106	2.092	2.092	2.090	2.090	2.222	2.054	2.524	2.524	2.524	2.524	644.65	21555.8
			10^{-3}	2.345	2.112	2.092	2.092	2.089	2.089	2.552	2.095	2.516	2.516	2.515	2.515	1079.4	21558.4
	triangular unstr	0.0	10^{-2}	1.912	2.146	1.999	2.182	1.951	2.181	2.612	2.168	2.002	2.161	2.105	2.157	87.844	17999.5
			10^{-3}	1.913	2.147	1.998	2.181	1.949	2.179	2.611	2.168	2.001	2.155	2.096	2.151	180.40	18008.2
p1–p3	hexa	0.0	10^{-2}	1.971	2.163	1.999	2.201	1.975	2.199	2.613	2.141	2.001	2.156	2.080	2.151	94.921	19190.6
			10^{-3}	1.972	2.163	1.999	2.199	1.972	2.197	2.615	2.141	2.001	2.150	2.071	2.145	209.63	19212.9
		0.1	10^{-2}	2.271	2.126	1.992	2.069	2.012	2.068	2.384	2.115	2.004	2.184	2.284	2.176	648.18	19866.6
			10^{-3}	2.292	2.131	1.991	2.067	2.009	2.066	2.600	2.164	2.002	2.176	2.268	2.168	849.42	19867.6
	triangular unstr	0.0	10^{-2}	2.301	2.106	1.992	2.089	2.038	2.087	2.222	2.054	2.002	2.117	2.186	2.113	717.87	21700.1
			10^{-3}	2.345	2.112	1.992	2.087	2.034	2.085	2.552	2.095	2.000	2.109	2.167	2.105	1197.1	21695.8
p1–p3	hexa	0.0	10^{-2}	1.912	2.146	1.999	2.196	1.951	2.195	2.612	2.168	2.002	2.177	2.105	2.173	90.087	18115.0
			10^{-3}	1.913	2.147	1.999	2.194	1.949	2.193	2.611	2.168	2.001	2.170	2.096	2.166	249.46	18130.7
		0.1	10^{-2}	1.971	2.163	1.999	2.217	1.975	2.215	2.613	2.141	2.001	2.172	2.080	2.167	102.92	19245.1
			10^{-3}	1.972	2.163	1.999	2.215	1.973	2.213	2.615	2.141	2.001	2.165	2.071	2.160	247.86	19286.1

$V_m - I_{\text{ion}}$	Mesh type	α	Δt [s]	Fit $N = 10-100$						Fit $N = 70-100$						t_Σ [s]	RSS $_\Sigma$ [MB]	
				V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G			
p2-NO	triangular unstr	0.0	10^{-2}	2.271	2.126	1.992	2.082	2.012	2.081	2.384	2.115	2.004	2.219	2.284	2.210	575.95	19986.2	
			10^{-3}	2.292	2.131	1.992	2.080	2.009	2.079	2.600	2.164	2.002	2.209	2.268	2.200	895.96	19987.1	
		0.1	10^{-2}	2.301	2.106	1.992	2.109	2.038	2.106	2.222	2.054	2.002	2.144	2.186	2.139	677.12	21763.5	
	hexa	0.0	10^{-2}	2.345	2.112	1.992	2.106	2.034	2.104	2.552	2.095	2.000	2.134	2.167	2.129	1063.9	21763.2	
			10^{-3}	2.127	2.567	2.176	2.176	2.176	2.176	2.016	2.179	2.139	2.139	2.139	2.139	85.517	18507.1	
		0.1	10^{-2}	2.128	2.568	2.142	2.142	2.143	2.143	2.020	2.183	2.025	2.025	2.026	2.026	159.22	18524.9	
	triangular unstr	0.0	10^{-2}	2.128	2.607	2.180	2.180	2.181	2.181	2.018	2.209	2.153	2.153	2.153	2.153	87.941	20275.2	
			10^{-3}	2.129	2.608	2.143	2.143	2.144	2.144	2.022	2.213	2.027	2.027	2.028	2.028	156.57	20279.8	
		0.1	10^{-2}	2.056	2.374	2.166	2.166	2.162	2.162	1.982	2.068	2.261	2.261	2.263	2.263	593.35	21544.9	
	p2-p1	triangular unstr	0.0	10^{-2}	2.058	2.376	2.090	2.090	2.086	2.086	1.991	2.077	1.998	1.998	2.002	2.002	832.28	21545.1
				10^{-3}	2.043	2.402	2.162	2.162	2.158	2.158	1.980	2.081	2.286	2.286	2.285	2.285	591.36	24379.3
			0.1	10^{-2}	2.045	2.405	2.079	2.079	2.075	2.075	1.989	2.091	1.996	1.996	1.997	1.997	894.20	24372.9
hexa		0.0	10^{-2}	2.149	2.608	2.004	2.716	2.109	2.720	2.019	2.210	2.002	2.416	2.034	2.422	113.49	18606.1	
			10^{-3}	2.150	2.609	2.003	2.689	2.102	2.694	2.023	2.214	2.001	2.304	2.011	2.312	136.92	18575.6	
		0.1	10^{-2}	2.152	2.651	2.004	2.757	2.107	2.760	2.021	2.247	2.002	2.470	2.033	2.476	95.066	20320.7	
triangular unstr		0.0	10^{-2}	2.153	2.653	2.003	2.728	2.100	2.732	2.026	2.251	2.001	2.350	2.011	2.358	160.65	20323.0	
			10^{-3}	2.056	2.374	2.166	2.166	2.162	2.162	1.982	2.068	2.261	2.261	2.263	2.263	613.49	21589.3	
		0.1	10^{-2}	2.058	2.376	2.090	2.090	2.086	2.086	1.991	2.077	1.998	1.998	2.002	2.002	913.79	21572.9	
p2-p2		hexa	0.0	10^{-2}	2.043	2.402	2.162	2.162	2.158	2.158	1.980	2.081	2.286	2.286	2.285	2.285	601.41	24401.3
				10^{-3}	2.045	2.405	2.079	2.079	2.075	2.075	1.989	2.091	1.996	1.996	1.997	1.997	933.55	24422.7
			0.1	10^{-2}	2.212	2.708	2.004	2.942	2.118	2.941	2.026	2.303	2.001	2.735	2.020	2.738	101.01	18797.0
	triangular unstr	0.0	10^{-2}	2.214	2.709	2.003	2.924	2.114	2.923	2.032	2.308	2.000	2.644	2.007	2.650	182.72	18784.6	
			10^{-3}	2.224	2.760	2.004	2.969	2.116	2.968	2.030	2.365	2.001	2.794	2.019	2.798	97.950	20436.8	
		0.1	10^{-2}	2.226	2.761	2.003	2.952	2.112	2.951	2.037	2.371	2.000	2.706	2.007	2.711	232.36	20423.3	
	hexa	0.0	10^{-2}	2.118	2.532	1.993	2.860	2.044	2.862	1.977	2.149	2.000	2.751	2.026	2.758	607.86	21808.2	
			10^{-3}	2.122	2.536	1.992	2.805	2.037	2.807	1.990	2.162	1.998	2.489	2.003	2.499	912.84	21817.9	
		0.1	10^{-2}	2.104	2.579	1.993	2.895	2.042	2.896	1.972	2.186	2.000	2.820	2.025	2.823	595.60	24545.1	
	p2-p3	triangular unstr	0.0	10^{-2}	2.108	2.583	1.992	2.840	2.035	2.842	1.987	2.200	1.998	2.553	2.002	2.561	1016.7	24550.7
				10^{-3}	2.212	2.708	2.004	2.940	2.118	2.940	2.026	2.303	2.001	2.739	2.020	2.743	115.81	18890.9
			0.1	10^{-2}	2.214	2.709	2.004	2.923	2.114	2.922	2.032	2.308	2.000	2.650	2.007	2.656	213.98	18882.5
hexa		0.0	10^{-2}	2.224	2.760	2.004	2.967	2.117	2.966	2.030	2.365	2.001	2.798	2.019	2.801	105.75	20496.0	
			10^{-3}	2.226	2.761	2.004	2.950	2.113	2.950	2.037	2.371	2.000	2.711	2.007	2.716	266.03	20500.2	
		0.1	10^{-2}	2.118	2.532	1.993	2.863	2.044	2.865	1.977	2.149	2.000	2.758	2.026	2.764	600.35	21930.4	
p3-NO		triangular unstr	0.0	10^{-2}	2.122	2.536	1.993	2.809	2.037	2.812	1.990	2.162	1.998	2.500	2.003	2.510	938.35	21942.5
				10^{-3}	2.104	2.579	1.993	2.897	2.042	2.898	1.972	2.186	2.000	2.825	2.025	2.829	571.33	24606.0
			0.1	10^{-2}	2.108	2.583	1.993	2.843	2.035	2.846	1.987	2.200	1.998	2.564	2.002	2.572	1037.7	24620.3
		hexa	0.0	10^{-2}	2.121	2.481	2.256	2.256	2.258	2.258	1.999	2.037	2.472	2.472	2.469	2.469	139.64	21801.9
				10^{-3}	2.125	2.485	2.132	2.132	2.135	2.135	2.013	2.050	2.025	2.025	2.024	2.024	173.20	21790.9
			0.1	10^{-2}	2.104	2.476	2.238	2.238	2.240	2.240	1.996	2.034	2.468	2.468	2.465	2.465	131.88	24501.4
	triangular unstr	0.0	10^{-2}	2.108	2.481	2.114	2.114	2.117	2.117	2.010	2.048	2.020	2.020	2.020	2.020	164.48	24491.5	
			10^{-3}	2.033	2.201	2.348	2.348	2.346	2.346	1.972	1.988	3.183	3.183	3.173	3.173	702.42	27521.8	
		0.1	10^{-2}	2.041	2.209	2.061	2.061	2.060	2.060	1.999	2.015	2.008	2.008	2.008	2.008	964.35	27515.3	
	p3-p1	hexa	0.0	10^{-2}	2.021	2.196	2.333	2.333	2.331	2.331	1.972	1.987	3.183	3.183	3.173	3.173	668.22	31503.9
				10^{-3}	2.029	2.204	2.046	2.046	2.046	2.046	1.999	2.014	2.008	2.008	2.007	2.007	994.75	31523.2
			0.1	10^{-2}	2.181	2.631	2.002	2.762	1.992	2.758	2.003	2.056	2.002	2.661	2.008	2.651	139.74	21832.7
triangular unstr		0.0	10^{-2}	2.187	2.637	2.001	2.598	1.989	2.596	2.020	2.073	2.000	2.068	1.999	2.066	266.14	21856.4	
			10^{-3}	2.156	2.626	2.002	2.756	1.991	2.752	1.998	2.051	2.002	2.653	2.008	2.642	137.21	24543.6	
		0.1	10^{-2}	2.161	2.631	2.001	2.592	1.988	2.589	2.016	2.069	2.000	2.060	1.999	2.059	237.47	24532.6	
p3-p2		hexa	0.0	10^{-2}	2.033	2.201	2.348	2.348	2.346	2.346	1.972	1.988	3.183	3.183	3.173	3.173	723.30	27552.4
				10^{-3}	2.041	2.209	2.061	2.061	2.060	2.060	1.999	2.015	2.008	2.008	2.008	2.008	1040.7	27557.2
			0.1	10^{-2}	2.021	2.196	2.333	2.333	2.331	2.331	1.972	1.987	3.183	3.183	3.173	3.173	697.04	31547.6
		triangular unstr	0.0	10^{-2}	2.029	2.204	2.046	2.046	2.046	2.046	1.999	2.014	2.008	2.008	2.007	2.007	1082.2	31544.0
				10^{-2}	3.520	3.971	2.000	3.744	1.999	3.748	4.171	4.124	2.001	1.457	2.006	1.395	127.20	21964.0
			0.1	10^{-2}	3.538	3.992	1.999	4.003	1.997	4.010	4.323	4.279	2.000	4.194	2.001	4.195	256.36	21952.8
	hexa	0.0	10^{-2}	3.547	4.002	1.999	3.757	1.998	3.761	4.105	4.100	2.001	1.347	2.006	1.283	134.32	24618.0	
			10^{-3}	3.569	4.026	1.999	4.033	1.997	4.040	4.299	4.271	2.000	4.212	2.000	4.213	293.86	24648.2	
		0.1	10^{-2}	3.749	3.924	1.992	2.844	2.005	2.843	2.166	2.959	2.000	-0.092	2.012	-0.097	721.62	27688.9	
	p3-p3	triangular unstr	0.0	10^{-2}	4.057	4.096	1.992	4.077	2.002	4.083	4.476	4.165	1.998	4.204	2.003	4.205	1039.0	27685.6
				10^{-3}	3.693	3.924	1.992	2.836	2.005	2.836	1.803	2.888	2.000	-0.098	2.012	-0.103	725.45	31682.7
			0.1	10^{-2}	4.077	4.105	1.992	4.094	2.001	4.098	4.460	4.139	1.998	4.195	2.003	4.198	1148.6	31684.3
hexa		0.0	10^{-2}	3.520	3.971	2.000	3.738	1.999	3.742	4.171	4.124	2.001	1.419	2.006	1.354	135.58	22012.0	
			10^{-3}	3.538	3.992	1.999	4.000	1.998	4.008	4.323	4.279	2.000	4.198	2.001	4.200	311.89	22022.3	
		0.1	10^{-2}	3.547	4.002	2.000	3.751	1.999	3.755	4.105	4.100	2.001	1.304	2.006	1.237	142.03	24707.6	
triangular unstr		0.0	10^{-2}	3.569	4.026	1.999	4.031	1.997	4.039	4.299	4.271	2.000	4.217	2.001	4.219	334.26	24706.1	
			10^{-2}	3.749	3.924	1.992	2.812	2.005	2.811	2.166	2.959	2.000	-0.108	2.012	-0.114	732.26	27747.5	
		0.1	10^{-2}	4.057	4.096	1.992	4.079	2.002	4.086	4.476	4.165	1.998	4.227	2.003	4.229	1166.7	27749.1	
p3-NO		triangular unstr	0.0	10^{-2}	3.693	3.924	1.992	2.803	2.005	2.803	1.803	2.888	2.000	-0.115	2.012	-0.120	691.27	31746.0
				10^{-3}	4.077	4.105	1.992	4.098	2.002	4.103	4.460	4.139	1.998	4.219	2.003	4.223	1198.0	31741.2

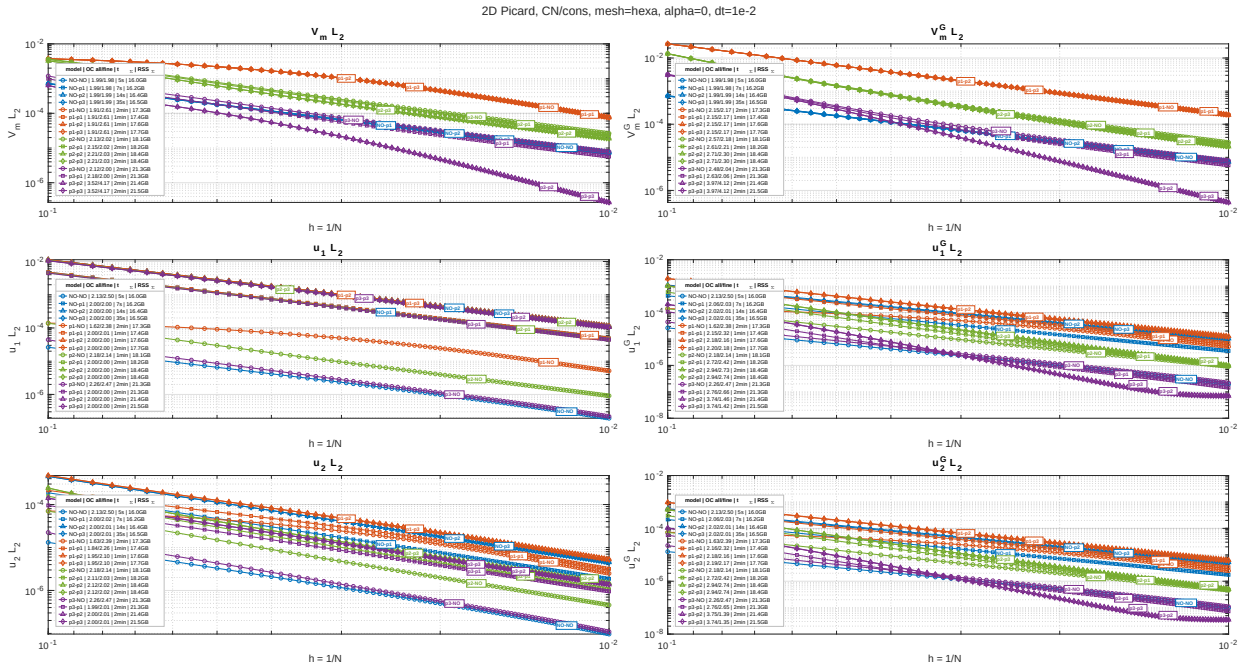


Figure 1: 2D convergence block for Picard, hexa mesh, $\alpha = 0$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m – I_{ion} model pair.

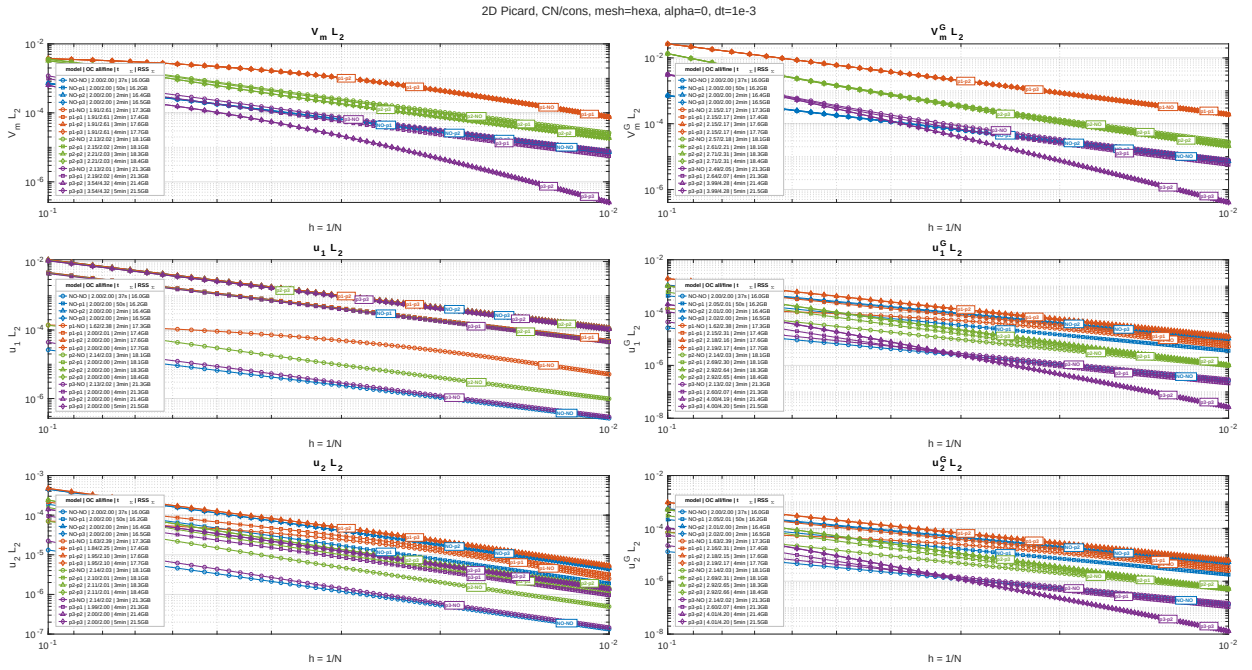


Figure 2: 2D convergence block for Picard, hexa mesh, $\alpha = 0$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the $V_m - I_{\text{ion}}$ model pair.

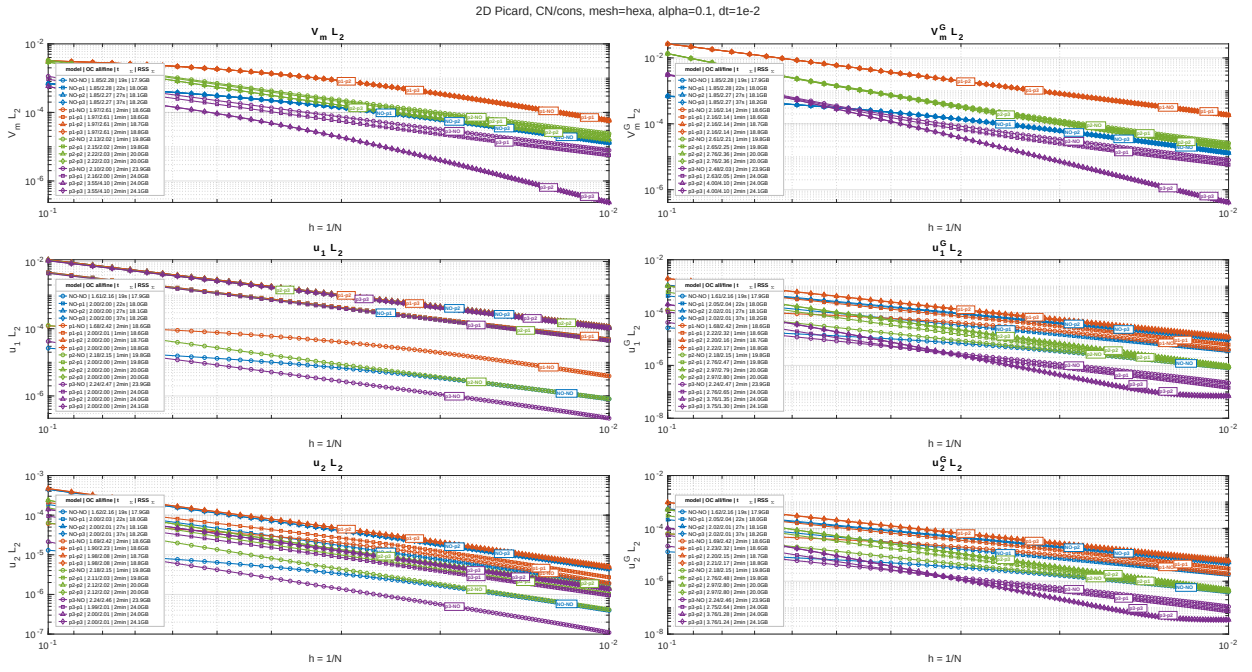


Figure 3: 2D convergence block for Picard, hexa mesh, $\alpha = 0.1$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the $V_m - I_{\text{ion}}$ model pair.

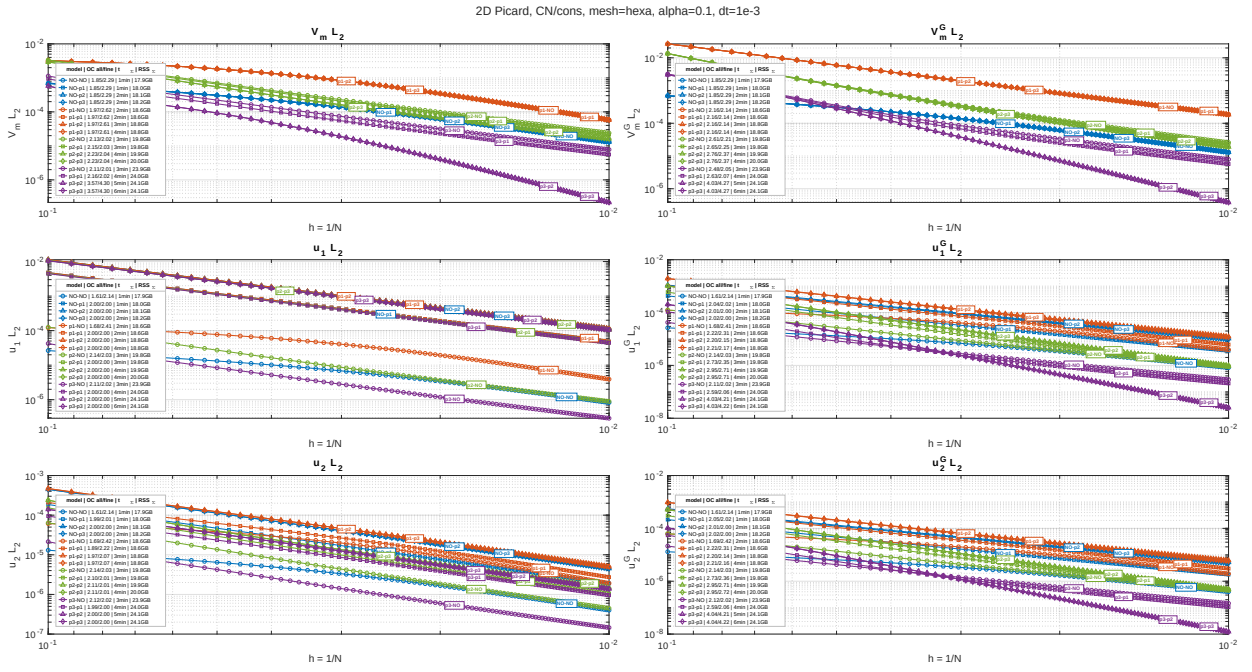


Figure 4: 2D convergence block for Picard, hexa mesh, $\alpha = 0.1$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the $V_m - I_{\text{ion}}$ model pair.

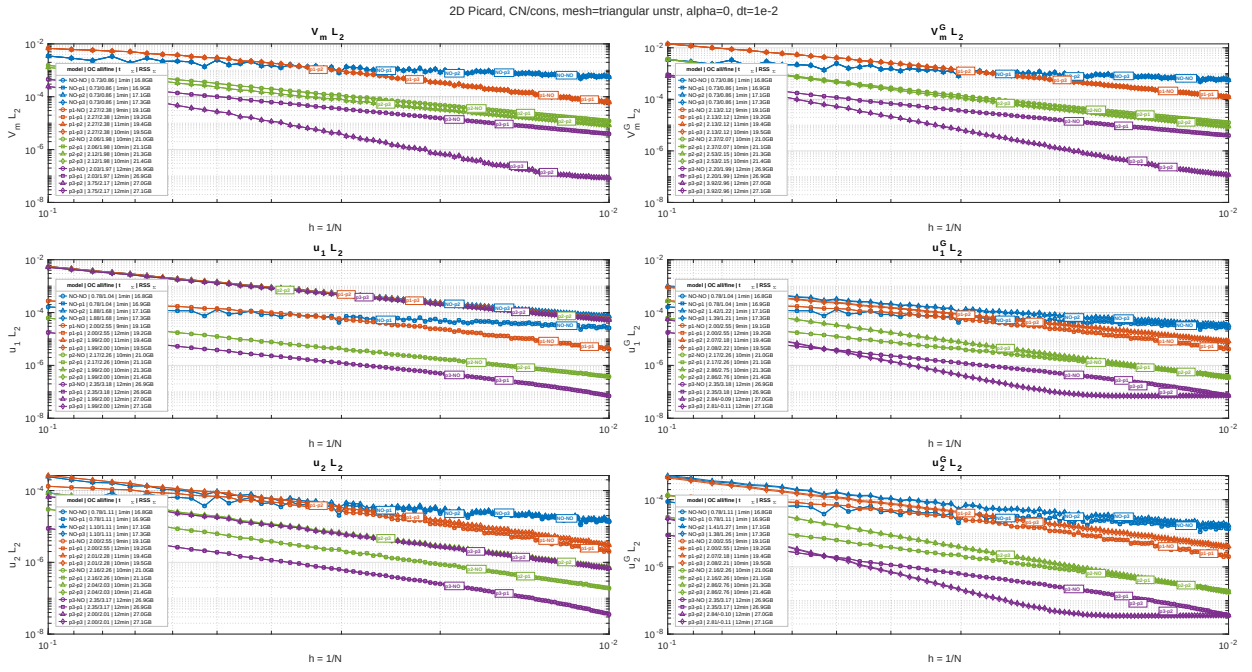
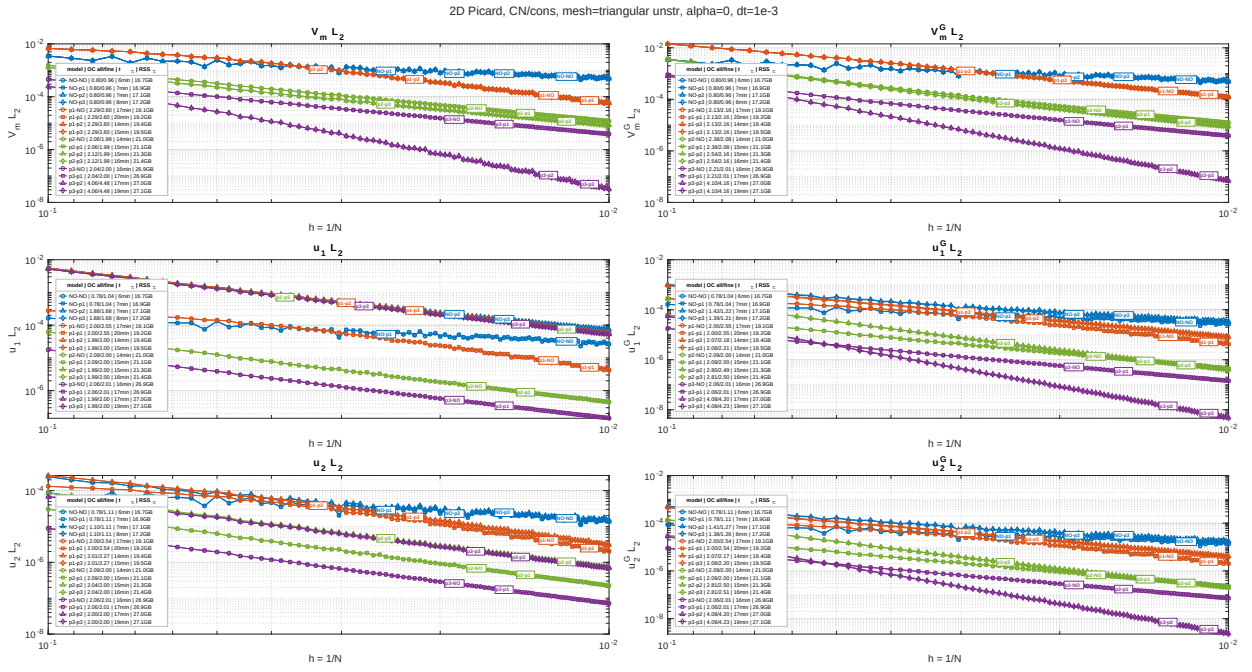
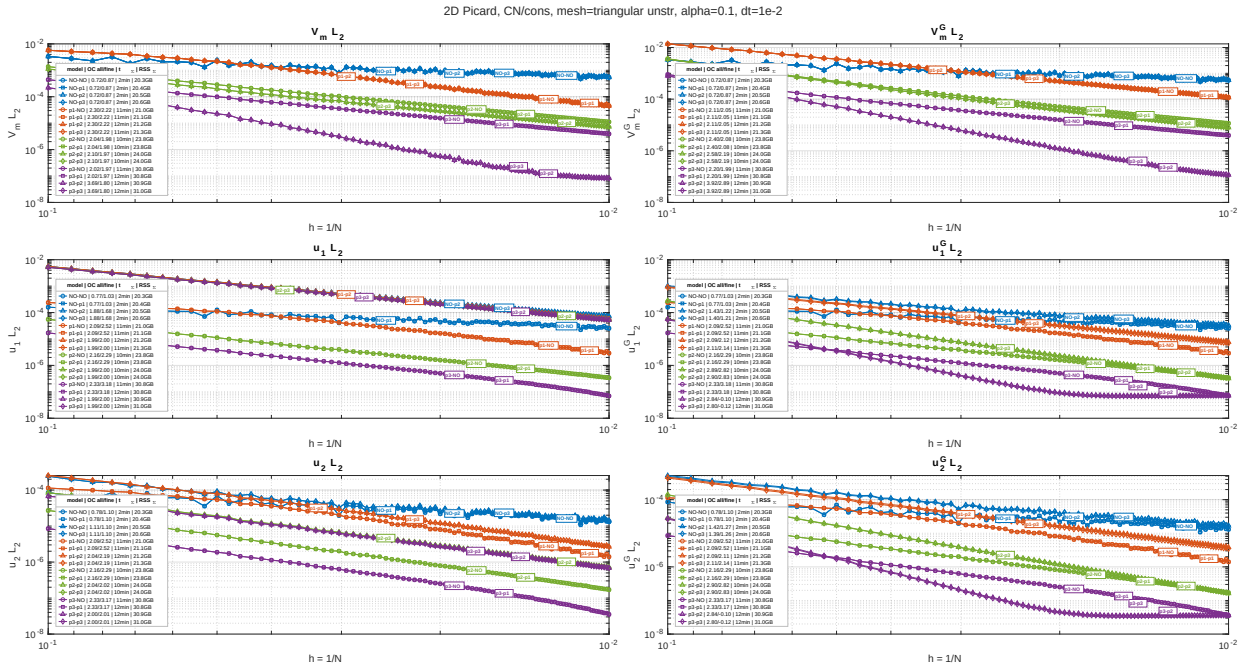


Figure 5: 2D convergence block for Picard, triangular unstr mesh, $\alpha = 0$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.





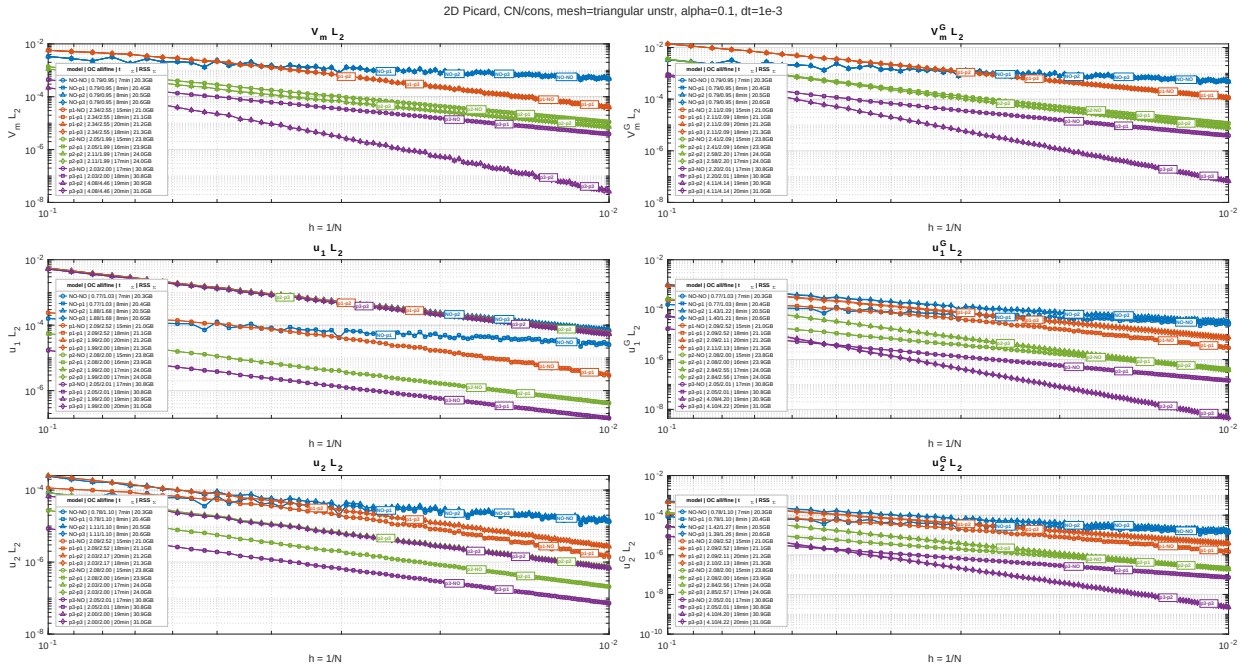


Figure 8: 2D convergence block for Picard, triangular unstr mesh, $\alpha = 0.1$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

11.1.2 diagonalIion, crankNicolson, consistent

Table 3: 2D manufactured results for `diagonalIion`, `crankNicolson`, and the `consistent` mass matrix. The first six convergence columns are log-log fits over meshes $N = 10$ – 100 ; the next six columns are fits over $N = 70$ – 100 . The time column is the sum of the reported wall-clock total over all mesh runs in the row, and the memory column is the sum of the reported peak resident memory in MB over the same mesh runs.

$V_m - I_{\text{ion}}$	Mesh type	α	Δt [s]	Fit $N = 10$ – 100						Fit $N = 70$ – 100						t_{Σ} [s]	RSS $_{\Sigma}$ [MB]
				V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G		
NO–NO	hexa	0.0	10^{-2}	1.992	1.992	2.135	2.135	2.134	2.134	1.985	1.985	2.500	2.500	2.497	2.497	10.461	16447.3
			10^{-3}	1.997	1.997	1.998	1.998	1.998	1.998	2.000	2.000	2.004	2.004	2.004	2.004	66.387	16461.7
		0.1	10^{-2}	1.849	1.849	1.612	1.612	1.618	1.618	2.278	2.278	2.159	2.159	2.163	2.163	34.701	18361.8
			10^{-3}	1.851	1.851	1.607	1.607	1.613	1.613	2.294	2.294	2.140	2.140	2.142	2.142	121.55	18342.1
	triangular unstr	0.0	10^{-2}	0.730	0.730	0.778	0.778	0.784	0.784	0.862	0.862	1.037	1.037	1.107	1.107	85.422	17192.2
			10^{-3}	0.798	0.798	0.777	0.777	0.783	0.783	0.963	0.963	1.038	1.038	1.107	1.107	375.50	17145.7
NO–p1	hexa	0.0	10^{-2}	1.992	1.992	2.000	2.057	2.005	2.061	1.985	1.985	2.002	2.031	2.023	2.031	13.699	16534.5
			10^{-3}	1.997	1.997	1.999	2.050	1.998	2.053	2.000	2.000	2.000	2.007	2.000	2.007	83.484	16618.9
		0.1	10^{-2}	1.849	1.849	2.000	2.049	1.997	2.053	2.278	2.278	2.002	2.039	2.030	2.039	45.175	18416.0
			10^{-3}	1.851	1.851	1.999	2.042	1.990	2.046	2.294	2.294	2.000	2.015	2.008	2.016	169.69	18446.5
	triangular unstr	0.0	10^{-2}	0.730	0.730	0.778	0.778	0.784	0.784	0.862	0.862	1.037	1.037	1.107	1.107	87.018	17338.7
			10^{-3}	0.798	0.798	0.777	0.777	0.783	0.783	0.963	0.963	1.038	1.038	1.107	1.107	499.64	17283.5
NO–p2	hexa	0.0	10^{-2}	1.992	1.992	1.999	2.015	2.002	2.017	1.986	1.986	2.001	2.010	2.011	2.010	28.704	16813.7
			10^{-3}	1.997	1.997	1.999	2.013	1.998	2.015	1.999	1.999	2.000	2.002	2.000	2.002	202.57	16847.9
		0.1	10^{-2}	1.851	1.851	1.999	2.015	2.001	2.017	2.274	2.274	2.001	2.011	2.013	2.011	52.057	18561.0
			10^{-3}	1.853	1.853	1.999	2.013	1.998	2.014	2.290	2.290	2.000	2.003	2.001	2.003	267.50	18578.8
	triangular unstr	0.0	10^{-2}	0.730	0.730	1.878	1.418	1.102	1.411	0.862	0.862	1.678	1.219	1.112	1.269	133.70	17598.5
			10^{-3}	0.799	0.799	1.878	1.417	1.101	1.410	0.963	0.963	1.677	1.218	1.112	1.269	347.95	17526.6
NO–p3	hexa	0.0	10^{-2}	1.992	1.992	1.999	2.015	2.002	2.017	1.986	1.986	2.001	2.010	2.011	2.010	47.744	16969.7
			10^{-3}	1.997	1.997	1.999	2.016	1.999	2.018	1.999	1.999	2.000	2.002	2.000	2.002	233.61	16957.7
		0.1	10^{-2}	1.851	1.851	1.999	2.016	1.998	2.018	2.290	2.290	2.000	2.013	2.013	2.013	50.120	18613.3
			10^{-3}	1.853	1.853	1.999	2.016	1.998	2.018	2.290	2.290	2.000	2.004	2.002	2.004	260.87	18624.0
	triangular unstr	0.0	10^{-2}	0.730	0.730	1.878	1.389	1.102	1.384	0.862	0.862	1.678	1.206	1.112	1.258	71.347	17710.6
			10^{-3}	0.799	0.799	1.878	1.388	1.101	1.383	0.963	0.963	1.677	1.205	1.112	1.258	405.61	17643.7
p1–NO	hexa	0.0	10^{-2}	1.992	1.992	1.999	2.019	2.002	2.021	1.986	1.986	2.001	2.011	2.011	2.011	47.744	16969.7
			10^{-3}	1.997	1.997	1.999	2.016	1.999	2.018	1.999	1.999	2.000	2.002	2.000	2.002	233.61	16957.7
		0.1	10^{-2}	1.851	1.851	1.999	2.016	1.998	2.018	2.290	2.290	2.000	2.013	2.013	2.013	50.120	18613.3
			10^{-3}	1.853	1.853	1.999	2.016	1.998	2.018	2.290	2.290	2.000	2.004	2.002	2.004	260.87	18624.0
	triangular unstr	0.0	10^{-2}	0.730	0.730	1.878	1.389	1.102	1.384	0.862	0.862	1.678	1.206	1.112	1.258	71.347	17710.6
			10^{-3}	0.799	0.799	1.878	1.388	1.101	1.383	0.963	0.963	1.677	1.205	1.112	1.258	405.61	17643.7
p1–p1	hexa	0.0	10^{-2}	1.911	2.146	1.616	1.616	1.626	1.626	2.612	2.168	2.381	2.381	2.387	2.387	80.953	17858.1
			10^{-3}	1.912	2.147	1.616	1.616	1.626	1.626	2.611	2.168	2.380	2.380	2.385	2.385	161.28	17839.1
		0.1	10^{-2}	1.971	2.163	1.677	1.677	1.687	1.687	2.614	2.141	2.416	2.416	2.422	2.422	94.616	19059.8
			10^{-3}	1.971	2.163	1.677	1.677	1.687	1.687	2.615	2.141	2.414	2.414	2.419	2.419	189.26	19067.4
	triangular unstr	0.0	10^{-2}	2.271	2.126	2.004	2.004	2.002	2.002	2.384	2.115	2.552	2.552	2.549	2.549	590.16	19872.9
			10^{-3}	2.292	2.131	2.004	2.004	2.003	2.003	2.600	2.164	2.545	2.545	2.542	2.542	773.26	19883.4
p1–p2	hexa	0.0	10^{-2}	2.301	2.106	2.092	2.092	2.090	2.090	2.222	2.054	2.524	2.524	2.524	2.524	559.07	21538.7
			10^{-3}	2.345	2.112	2.092	2.092	2.089	2.089	2.552	2.095	2.516	2.516	2.515	2.515	976.30	21554.6
		0.1	10^{-2}	1.911	2.146	1.998	2.154	1.838	2.160	2.612	2.168	2.007	2.316	2.258	2.316	85.742	17904.5
			10^{-3}	1.912	2.147	1.997	2.152	1.836	2.158	2.611	2.168	2.005	2.309	2.249	2.308	204.40	17929.4
	triangular unstr	0.0	10^{-2}	1.971	2.163	1.999	2.221	1.898	2.227	2.614	2.141	2.005	2.319	2.234	2.318	106.80	19111.6
			10^{-3}	1.971	2.163	1.999	2.219	1.895	2.225	2.615	2.141	2.003	2.309	2.222	2.308	241.70	19121.1
p1–p3	hexa	0.0	10^{-2}	2.271	2.126	2.004	2.004	2.002	2.002	2.384	2.115	2.552	2.552	2.549	2.549	598.75	19898.8
			10^{-3}	2.292	2.131	2.004	2.004	2.003	2.003	2.600	2.164	2.545	2.545	2.542	2.542	842.03	19912.6
		0.1	10^{-2}	2.301	2.106	2.092	2.092	2.090	2.090	2.222	2.054	2.524	2.524	2.524	2.524	560.43	21570.5
			10^{-3}	2.345	2.112	2.092	2.092	2.089	2.089	2.552	2.095	2.516	2.516	2.515	2.515	1027.9	21576.0
	triangular unstr	0.0	10^{-2}	1.912	2.146	1.999	2.182	1.951	2.181	2.612	2.168	2.002	2.161	2.105	2.157	101.45	18110.1
			10^{-3}	1.913	2.147	1.998	2.181	1.949	2.179	2.611	2.168	2.001	2.155	2.096	2.151	320.49	18117.7
p2–NO	hexa	0.0	10^{-2}	1.971	2.163	1.999	2.201	1.975	2.199	2.613	2.141	2.001	2.156	2.080	2.151	122.27	19231.4
			10^{-3}	1.972	2.163	1.999	2.199	1.972	2.197	2.615	2.141	2.001	2.150	2.071	2.145	327.66	19246.6
		0.1	10^{-2}	2.271	2.126	1.992	2.069	2.012	2.068	2.384	2.115	2.004	2.184	2.284	2.176	625.13	20140.2
			10^{-3}	2.292	2.131	1.991	2.067	2.009	2.066	2.600	2.164	2.002	2.176	2.268	2.168	931.04	20140.0
	triangular unstr	0.0	10^{-2}	2.301	2.106	1.992	2.089	2.038	2.087	2.222	2.054	2.002	2.117	2.186	2.113	570.64	21716.5
			10^{-3}	2.345	2.112	1.992	2.087	2.034	2.085	2.552	2.095	2.000	2.109	2.167	2.105	1145.7	21718.7
p2–p1	hexa	0.0	10^{-2}	1.912	2.146	1.999	2.196	1.951	2.195	2.612	2.168	2.002	2.177	2.105	2.173	108.74	18227.1
			10^{-3}	1.913	2.147	1.999	2.194	1.949	2.193	2.611	2.168	2.001	2.170	2.096	2.166	334.71	18237.7
		0.1	10^{-2}	1.971	2.163	1.999	2.217	1.975	2.215	2.613	2.141	2.001	2.172	2.080	2.167	117.87	19282.9
			10^{-3}	1.972	2.163	1.999	2.215	1.973	2.213	2.615	2.141	2.001	2.165	2.071	2.160	331.70	19282.7
	triangular unstr	0.0	10^{-2}	2.271	2.126	1.992	2.082	2.012	2.081	2.384	2.115	2.004	2.219	2.284	2.210	663.03	20261.8
			$10^{-3}</$														

$V_m - I_{\text{ion}}$	Mesh type	α	Δt [s]	Fit $N = 10-100$						Fit $N = 70-100$						t_{Σ} [s]	RSS $_{\Sigma}$ [MB]
				V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G		
p2-p2	triangular unstr	0.1	10^{-3}	2.150	2.609	2.003	2.689	2.102	2.694	2.023	2.214	2.001	2.304	2.011	2.312	229.19	18746.2
			10^{-2}	2.152	2.651	2.004	2.757	2.107	2.760	2.021	2.247	2.002	2.470	2.033	2.476	131.63	20347.1
			10^{-3}	2.153	2.653	2.003	2.728	2.100	2.732	2.026	2.251	2.001	2.350	2.011	2.358	250.07	20328.7
		0.0	10^{-2}	2.056	2.374	2.166	2.166	2.161	2.161	1.982	2.068	2.260	2.260	2.263	2.263	643.71	21946.4
			10^{-3}	2.058	2.376	2.090	2.090	2.086	2.086	1.991	2.077	1.998	1.998	2.002	2.002	944.25	21969.6
			10^{-2}	2.043	2.402	2.162	2.162	2.158	2.158	1.980	2.081	2.286	2.286	2.285	2.285	626.12	24414.7
	hexa	0.1	10^{-3}	2.045	2.405	2.079	2.079	2.075	2.075	1.989	2.091	1.996	1.996	1.997	1.997	1097.3	24415.8
			10^{-2}	2.212	2.708	2.004	2.942	2.118	2.941	2.026	2.303	2.001	2.734	2.020	2.738	133.29	18953.3
			10^{-3}	2.214	2.709	2.003	2.924	2.114	2.923	2.032	2.308	2.000	2.644	2.007	2.650	315.69	18954.4
		0.0	10^{-2}	2.224	2.760	2.004	2.969	2.116	2.968	2.030	2.365	2.001	2.794	2.019	2.797	154.46	20458.3
			10^{-3}	2.226	2.761	2.003	2.952	2.112	2.951	2.037	2.371	2.000	2.706	2.007	2.711	388.09	20460.4
			10^{-2}	2.118	2.532	1.993	2.860	2.044	2.862	1.977	2.149	2.000	2.751	2.026	2.757	655.40	22216.4
p2-p3	triangular unstr	0.1	10^{-3}	2.122	2.536	1.992	2.805	2.037	2.807	1.990	2.162	1.998	2.489	2.003	2.499	1056.9	22218.4
			10^{-2}	2.104	2.579	1.993	2.895	2.042	2.896	1.972	2.186	2.000	2.820	2.025	2.823	649.16	24557.3
			10^{-3}	2.108	2.583	1.992	2.840	2.035	2.842	1.987	2.200	1.998	2.553	2.002	2.561	1200.0	24559.1
		0.0	10^{-2}	2.212	2.708	2.004	2.940	2.118	2.940	2.026	2.303	2.001	2.739	2.020	2.743	129.78	19057.3
			10^{-3}	2.214	2.709	2.004	2.923	2.114	2.922	2.032	2.308	2.000	2.650	2.007	2.656	377.79	19059.3
			10^{-2}	2.224	2.760	2.004	2.967	2.117	2.966	2.030	2.365	2.001	2.798	2.019	2.801	196.31	20537.1
	hexa	0.1	10^{-3}	2.226	2.761	2.004	2.950	2.113	2.950	2.037	2.371	2.000	2.711	2.007	2.716	499.25	20511.8
			10^{-2}	2.118	2.532	1.993	2.863	2.044	2.865	1.977	2.149	2.000	2.758	2.026	2.764	683.61	22289.2
			10^{-3}	2.122	2.536	1.993	2.809	2.037	2.812	1.990	2.162	1.998	2.500	2.003	2.510	1067.6	22304.9
		0.0	10^{-2}	2.104	2.579	1.993	2.897	2.041	2.898	1.972	2.186	2.000	2.825	2.025	2.829	626.99	24625.8
			10^{-3}	2.108	2.583	1.993	2.843	2.035	2.846	1.987	2.200	1.998	2.564	2.002	2.572	1257.2	24623.2
			10^{-2}	2.121	2.481	2.256	2.256	2.258	2.258	1.999	2.037	2.472	2.472	2.469	2.469	127.96	21803.9
p3-NO	triangular unstr	0.1	10^{-3}	2.125	2.485	2.132	2.132	2.135	2.135	2.013	2.050	2.025	2.025	2.024	2.024	245.34	21804.9
			10^{-2}	2.104	2.476	2.238	2.238	2.240	2.240	1.996	2.034	2.467	2.467	2.464	2.464	175.14	24520.5
			10^{-3}	2.108	2.481	2.114	2.114	2.117	2.117	2.010	2.048	2.020	2.020	2.020	2.020	377.83	24497.7
		0.0	10^{-2}	2.033	2.201	2.348	2.348	2.345	2.345	1.972	1.988	3.183	3.183	3.172	3.172	725.87	27537.3
			10^{-3}	2.041	2.209	2.061	2.061	2.060	2.060	1.999	2.015	2.008	2.008	2.008	2.008	1086.6	27536.0
			10^{-2}	2.021	2.196	2.333	2.333	2.331	2.331	1.972	1.987	3.182	3.182	3.171	3.171	700.89	31526.0
	hexa	0.1	10^{-3}	2.029	2.204	2.046	2.046	2.046	2.046	1.999	2.014	2.008	2.008	2.007	2.007	999.45	31524.1
			10^{-2}	2.181	2.631	2.002	2.762	1.992	2.757	2.003	2.056	2.002	2.661	2.008	2.650	151.23	21847.0
			10^{-3}	2.187	2.637	2.001	2.598	1.989	2.596	2.020	2.073	2.000	2.068	1.999	2.066	358.46	21855.3
		0.0	10^{-2}	2.156	2.626	2.002	2.756	1.991	2.752	1.998	2.051	2.002	2.653	2.008	2.642	196.90	24560.6
			10^{-3}	2.161	2.631	2.001	2.592	1.988	2.589	2.016	2.069	2.000	2.060	1.999	2.059	534.78	24560.3
			10^{-2}	2.033	2.201	2.348	2.348	2.345	2.345	1.972	1.988	3.183	3.183	3.172	3.172	792.64	27566.0
p3-p1	triangular unstr	0.1	10^{-3}	2.041	2.209	2.061	2.061	2.060	2.060	1.999	2.015	2.008	2.008	2.008	2.008	1511.9	27558.5
			10^{-2}	2.021	2.196	2.333	2.333	2.331	2.331	1.972	1.987	3.182	3.182	3.171	3.171	711.74	31533.8
			10^{-3}	2.029	2.204	2.046	2.046	2.046	2.046	1.999	2.014	2.008	2.008	2.007	2.007	1438.6	31546.0
		0.0	10^{-2}	3.520	3.971	2.000	3.744	1.999	3.748	4.171	4.124	2.001	1.458	2.006	1.396	178.12	21950.5
			10^{-3}	3.538	3.992	1.999	4.003	1.997	4.010	4.323	4.279	2.000	4.194	2.001	4.195	430.39	21968.9
			10^{-2}	3.547	4.002	1.999	3.757	1.998	3.761	4.105	4.100	2.001	1.348	2.006	1.284	181.72	24663.2
	hexa	0.1	10^{-3}	3.569	4.026	1.999	4.033	1.997	4.040	4.299	4.271	2.000	4.212	2.000	4.213	512.98	24660.5
			10^{-2}	3.749	3.924	1.992	2.844	2.005	2.843	2.165	2.958	2.000	-0.092	2.012	-0.097	974.21	27701.8
			10^{-3}	4.057	4.096	1.992	4.077	2.002	4.083	4.476	4.165	1.998	4.204	2.003	4.204	1565.6	27688.4
		0.0	10^{-2}	3.693	3.924	1.992	2.836	2.005	2.836	1.802	2.887	2.000	-0.098	2.012	-0.103	907.50	31675.3
			10^{-3}	4.077	4.105	1.992	4.094	2.001	4.098	4.460	4.138	1.998	4.195	2.003	4.198	1452.0	31683.3
			10^{-2}	3.520	3.971	2.000	3.738	1.999	3.742	4.171	4.124	2.001	1.420	2.006	1.355	206.65	22042.1
p3-p3	triangular unstr	0.1	10^{-3}	3.538	3.992	1.999	4.000	1.998	4.008	4.323	4.279	2.000	4.198	2.001	4.200	451.44	22029.4
			10^{-2}	3.547	4.002	2.000	3.751	1.999	3.755	4.105	4.100	2.001	1.305	2.006	1.239	227.33	24736.5
			10^{-3}	3.569	4.026	1.999	4.031	1.997	4.039	4.299	4.271	2.000	4.217	2.001	4.219	669.66	24733.5
		0.0	10^{-2}	3.748	3.924	1.992	2.812	2.005	2.811	2.165	2.958	2.000	-0.108	2.012	-0.114	802.37	27772.6
			10^{-3}	4.057	4.096	1.992	4.079	2.002	4.086	4.476	4.165	1.998	4.227	2.003	4.229	1653.7	27773.2
			10^{-2}	3.693	3.923	1.992	2.803	2.005	2.803	1.802	2.887	2.000	-0.115	2.012	-0.120	733.20	31735.5
	hexa	0.1	10^{-3}	4.077	4.105	1.992	4.098	2.002	4.103	4.460	4.138	1.998	4.218	2.003	4.222	1605.1	31748.6

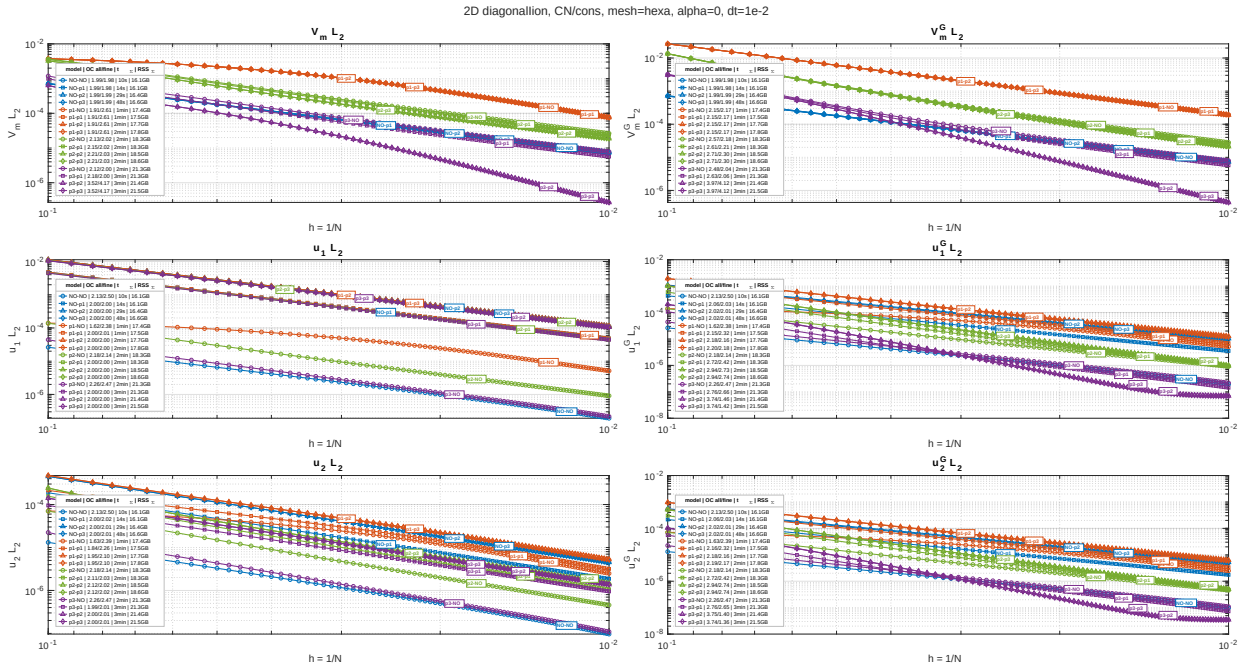


Figure 9: 2D convergence block for diagonalion, hexa mesh, $\alpha = 0$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

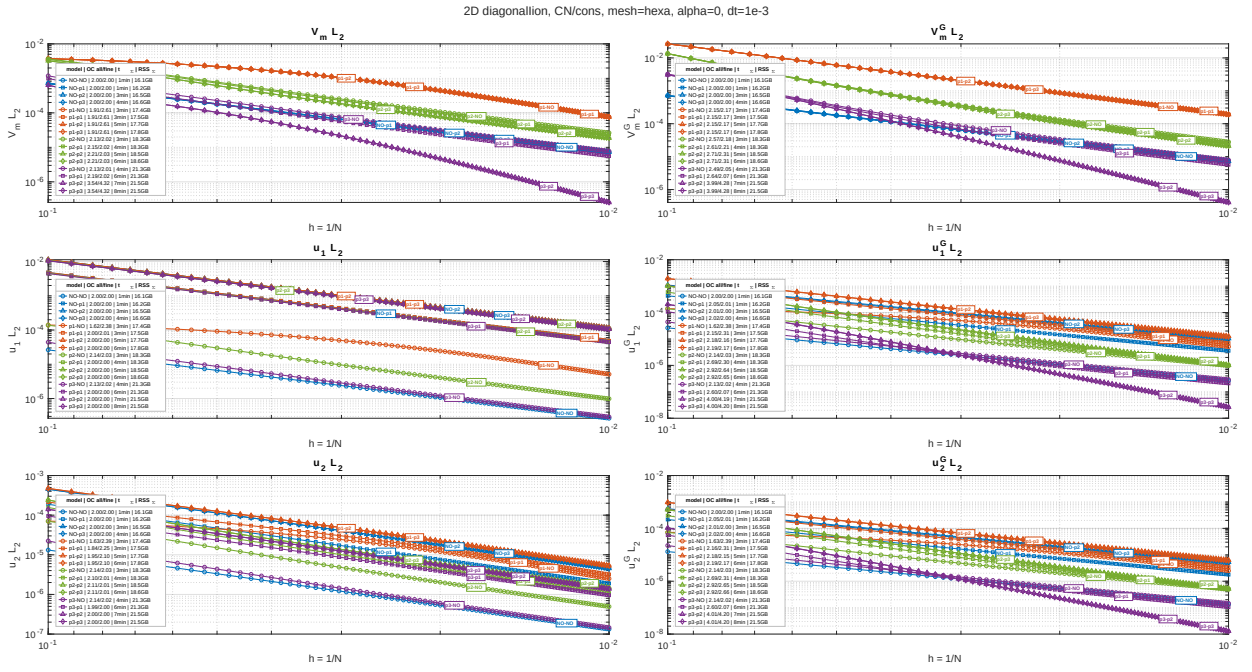


Figure 10: 2D convergence block for `diagonalIion`, hexa mesh, $\alpha = 0$, $\Delta t = 10^{-3}$, `crankNicolson` time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

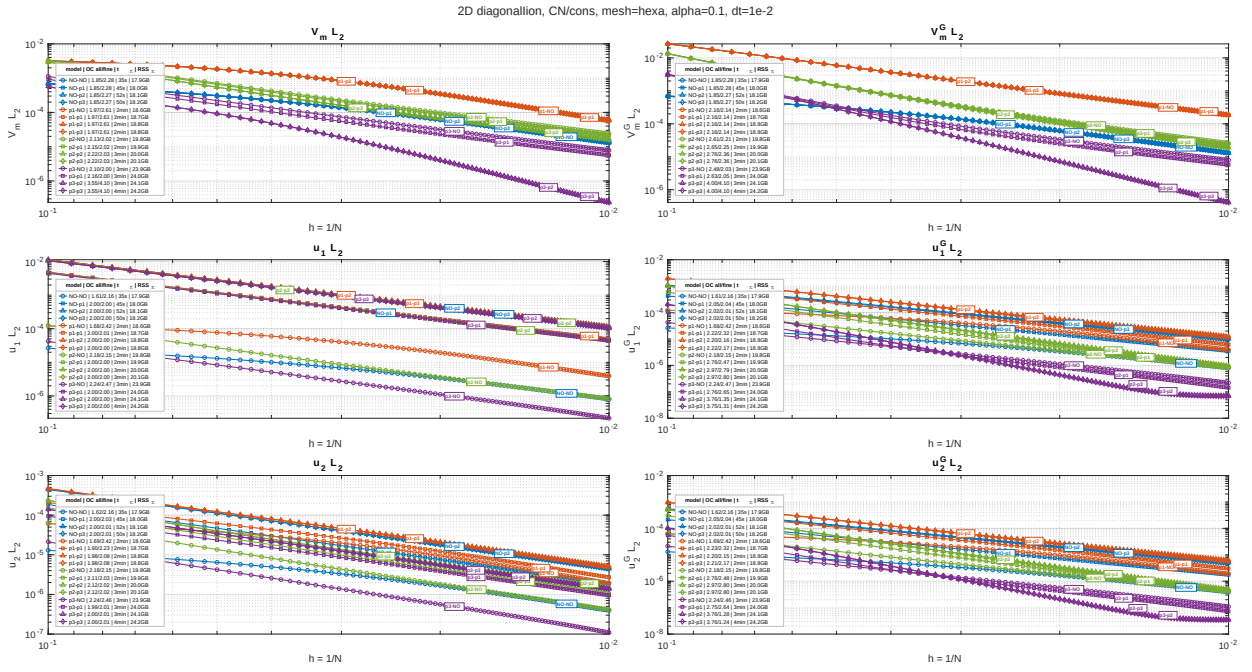


Figure 11: 2D convergence block for diagonalIon, hexa mesh, $\alpha = 0.1$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m-I_{ion} model pair.

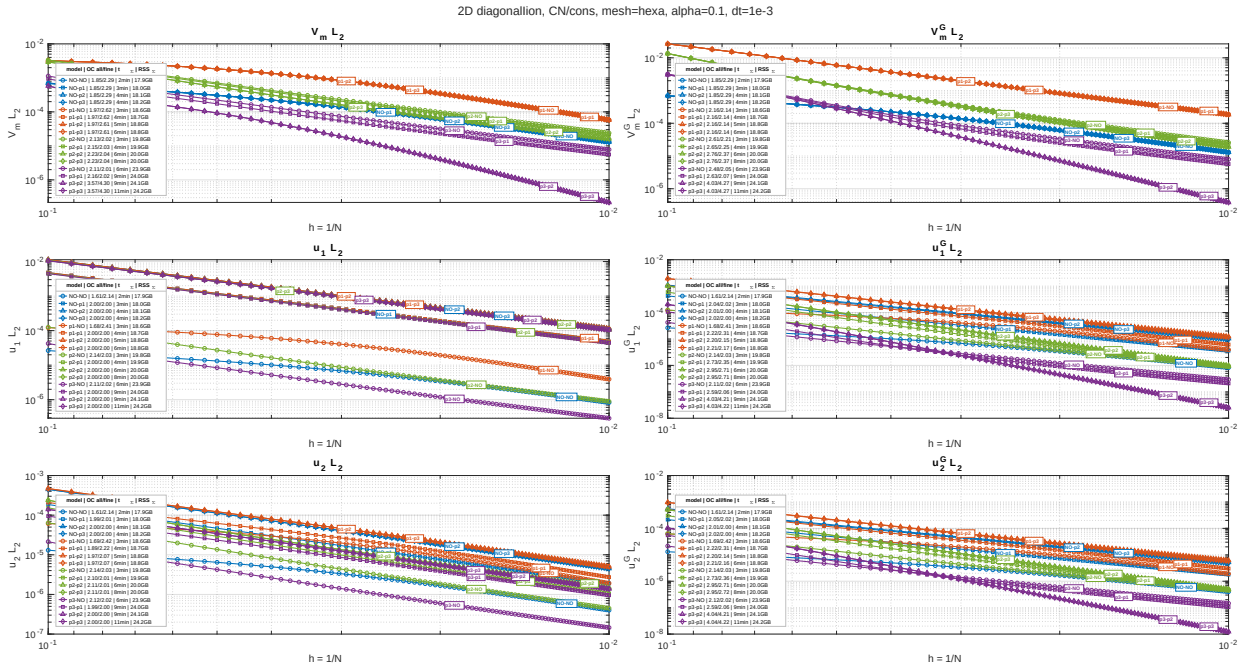


Figure 12: 2D convergence block for `diagonalIon`, hexa mesh, $\alpha = 0.1$, $\Delta t = 10^{-3}$, `crankNicolson` time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

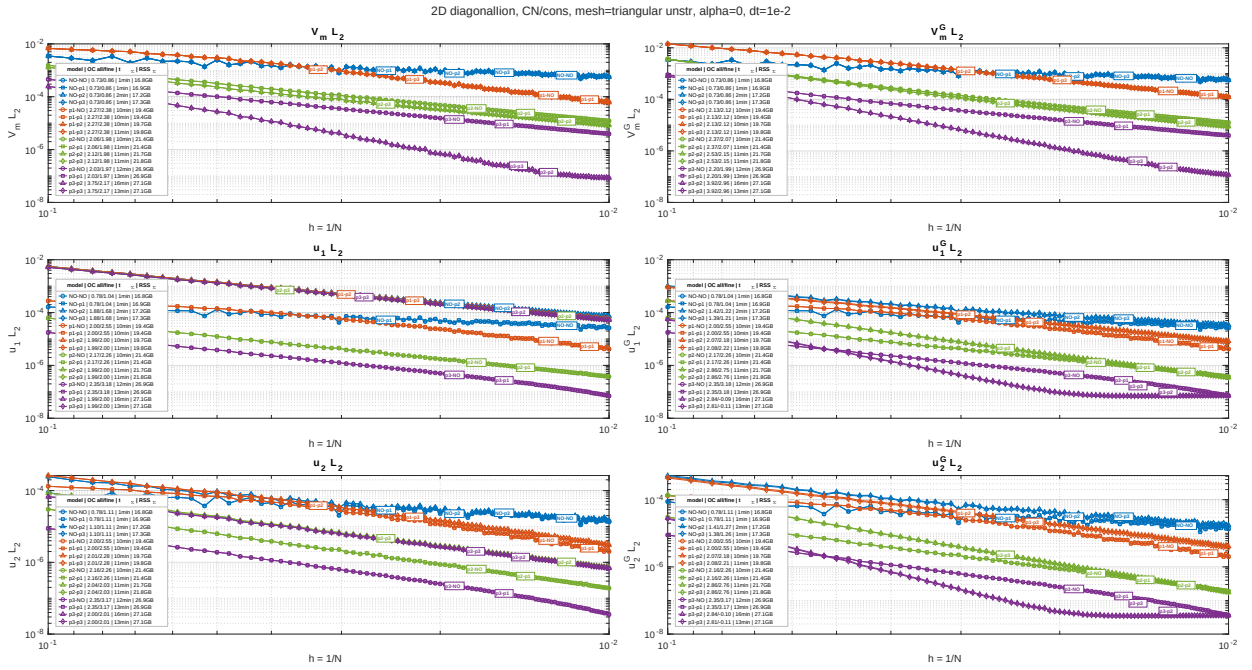


Figure 13: 2D convergence block for `diagonalion`, triangular unstr mesh, $\alpha = 0$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

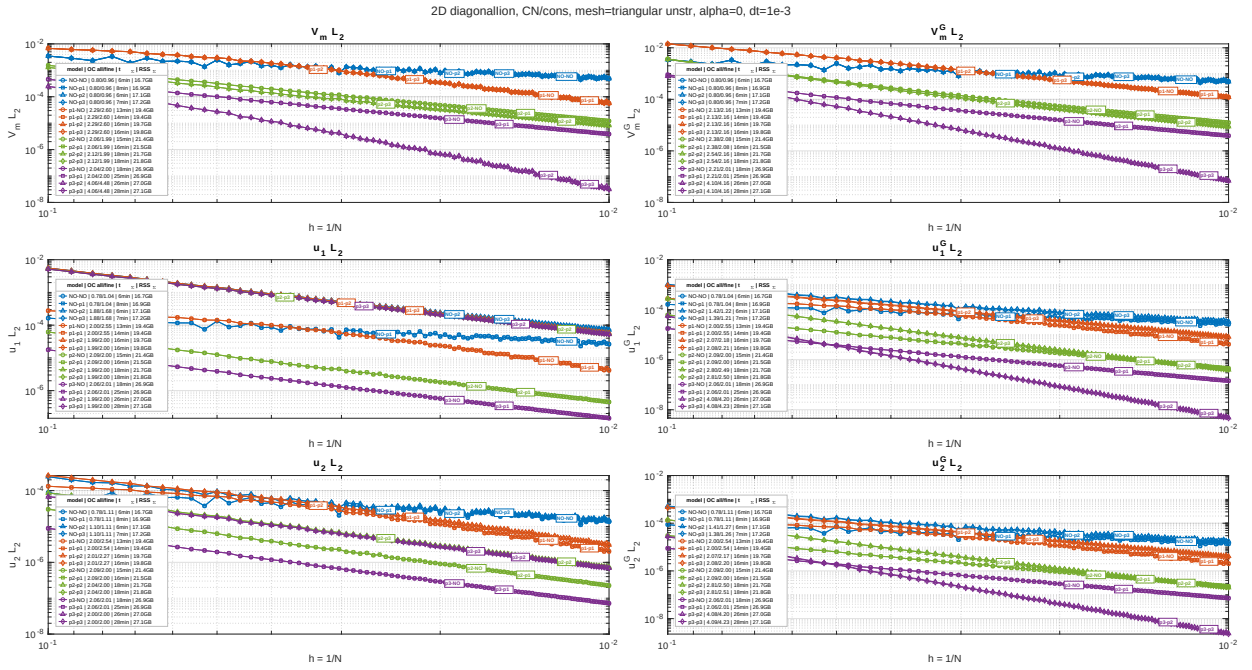


Figure 14: 2D convergence block for `diagonalion`, triangular unstr mesh, $\alpha = 0$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m – I_{ion} model pair.

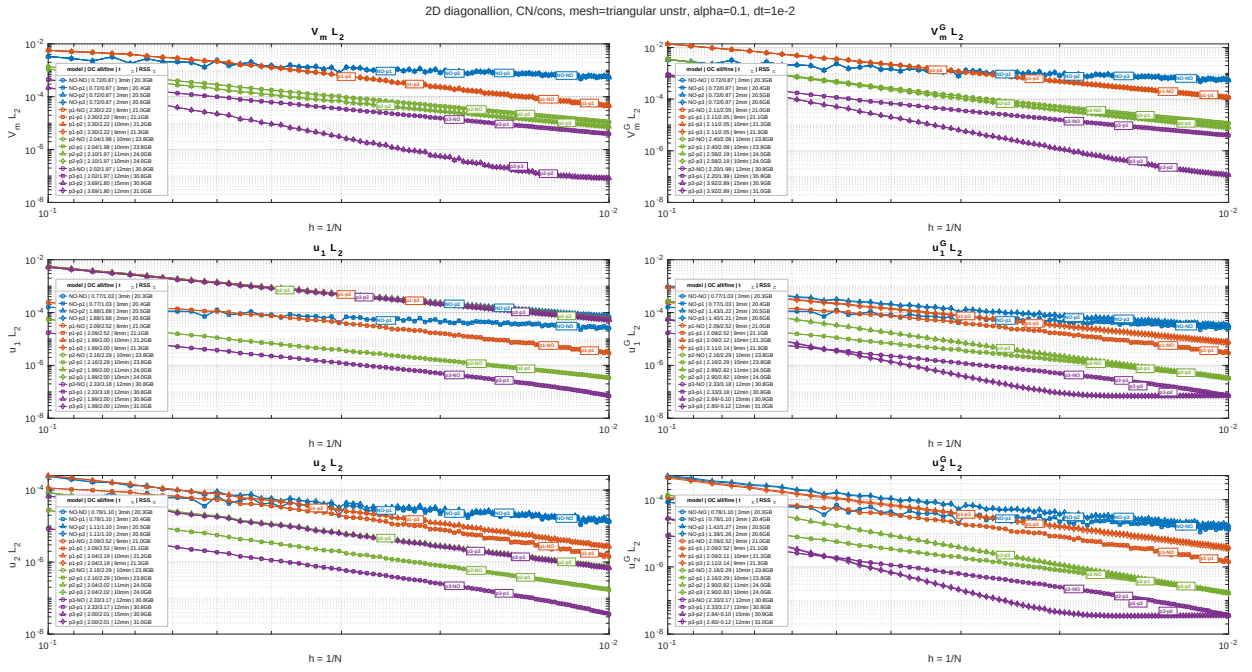


Figure 15: 2D convergence block for `diagonalion`, triangular unstr mesh, $\alpha = 0.1$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

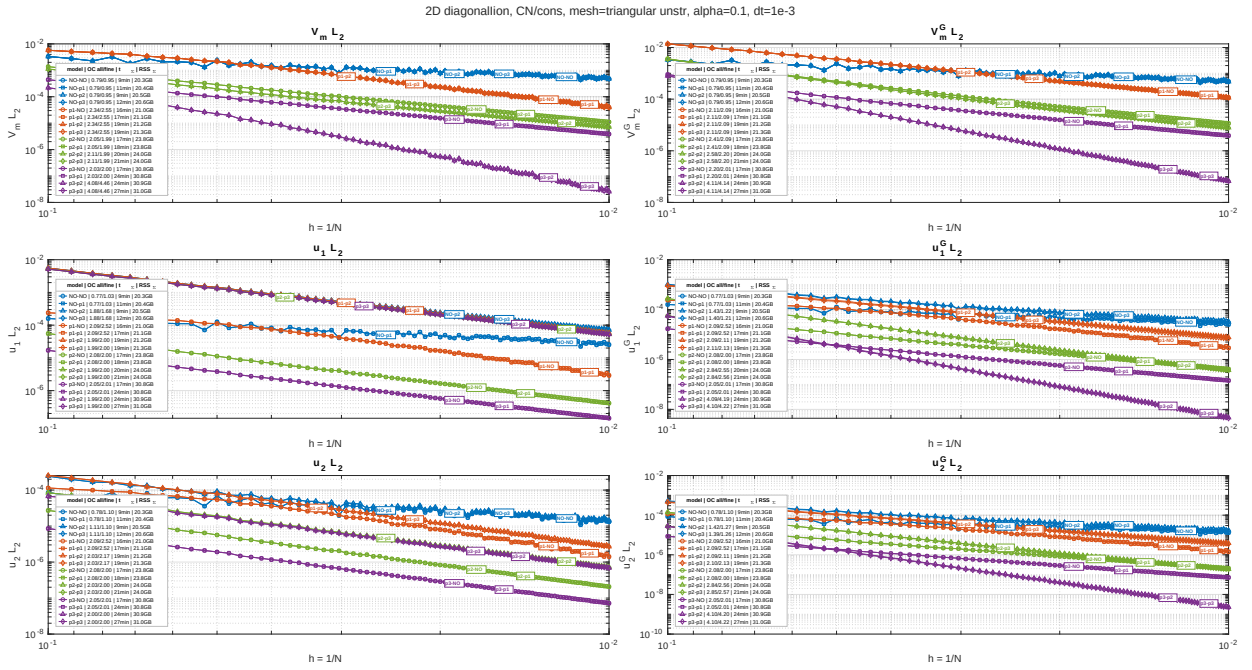


Figure 16: 2D convergence block for diagonalion, triangular unstr mesh, $\alpha = 0.1$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m-I_{ion} model pair.

11.1.3 JFNK, crankNicolson, consistent

Table 4: 2D manufactured results for JFNK, crankNicolson, and the consistent mass matrix. The first six convergence columns are log-log fits over meshes $N = 10$ –100; the next six columns are fits over $N = 70$ –100. The time column is the sum of the reported wall-clock total over all mesh runs in the row, and the memory column is the sum of the reported peak resident memory in MB over the same mesh runs.

		Fit $N = 10$ –100										Fit $N = 70$ –100							
$V_m - I_{\text{ion}}$	Mesh type	α	Δt [s]	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	t_{Σ} [s]	RSS $_{\Sigma}$ [MB]		
NO–NO	hexa	0.0	10^{-2}	1.992	1.992	2.135	2.135	2.134	2.134	1.985	1.985	2.500	2.500	2.497	2.497	9.266	16550.6		
			10^{-3}	1.997	1.997	1.998	1.998	1.998	1.998	2.000	2.000	2.004	2.004	2.004	2.004	62.774	16499.9		
		0.1	10^{-2}	1.849	1.849	1.612	1.612	1.618	1.618	2.278	2.278	2.159	2.159	2.163	2.163	30.513	18377.9		
	10^{-3}		1.851	1.851	1.607	1.607	1.613	1.613	2.294	2.294	2.140	2.140	2.142	2.142	104.37	18378.4			
	triangular unstr	0.0	10^{-2}	0.730	0.730	0.778	0.778	0.784	0.784	0.862	0.862	1.037	1.037	1.107	1.107	78.833	17294.1		
			10^{-3}	0.798	0.798	0.777	0.777	0.783	0.783	0.963	0.963	1.038	1.038	1.107	1.107	229.82	17289.7		
0.1		10^{-2}	0.722	0.722	0.772	0.772	0.777	0.777	0.871	0.871	1.027	1.027	1.097	1.097	105.57	20811.1			
	10^{-3}	0.785	0.785	0.771	0.771	0.776	0.776	0.953	0.953	1.027	1.027	1.097	1.097	335.59	20799.8				
NO–p1	hexa	0.0	10^{-2}	1.992	1.992	2.000	2.057	2.005	2.061	1.985	1.985	2.002	2.031	2.023	2.031	13.090	16727.8		
			10^{-3}	1.997	1.997	1.999	2.050	1.998	2.053	2.000	2.000	2.000	2.007	2.000	2.007	87.918	16718.4		
		0.1	10^{-2}	1.849	1.849	2.000	2.049	1.997	2.053	2.278	2.278	2.002	2.039	2.030	2.039	24.074	18448.5		
	10^{-3}		1.851	1.851	1.999	2.042	1.990	2.046	2.294	2.294	2.000	2.015	2.008	2.016	112.16	18447.0			
	triangular unstr	0.0	10^{-2}	0.730	0.730	0.778	0.778	0.784	0.784	0.862	0.862	1.037	1.037	1.107	1.107	95.302	17470.9		
			10^{-3}	0.798	0.798	0.777	0.777	0.783	0.783	0.963	0.963	1.038	1.038	1.107	1.107	299.64	17478.6		
0.1		10^{-2}	0.722	0.722	0.772	0.772	0.777	0.777	0.871	0.871	1.027	1.027	1.097	1.097	132.10	20899.6			
	10^{-3}	0.785	0.785	0.771	0.771	0.776	0.776	0.953	0.953	1.027	1.027	1.097	1.097	455.19	20904.7				
NO–p2	hexa	0.0	10^{-2}	1.992	1.992	1.999	2.015	2.002	2.017	1.986	1.986	2.001	2.010	2.011	2.010	29.398	17030.0		
			10^{-3}	1.997	1.997	1.999	2.013	1.998	2.015	1.999	1.999	2.000	2.002	2.000	2.002	219.26	17038.3		
		0.1	10^{-2}	1.851	1.851	1.999	2.015	2.001	2.017	2.274	2.274	2.001	2.011	2.013	2.011	38.154	18577.5		
	10^{-3}		1.853	1.853	1.999	2.013	1.998	2.014	2.290	2.290	2.000	2.003	2.001	2.003	209.11	18552.2			
	triangular unstr	0.0	10^{-2}	0.730	0.730	1.878	1.418	1.102	1.411	0.862	0.862	1.678	1.219	1.112	1.269	195.38	17820.7		
			10^{-3}	0.799	0.799	1.878	1.417	1.101	1.410	0.963	0.963	1.677	1.218	1.112	1.269	541.47	17817.7		
0.1		10^{-2}	0.722	0.722	1.880	1.427	1.107	1.420	0.871	0.871	1.682	1.220	1.104	1.270	175.66	21048.0			
	10^{-3}	0.786	0.786	1.880	1.426	1.105	1.419	0.953	0.953	1.680	1.219	1.104	1.270	539.30	21041.9				
NO–p3	hexa	0.0	10^{-2}	1.992	1.992	1.999	2.019	2.002	2.021	1.986	1.986	2.001	2.011	2.011	2.011	48.439	17196.3		
			10^{-3}	1.997	1.997	1.999	2.016	1.999	2.018	1.999	1.999	2.000	2.002	2.000	2.002	253.15	17191.9		
		0.1	10^{-2}	1.851	1.851	1.999	2.019	2.001	2.021	2.274	2.274	2.001	2.013	2.013	2.013	49.173	18656.2		
	10^{-3}		1.853	1.853	1.999	2.016	1.998	2.018	2.290	2.290	2.000	2.004	2.002	2.004	265.83	18636.6			
	triangular unstr	0.0	10^{-2}	0.730	0.730	1.878	1.389	1.102	1.384	0.862	0.862	1.678	1.206	1.112	1.258	201.79	17995.4		
			10^{-3}	0.799	0.799	1.878	1.388	1.101	1.383	0.963	0.963	1.677	1.205	1.112	1.258	777.03	18010.7		
0.1		10^{-2}	0.722	0.722	1.881	1.398	1.107	1.392	0.871	0.871	1.682	1.207	1.104	1.259	233.70	21114.5			
	10^{-3}	0.786	0.786	1.880	1.397	1.106	1.391	0.953	0.953	1.680	1.206	1.104	1.258	770.03	21110.3				
p1–NO	hexa	0.0	10^{-2}	1.911	2.146	1.616	1.616	1.626	1.626	2.612	2.168	2.381	2.381	2.387	2.387	77.310	17878.7		
			10^{-3}	1.912	2.147	1.616	1.616	1.626	1.626	2.611	2.168	2.380	2.380	2.385	2.385	150.02	17887.6		
		0.1	10^{-2}	1.971	2.163	1.677	1.677	1.687	1.687	2.614	2.141	2.416	2.416	2.422	2.422	99.193	19092.2		
	10^{-3}		1.971	2.163	1.677	1.677	1.687	1.687	2.615	2.141	2.414	2.414	2.419	2.419	175.68	19071.9			
	triangular unstr	0.0	10^{-2}	2.271	2.126	2.004	2.004	2.002	2.002	2.384	2.115	2.552	2.552	2.549	2.549	627.40	19950.2		
			10^{-3}	2.292	2.131	2.004	2.004	2.003	2.003	2.600	2.164	2.545	2.545	2.542	2.542	839.15	19894.4		
0.1		10^{-2}	2.301	2.106	2.092	2.092	2.090	2.090	2.222	2.054	2.524	2.524	2.524	2.524	630.44	21567.4			
	10^{-3}	2.345	2.112	2.092	2.092	2.089	2.089	2.552	2.095	2.516	2.516	2.515	2.515	966.93	21562.5				
p1–p1	hexa	0.0	10^{-2}	1.911	2.146	1.998	2.154	1.838	2.160	2.612	2.168	2.007	2.316	2.258	2.316	81.233	17974.1		
			10^{-3}	1.912	2.147	1.997	2.152	1.836	2.158	2.611	2.168	2.005	2.309	2.249	2.308	183.83	17987.7		
		0.1	10^{-2}	1.971	2.163	1.999	2.221	1.898	2.227	2.614	2.141	2.005	2.319	2.234	2.318	102.12	19138.4		
	10^{-3}		1.971	2.163	1.999	2.219	1.895	2.225	2.615	2.141	2.003	2.309	2.222	2.308	184.59	19116.1			
	triangular unstr	0.0	10^{-2}	2.271	2.126	2.004	2.004	2.002	2.002	2.384	2.115	2.552	2.552	2.549	2.549	564.30	20001.6		
			10^{-3}	2.292	2.131	2.004	2.004	2.003	2.003	2.600	2.164	2.545	2.545	2.542	2.542	799.04	19938.2		
0.1		10^{-2}	2.301	2.106	2.092	2.092	2.090	2.090	2.222	2.054	2.524	2.524	2.524	2.524	637.92	21601.4			
	10^{-3}	2.345	2.112	2.092	2.092	2.089	2.089	2.552	2.095	2.516	2.516	2.515	2.515	852.76	21573.2				
p1–p2	hexa	0.0	10^{-2}	1.912	2.146	1.999	2.182	1.951	2.181	2.612	2.168	2.002	2.161	2.105	2.157	98.467	18270.7		
			10^{-3}	1.913	2.147	1.998	2.181	1.949	2.179	2.611	2.168	2.001	2.155	2.096	2.151	300.35	18274.8		
		0.1	10^{-2}	1.971	2.163	1.999	2.201	1.975	2.199	2.613	2.141	2.001	2.156	2.080	2.151	101.53	19255.2		
	10^{-3}		1.972	2.163	1.999	2.199	1.972	2.197	2.615	2.141	2.001	2.150	2.071	2.145	287.40	19229.6			
	triangular unstr	0.0	10^{-2}	2.271	2.126	1.992	2.069	2.012	2.068	2.384	2.115	2.004	2.184	2.284	2.176	612.59	20336.8		
			10^{-3}	2.292	2.131	1.991	2.067	2.009	2.066	2.600	2.164	2.002	2.176	2.268	2.168	920.70	20295.7		
0.1		10^{-2}	2.301	2.106	1.992	2.089	2.038	2.087	2.222	2.054	2.002	2.117	2.186	2.113	623.45	21725.5			
	10^{-3}	2.345	2.112	1.992	2.087	2.034	2.085	2.552	2.095	2.000	2.109	2.167	2.105	1012.2	21716.7				
p1–p3	hexa	0.0	10^{-2}	1.912	2.146	1.999	2.196	1.951	2.195	2.612	2.168	2.002	2.177	2.105	2.173	102.50	18421.3		
			10^{-3}	1.913	2.147	1.999	2.194	1.949	2.193	2.611	2.168	2.001	2.170	2.096	2.166	315.72	18429.8		
		0.1	10^{-2}	1.971	2.163	1.999	2.217	1.975	2.215	2.613	2.141	2.001	2.172	2.080	2.167	109.22	19318.6		
	10^{-3}		1.972	2.163	1.999	2.215	1.973	2.213	2.615	2.141	2.001	2.165	2.071	2.160	344.67	19322.5			
	triangular unstr	0.0	10^{-2}	2.271	2.126	1.992	2.082	2.012	2.081	2.384	2.115	2.004	2.219	2.284	2.210	633.00	20513.5		
			10^{-3}	2.292	2.131	1.992	2.080	2.009	2.079	2.600	2.164	2.002	2.209	2.268	2.200	1028.1	20451.8		
0.1		10^{-2}	2.301	2.106	1.992	2.109	2.038	2.106	2.222	2.054	2.002	2.144	2.186	2.139	616.82	21804.6			
	10^{-3}	2.345	2.112	1.992	2.106	2.034	2.104	2.552	2.095	2.000	2.134	2.167	2.129	1107.5	21798.8				
p2–NO	hexa	0.0	10^{-2}	2.127	2.567	2.176	2.176	2.176	2.176	2.016	2.179	2.139	2.139	2.139	93.266	18711.6			
			10^{-3}	2.128	2.568	2.142	2.142	2.143	2.143	2.020	2.183	2.025	2.025	2.026	2.026	154.77	18705.6		
		0.1	10^{-2}	2.128	2.607	2.180	2.180	2.181	2.181	2.018	2.209	2.153	2.153	2.153	97.913	20308.0			
	10^{-3}		2.129	2.608	2.143	2.143	2.144	2.144	2.022	2.213	2.027	2.027	2.028	2.028	177.02	20298.4			
	triangular unstr	0.0	10^{-2}	2.056	2.374	2.166	2.166	2.161	2.161	1.982	2.068	2.260	2.260</						

$V_m - I_{\text{ion}}$	Mesh type	α	Δt [s]	Fit $N = 10-100$						Fit $N = 70-100$						t_{Σ} [s]	RSS $_{\Sigma}$ [MB]
				V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G		
p2-p2	triangular unstr	0.1	10^{-3}	2.150	2.609	2.003	2.689	2.102	2.694	2.023	2.214	2.001	2.304	2.011	2.312	206.03	18811.8
			10^{-2}	2.152	2.651	2.004	2.757	2.107	2.760	2.021	2.247	2.002	2.470	2.033	2.476	110.50	20350.3
			10^{-3}	2.153	2.653	2.003	2.728	2.100	2.732	2.026	2.251	2.001	2.350	2.011	2.358	229.20	20346.2
		0.0	10^{-2}	2.056	2.374	2.166	2.166	2.161	2.161	1.982	2.068	2.260	2.260	2.263	2.263	581.18	22050.9
			10^{-3}	2.058	2.376	2.090	2.090	2.086	2.086	1.991	2.077	1.998	1.998	2.002	2.002	935.71	22002.3
			10^{-2}	2.043	2.402	2.162	2.162	2.158	2.158	1.980	2.081	2.286	2.286	2.285	2.285	626.10	24445.3
	hexa	0.1	10^{-3}	2.045	2.405	2.079	2.079	2.075	2.075	1.989	2.091	1.996	1.996	1.997	1.997	863.88	24427.9
			10^{-2}	2.212	2.708	2.004	2.942	2.118	2.941	2.026	2.303	2.001	2.734	2.020	2.738	116.93	19113.0
			10^{-3}	2.214	2.709	2.003	2.924	2.114	2.923	2.032	2.308	2.000	2.644	2.007	2.650	290.14	19087.4
		0.0	10^{-2}	2.224	2.760	2.004	2.969	2.116	2.968	2.030	2.365	2.001	2.794	2.019	2.797	111.94	20456.4
			10^{-3}	2.226	2.761	2.003	2.952	2.112	2.951	2.037	2.371	2.000	2.706	2.007	2.711	316.92	20450.9
			10^{-2}	2.118	2.532	1.993	2.860	2.044	2.862	1.977	2.149	2.000	2.751	2.026	2.757	618.26	22383.5
p2-p3	triangular unstr	0.1	10^{-3}	2.122	2.536	1.992	2.805	2.037	2.807	1.990	2.162	1.998	2.489	2.003	2.499	1036.4	22340.1
			10^{-2}	2.104	2.579	1.993	2.895	2.042	2.896	1.972	2.186	2.000	2.820	2.025	2.823	648.71	24583.9
			10^{-3}	2.108	2.583	1.992	2.840	2.035	2.842	1.987	2.200	1.998	2.553	2.002	2.561	987.41	24553.8
		0.0	10^{-2}	2.212	2.708	2.004	2.940	2.118	2.940	2.026	2.303	2.001	2.739	2.020	2.743	129.66	19251.7
			10^{-3}	2.214	2.709	2.004	2.923	2.114	2.922	2.032	2.308	2.000	2.650	2.007	2.656	346.39	19235.2
			10^{-2}	2.224	2.760	2.004	2.967	2.117	2.966	2.030	2.365	2.001	2.798	2.019	2.801	124.41	20530.2
	hexa	0.1	10^{-3}	2.226	2.761	2.004	2.950	2.113	2.950	2.037	2.371	2.000	2.711	2.007	2.716	366.70	20522.7
			10^{-2}	2.118	2.532	1.993	2.863	2.044	2.865	1.977	2.149	2.000	2.758	2.026	2.764	637.22	22551.2
			10^{-3}	2.122	2.536	1.993	2.809	2.037	2.812	1.990	2.162	1.998	2.500	2.003	2.510	1128.1	22514.3
		0.0	10^{-2}	2.104	2.579	1.993	2.897	2.041	2.898	1.972	2.186	2.000	2.825	2.025	2.829	691.48	24650.1
			10^{-3}	2.108	2.583	1.993	2.843	2.035	2.846	1.987	2.200	1.998	2.564	2.002	2.572	1084.0	24631.3
			10^{-2}	2.121	2.481	2.256	2.256	2.258	2.258	1.999	2.037	2.472	2.472	2.469	2.469	147.82	21837.0
p3-NO	triangular unstr	0.1	10^{-3}	2.125	2.485	2.132	2.132	2.135	2.135	2.013	2.050	2.025	2.025	2.024	2.024	192.01	21809.6
			10^{-2}	2.104	2.476	2.238	2.238	2.240	2.240	1.996	2.034	2.467	2.467	2.464	2.464	152.52	24499.4
			10^{-3}	2.108	2.481	2.114	2.114	2.117	2.117	2.010	2.048	2.020	2.020	2.020	2.020	320.68	24530.8
		0.0	10^{-2}	2.033	2.201	2.348	2.348	2.345	2.345	1.972	1.988	3.183	3.183	3.172	3.172	689.38	27545.2
			10^{-3}	2.041	2.209	2.061	2.061	2.060	2.060	1.999	2.015	2.008	2.008	2.008	2.008	968.40	27545.1
			10^{-2}	2.021	2.196	2.333	2.333	2.331	2.331	1.972	1.987	3.182	3.182	3.171	3.171	760.32	31535.9
	hexa	0.1	10^{-3}	2.029	2.204	2.046	2.046	2.046	2.046	1.999	2.014	2.008	2.008	2.007	2.007	884.29	31519.3
			10^{-2}	2.181	2.631	2.002	2.762	1.992	2.757	2.003	2.056	2.002	2.661	2.008	2.650	148.12	21874.5
			10^{-3}	2.187	2.637	2.001	2.598	1.989	2.596	2.020	2.073	2.000	2.068	1.999	2.066	290.60	21859.5
		0.0	10^{-2}	2.156	2.626	2.002	2.756	1.991	2.752	1.998	2.051	2.002	2.653	2.008	2.642	178.17	24556.5
			10^{-3}	2.161	2.631	2.001	2.592	1.988	2.589	2.016	2.069	2.000	2.060	1.999	2.059	432.06	24569.5
			10^{-2}	2.033	2.201	2.348	2.348	2.345	2.345	1.972	1.988	3.183	3.183	3.172	3.172	693.51	27581.7
p3-p1	triangular unstr	0.1	10^{-3}	2.041	2.209	2.061	2.061	2.060	2.060	1.999	2.015	2.008	2.008	2.008	2.008	1230.9	27569.1
			10^{-2}	2.021	2.196	2.333	2.333	2.331	2.331	1.972	1.987	3.182	3.182	3.171	3.171	800.77	31560.1
			10^{-3}	2.029	2.204	2.046	2.046	2.046	2.046	1.999	2.014	2.008	2.008	2.007	2.007	1092.4	31558.3
		0.0	10^{-2}	3.520	3.971	2.000	3.744	1.999	3.748	4.171	4.124	2.001	1.458	2.006	1.396	170.50	21996.1
			10^{-3}	3.538	3.992	1.999	4.003	1.997	4.010	4.324	4.279	2.000	4.194	2.001	4.196	381.25	21971.2
			10^{-2}	3.547	4.002	1.999	3.757	1.998	3.761	4.105	4.100	2.001	1.348	2.006	1.284	202.72	24666.6
	hexa	0.1	10^{-3}	3.569	4.026	1.999	4.033	1.997	4.040	4.299	4.272	2.000	4.212	2.000	4.213	511.96	24681.8
			10^{-2}	3.749	3.924	1.992	2.844	2.005	2.843	2.167	2.959	2.000	-0.092	2.012	-0.097	805.00	27710.0
			10^{-3}	4.057	4.096	1.992	4.077	2.002	4.083	4.476	4.164	1.998	4.204	2.003	4.204	1724.5	27714.9
		0.0	10^{-2}	3.693	3.924	1.992	2.836	2.005	2.836	1.804	2.888	2.000	-0.098	2.012	-0.103	699.63	31685.8
			10^{-3}	4.077	4.105	1.992	4.094	2.001	4.098	4.461	4.139	1.998	4.195	2.003	4.198	1145.9	31669.6
			10^{-2}	3.520	3.971	2.000	3.738	1.999	3.742	4.171	4.124	2.001	1.420	2.006	1.355	174.93	22054.6
p3-p3	triangular unstr	0.1	10^{-3}	3.538	3.992	1.999	4.000	1.998	4.008	4.324	4.279	2.000	4.198	2.001	4.200	440.53	22050.9
			10^{-2}	3.547	4.002	2.000	3.751	1.999	3.756	4.105	4.100	2.001	1.305	2.006	1.239	246.90	24749.6
			10^{-3}	3.569	4.026	1.999	4.031	1.997	4.039	4.299	4.272	2.000	4.217	2.001	4.219	472.62	24729.1
		0.0	10^{-2}	3.749	3.924	1.992	2.812	2.005	2.811	2.167	2.959	2.000	-0.108	2.012	-0.114	882.90	27779.4
			10^{-3}	4.057	4.096	1.992	4.079	2.002	4.086	4.477	4.165	1.998	4.227	2.003	4.229	1466.0	27775.6
			10^{-2}	3.693	3.923	1.992	2.803	2.005	2.803	1.804	2.888	2.000	-0.115	2.012	-0.120	886.85	31746.3
	hexa	0.1	10^{-3}	4.077	4.105	1.992	4.098	2.002	4.103	4.461	4.139	1.998	4.219	2.003	4.222	1625.2	31740.2

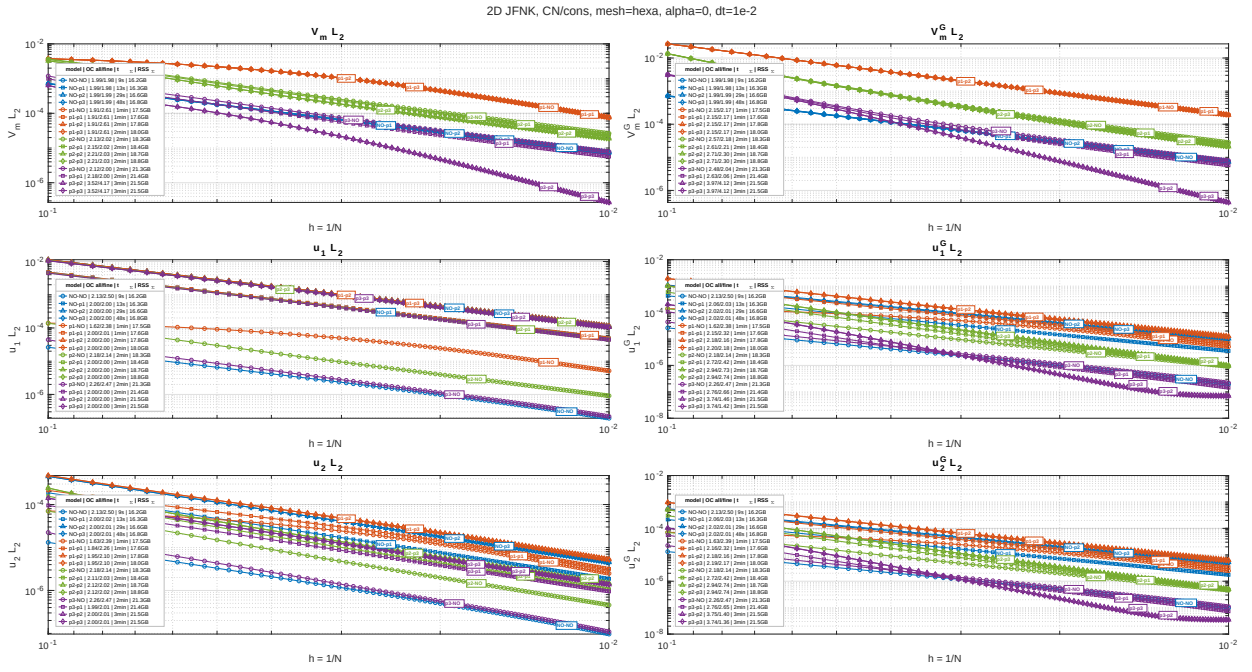


Figure 17: 2D convergence block for JFNK, hexa mesh, $\alpha = 0$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m-I_{ion} model pair.

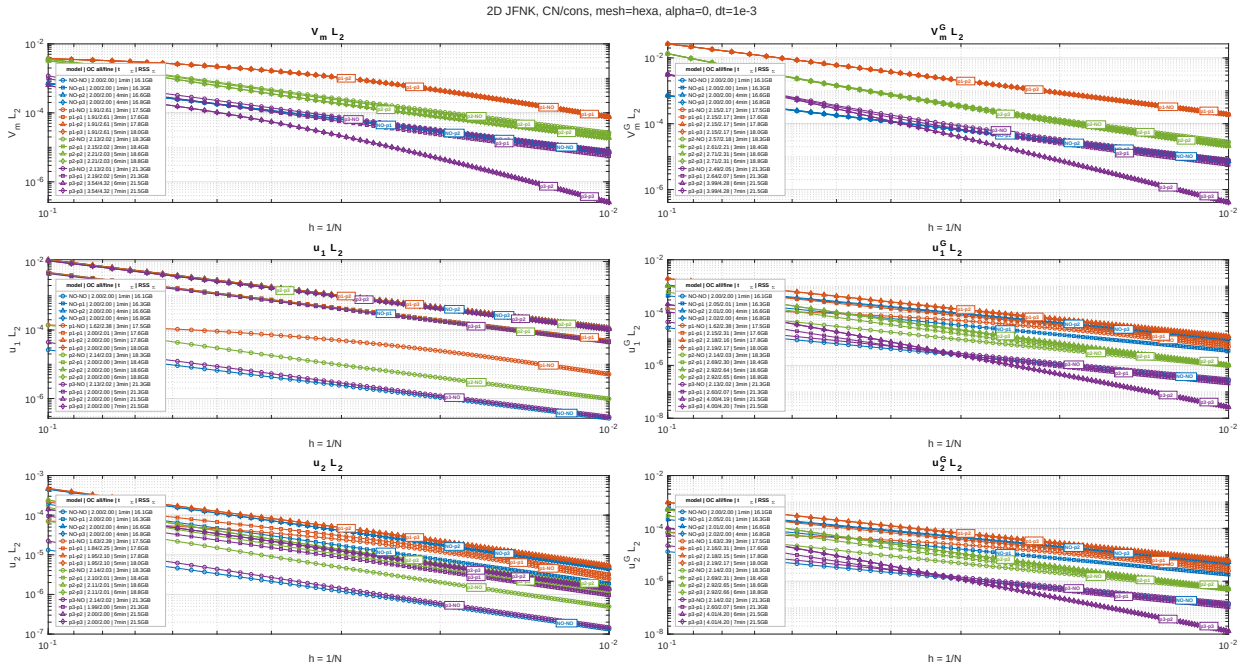


Figure 18: 2D convergence block for JFNK, hexa mesh, $\alpha = 0$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

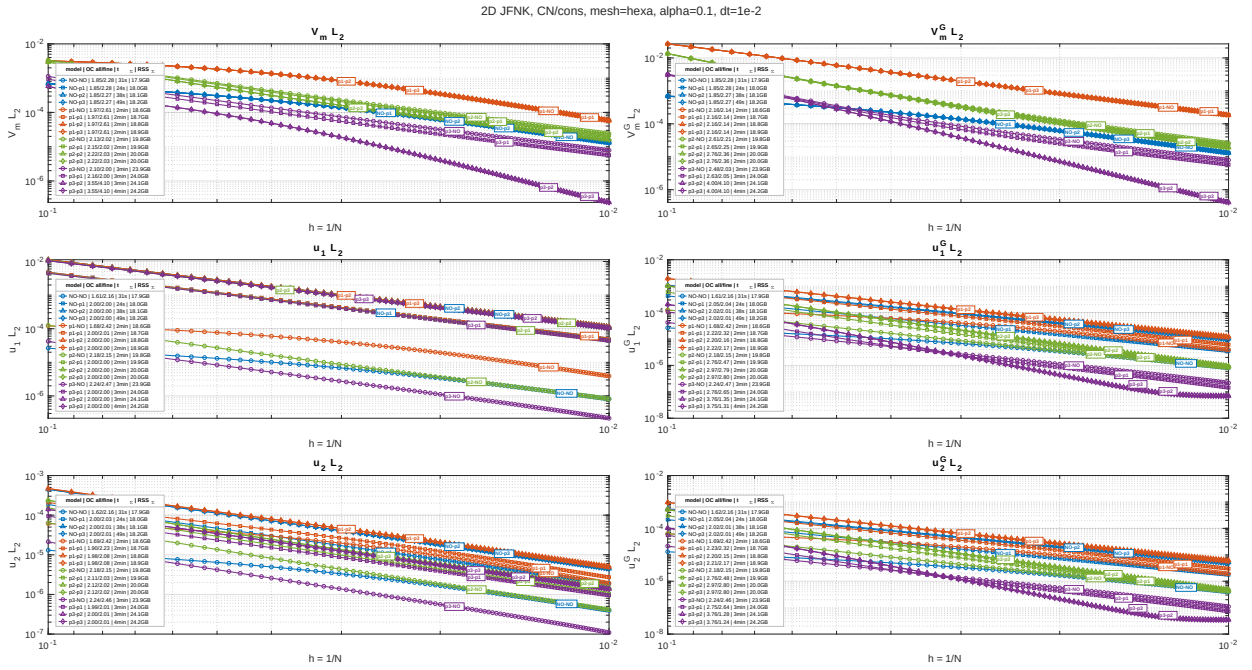


Figure 19: 2D convergence block for JFNK, hexa mesh, $\alpha = 0.1$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the $V_m - I_{\text{ion}}$ model pair.

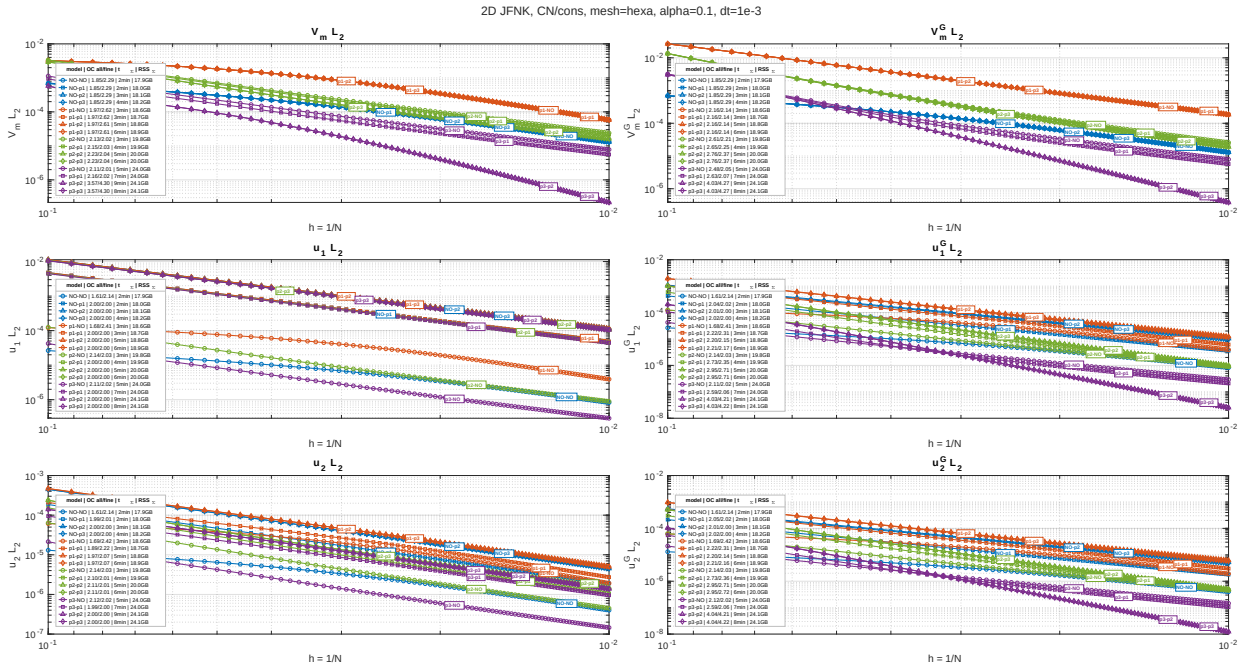


Figure 20: 2D convergence block for JFNK, hexa mesh, $\alpha = 0.1$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m-I_{ion} model pair.

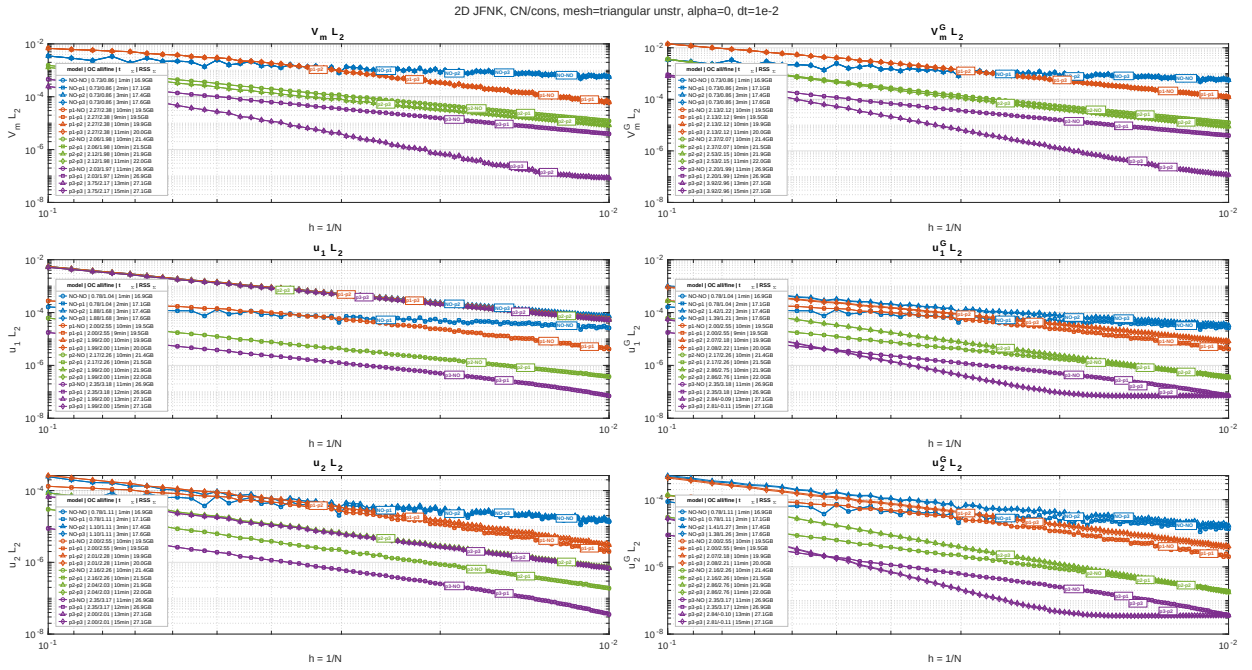


Figure 21: 2D convergence block for JFNK, triangular unstr mesh, $\alpha = 0$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m-I_{ion} model pair.

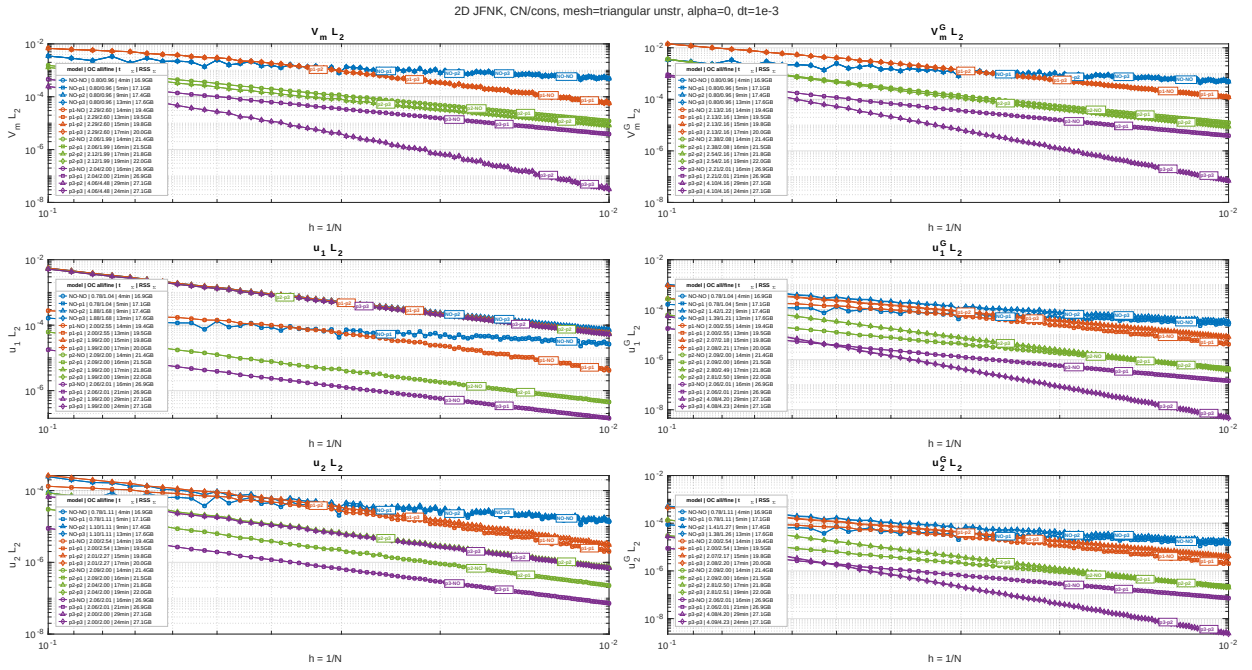


Figure 22: 2D convergence block for JFNK, triangular unstr mesh, $\alpha = 0$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

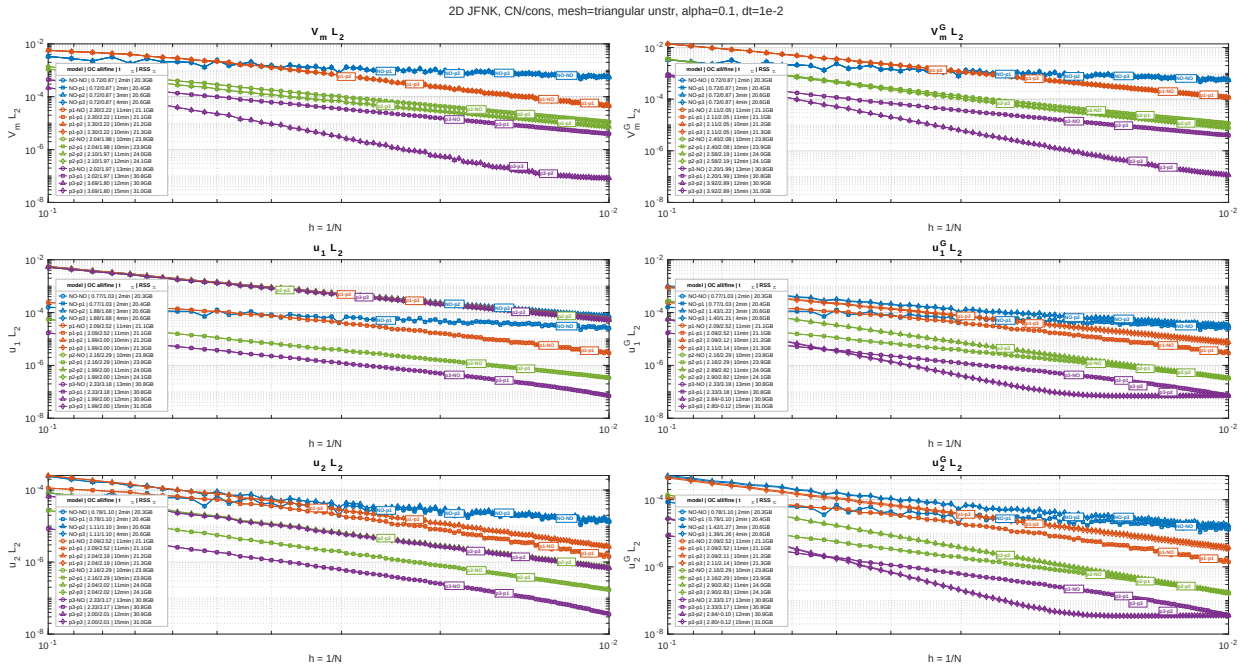


Figure 23: 2D convergence block for JFNK, triangular unstr mesh, $\alpha = 0.1$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

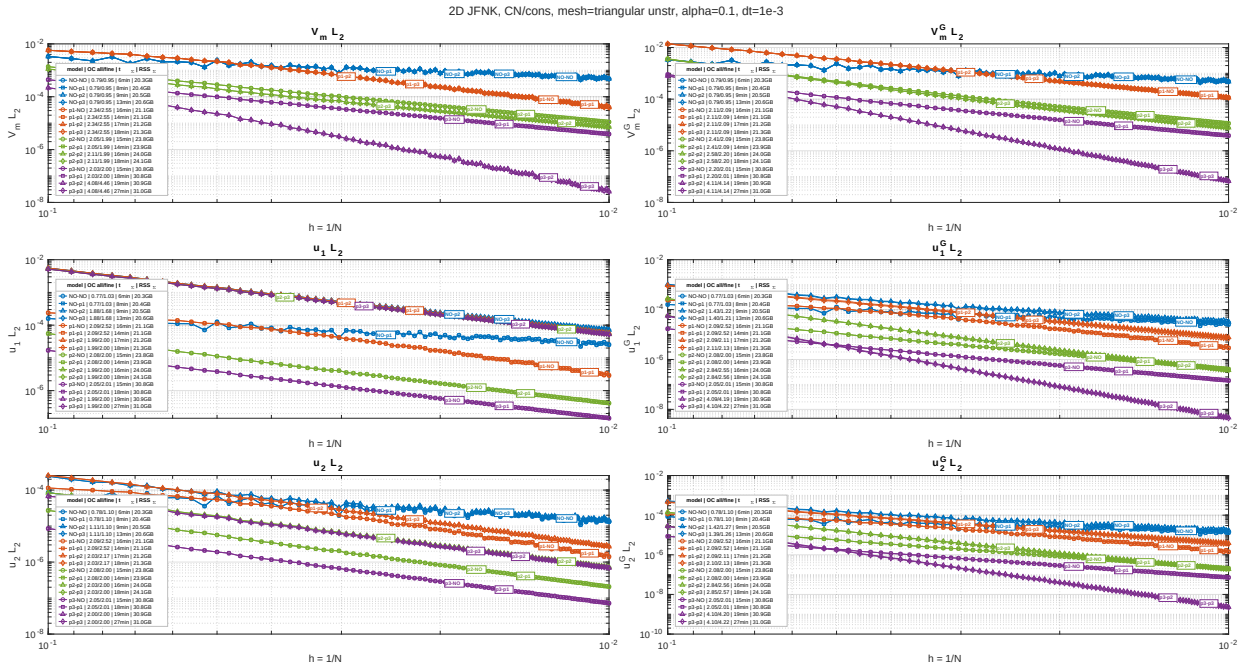


Figure 24: 2D convergence block for JFNK, triangular unstr mesh, $\alpha = 0.1$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

11.2 Two-dimensional results: comments

The current 2D tables fix the time scheme and mass matrix to `crankNicolson` and `consistent`. Under that fixed discretisation, `Picard`, `diagonalIion`, and `JFNK` produce the same observed convergence orders to the precision reported in the tables. The practical distinction in this MMS sweep is therefore computational cost and nonlinear robustness rather than asymptotic accuracy.

On the structured hexa mesh, the clearest high-order verification remains `p3` voltage reconstruction paired with `p2` or `p3` source/state quadrature. With $\Delta t = 10^{-3}$ these rows recover approximately fourth-order reconstructed voltage and state diagnostics over $N = 10$ –100, and the fine-mesh fits stay close to fourth order.

The unstructured triangular mesh is more sensitive to the selected reconstruction pair and to the time step. Splitting the plots by α makes this easier to read: the $\alpha = 0$ and $\alpha = 0.1$ cases no longer share the same axes, while the inline labels keep the model pair visible without relying only on the legend.

11.3 Three-dimensional manufactured problem

The 3D tables and figures in this subsection are generated from `/home/pablo/OpenFOAM/pablo-v2312/run/tutorials_electro/highOrderManufacturedFDImplicitPETSc/results/example0/3D_run_20260522_235518/`. All entries use `crankNicolson` time integration and the `consistent` mass matrix. The convergence fits use $N = 10$ –30 and $N = 20$ –30, since the 3D sweep does not reach the finer 2D meshes.

11.3.1 Picard, crankNicolson, consistent

Table 5: 3D manufactured results for `Picard`, `crankNicolson`, and the `consistent` mass matrix. The first six convergence columns are log–log fits over meshes $N = 10$ –30; the next six columns are fits over $N = 20$ –30. The time column is the sum of the reported wall-clock total over all mesh runs in the row, and the memory column is the sum of the reported peak resident memory in MB over the same mesh runs.

$V_m - I_{\text{ion}}$	Mesh type	α	Δt [s]	Fit $N = 10-30$						Fit $N = 20-30$						t_{Σ} [s]	RSS $_{\Sigma}$ [MB]
				V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G		
NO-NO	hexa	0.0	10^{-2}	1.985	1.985	1.994	1.994	1.994	1.994	1.993	1.993	2.009	2.009	2.009	2.009	13.701	4034.4
			10^{-3}	1.985	1.985	1.985	1.985	1.985	1.985	1.993	1.993	1.993	1.993	1.993	1.993	86.614	4035.7
		0.1	10^{-2}	1.436	1.436	1.297	1.297	1.306	1.306	1.411	1.411	1.214	1.214	1.223	1.223	22.097	6312.2
	10^{-3}		1.438	1.438	1.293	1.293	1.302	1.302	1.413	1.413	1.211	1.211	1.220	1.220	67.106	6315.1	
	tetra unstr	0.0	10^{-2}	0.664	0.664	0.375	0.375	0.373	0.373	0.826	0.826	0.617	0.617	0.616	0.616	21.144	4988.3
			10^{-3}	0.663	0.663	0.377	0.377	0.375	0.375	0.829	0.829	0.617	0.617	0.617	0.617	135.14	4989.6
0.1		10^{-2}	0.677	0.677	0.398	0.398	0.396	0.396	0.823	0.823	0.629	0.629	0.628	0.628	105.72	12055.9	
	10^{-3}	0.677	0.677	0.400	0.400	0.398	0.398	0.828	0.828	0.629	0.629	0.629	0.629	252.56	12057.8		
NO-p1	hexa	0.0	10^{-2}	1.984	1.984	1.992	2.072	1.992	2.077	1.992	1.992	1.996	2.038	1.997	2.041	28.095	4465.1
			10^{-3}	1.985	1.985	1.992	2.071	1.992	2.077	1.993	1.993	1.996	2.038	1.996	2.041	168.28	4465.5
		0.1	10^{-2}	1.459	1.459	1.992	2.073	1.993	2.079	1.429	1.429	1.996	2.038	1.996	2.041	37.380	6564.9
	10^{-3}		1.461	1.461	1.992	2.073	1.992	2.079	1.431	1.431	1.996	2.038	1.995	2.041	162.93	6564.0	
	tetra unstr	0.0	10^{-2}	0.664	0.664	0.375	0.375	0.373	0.373	0.826	0.826	0.617	0.617	0.616	0.616	29.082	5234.8
			10^{-3}	0.663	0.663	0.377	0.377	0.375	0.375	0.829	0.829	0.617	0.617	0.617	0.617	140.66	5234.5
0.1		10^{-2}	0.677	0.677	0.398	0.398	0.396	0.396	0.823	0.823	0.629	0.629	0.628	0.628	97.186	12251.2	
	10^{-3}	0.677	0.677	0.400	0.400	0.398	0.398	0.828	0.828	0.629	0.629	0.629	0.629	230.04	12251.4		
NO-p2	hexa	0.0	10^{-2}	1.983	1.983	1.984	2.014	1.985	2.018	1.992	1.992	1.992	2.007	1.993	2.009	72.453	5644.5
			10^{-3}	1.984	1.984	1.984	2.014	1.984	2.018	1.992	1.992	1.992	2.007	1.993	2.009	438.08	5647.7
		0.1	10^{-2}	1.482	1.482	1.984	2.015	1.985	2.018	1.447	1.447	1.992	2.007	1.993	2.010	84.100	7214.1
	10^{-3}		1.483	1.483	1.984	2.014	1.985	2.018	1.450	1.450	1.992	2.007	1.992	2.009	441.07	7215.9	
	tetra unstr	0.0	10^{-2}	0.664	0.664	1.832	0.861	0.516	0.859	0.826	0.826	1.792	0.867	0.652	0.866	43.013	5699.3
			10^{-3}	0.663	0.663	1.832	0.862	0.518	0.860	0.829	0.829	1.792	0.867	0.653	0.866	253.95	5704.6
0.1		10^{-2}	0.677	0.677	1.844	0.883	0.546	0.881	0.823	0.823	1.812	0.884	0.668	0.883	115.92	12501.1	
	10^{-3}	0.677	0.677	1.843	0.883	0.547	0.882	0.828	0.828	1.812	0.884	0.668	0.883	327.91	12501.2		
NO-p3	hexa	0.0	10^{-2}	1.983	1.983	1.983	2.010	1.984	2.014	1.992	1.992	1.992	2.005	1.993	2.008	159.70	7997.1
			10^{-3}	1.984	1.984	1.983	2.010	1.984	2.014	1.992	1.992	1.992	2.005	1.992	2.008	1011.8	7998.2
		0.1	10^{-2}	1.482	1.482	1.984	2.011	1.985	2.015	1.447	1.447	1.992	2.006	1.992	2.008	156.73	9198.0
	10^{-3}		1.483	1.483	1.984	2.011	1.984	2.015	1.450	1.450	1.992	2.005	1.992	2.008	957.54	9195.7	
	tetra unstr	0.0	10^{-2}	0.664	0.664	1.828	0.863	0.513	0.862	0.826	0.826	1.790	0.868	0.651	0.868	71.823	6602.5
			10^{-3}	0.663	0.663	1.828	0.864	0.514	0.862	0.829	0.829	1.790	0.869	0.652	0.868	457.00	6601.5
0.1		10^{-2}	0.677	0.677	1.840	0.885	0.542	0.884	0.823	0.823	1.810	0.885	0.667	0.885	146.28	12996.8	
	10^{-3}	0.677	0.677	1.840	0.885	0.543	0.884	0.828	0.828	1.810	0.886	0.667	0.885	564.41	13000.6		
p1-NO	hexa	0.0	10^{-2}	1.967	2.095	1.927	1.927	1.930	1.930	1.919	2.125	1.838	1.838	1.845	1.845	77.496	7145.3
			10^{-3}	1.967	2.095	1.924	1.924	1.927	1.927	1.920	2.125	1.833	1.833	1.840	1.840	155.87	7144.0
		0.1	10^{-2}	2.067	2.093	2.047	2.047	2.048	2.048	2.066	2.124	2.033	2.033	2.035	2.035	67.389	8698.8

		Fit $N = 10-30$						Fit $N = 20-30$									
$V_m - I_{\text{ion}}$	Mesh type	α	Δt [s]	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	t_Σ [s]	RSS $_\Sigma$ [MB]
p1-p1	tetra unstr	0.0	10^{-3}	2.067	2.093	2.044	2.044	2.044	2.044	2.067	2.124	2.028	2.028	2.029	2.029	136.81	8698.4
			10^{-2}	1.474	1.866	1.140	1.140	1.142	1.142	1.771	1.928	1.419	1.419	1.428	1.428	207.91	9888.9
			10^{-3}	1.475	1.866	1.142	1.142	1.144	1.144	1.768	1.927	1.422	1.422	1.432	1.432	398.80	9891.6
		0.1	10^{-2}	1.613	1.909	1.262	1.262	1.264	1.264	1.889	1.963	1.556	1.556	1.558	1.558	209.36	11566.5
			10^{-3}	1.612	1.909	1.265	1.265	1.267	1.267	1.884	1.962	1.558	1.558	1.561	1.561	397.09	11569.4
			10^{-3}	1.967	2.095	1.992	2.157	1.999	2.164	1.919	2.125	1.997	2.201	2.001	2.201	87.931	7353.8
	hexa	0.0	10^{-2}	1.968	2.095	1.992	2.157	1.999	2.164	1.920	2.125	1.997	2.201	2.001	2.200	254.63	7353.9
			10^{-3}	2.067	2.093	1.993	2.155	2.002	2.162	2.066	2.124	1.997	2.199	2.005	2.199	78.820	8903.7
			10^{-3}	2.067	2.093	1.993	2.155	2.001	2.162	2.067	2.124	1.997	2.199	2.004	2.199	222.49	8905.5
		0.1	10^{-2}	1.474	1.866	1.140	1.140	1.142	1.142	1.771	1.928	1.419	1.419	1.428	1.428	222.68	9942.5
			10^{-3}	1.475	1.866	1.142	1.142	1.144	1.144	1.768	1.927	1.422	1.422	1.432	1.432	435.69	9944.3
			10^{-3}	1.613	1.909	1.262	1.262	1.264	1.264	1.889	1.963	1.556	1.556	1.558	1.558	219.88	11623.1
p1-p2	tetra unstr	0.0	10^{-3}	1.612	1.909	1.265	1.265	1.267	1.267	1.884	1.962	1.558	1.558	1.561	1.561	427.55	11621.8
			10^{-2}	1.968	2.095	1.984	2.094	1.989	2.098	1.920	2.125	1.993	2.129	1.997	2.128	127.68	8029.7
			10^{-3}	1.968	2.095	1.984	2.094	1.989	2.098	1.921	2.125	1.993	2.129	1.996	2.128	481.37	8024.5
		0.1	10^{-2}	2.067	2.093	1.985	2.093	1.991	2.096	2.066	2.124	1.993	2.128	1.998	2.126	120.62	9552.9
			10^{-3}	2.067	2.093	1.985	2.092	1.991	2.096	2.067	2.124	1.993	2.128	1.998	2.126	472.63	9555.5
			10^{-3}	1.475	1.866	1.918	1.862	1.726	1.866	1.771	1.928	1.953	1.877	1.777	1.880	215.29	10400.8
	hexa	0.0	10^{-2}	1.475	1.866	1.918	1.862	1.726	1.866	1.768	1.927	1.953	1.877	1.777	1.880	493.81	10403.2
			10^{-3}	1.613	1.909	1.920	1.897	1.790	1.900	1.889	1.963	1.955	1.921	1.848	1.922	230.12	11946.2
			10^{-3}	1.612	1.909	1.920	1.897	1.790	1.900	1.884	1.962	1.955	1.921	1.849	1.923	531.81	11946.4
		0.1	10^{-2}	1.968	2.095	1.984	2.091	1.989	2.094	1.920	2.125	1.993	2.126	1.996	2.125	195.10	10159.3
			10^{-3}	1.968	2.095	1.984	2.091	1.989	2.094	1.921	2.125	1.993	2.126	1.996	2.124	900.71	10144.0
			10^{-3}	2.067	2.093	1.984	2.089	1.990	2.093	2.066	2.124	1.993	2.124	1.998	2.123	195.10	10948.2
p1-p3	tetra unstr	0.0	10^{-2}	2.067	2.093	1.984	2.089	1.990	2.093	2.067	2.124	1.993	2.124	1.998	2.123	963.73	10940.8
			10^{-3}	1.475	1.866	1.914	1.870	1.723	1.874	1.771	1.928	1.951	1.889	1.775	1.892	264.06	11323.2
			10^{-3}	1.475	1.866	1.914	1.870	1.723	1.873	1.768	1.927	1.951	1.889	1.775	1.892	780.16	11337.6
		0.1	10^{-2}	1.613	1.909	1.916	1.900	1.786	1.903	1.889	1.963	1.953	1.927	1.846	1.929	256.05	12695.5
			10^{-3}	1.612	1.909	1.916	1.900	1.786	1.903	1.884	1.962	1.953	1.927	1.847	1.929	679.56	12707.2
			10^{-3}	1.947	3.025	1.944	1.944	1.948	1.948	2.001	2.766	2.003	2.003	2.005	2.005	126.94	25863.3
	hexa	0.0	10^{-2}	1.947	3.025	1.944	1.944	1.948	1.948	2.001	2.766	2.000	2.000	2.003	2.003	210.78	25865.1
			10^{-3}	1.946	3.053	1.943	1.943	1.947	1.947	2.000	2.807	2.002	2.002	2.005	2.005	127.33	23781.8
			10^{-3}	1.946	3.053	1.941	1.941	1.946	1.946	2.000	2.807	1.999	1.999	2.002	2.002	207.61	23781.5
		0.1	10^{-2}	2.043	2.626	1.868	1.868	1.869	1.869	2.248	2.602	2.078	2.078	2.084	2.084	329.21	17527.9
			10^{-3}	2.044	2.627	1.867	1.867	1.868	1.868	2.248	2.603	2.077	2.077	2.084	2.084	654.46	17527.1
			10^{-3}	2.198	2.743	1.984	1.984	1.987	1.987	2.433	2.768	2.271	2.271	2.279	2.279	299.85	21384.0
p2-NO	tetra unstr	0.0	10^{-2}	2.198	2.744	1.985	1.985	1.987	1.987	2.429	2.767	2.271	2.271	2.279	2.279	551.02	21382.3
			10^{-3}	2.003	3.107	2.011	3.133	2.845	3.122	2.035	2.884	2.004	3.005	2.449	2.999	164.63	26074.9
			10^{-3}	2.004	3.107	2.011	3.133	2.844	3.122	2.035	2.884	2.004	3.004	2.447	2.998	435.91	26067.6
		0.1	10^{-2}	2.008	3.133	2.011	3.142	2.888	3.131	2.037	2.925	2.004	3.021	2.476	3.015	138.88	23991.3
			10^{-3}	2.008	3.133	2.011	3.142	2.886	3.131	2.037	2.925	2.004	3.021	2.473	3.014	313.10	23991.7
			10^{-3}	2.043	2.626	1.868	1.868	1.869	1.869	2.248	2.602	2.078	2.078	2.084	2.084	342.41	17573.5
	hexa	0.0	10^{-2}	2.044	2.627	1.867	1.867	1.868	1.868	2.248	2.603	2.077	2.077	2.084	2.084	657.39	17584.1
			10^{-3}	2.198	2.743	1.984	1.984	1.987	1.987	2.433	2.768	2.271	2.271	2.279	2.279	330.06	21400.4
			10^{-3}	2.198	2.744	1.985	1.985	1.987	1.987	2.429	2.767	2.271	2.271	2.279	2.279	692.09	21405.3
		0.1	10^{-2}	2.041	3.142	2.004	3.230	2.898	3.215	2.059	2.936	2.001	3.092	2.432	3.083	236.16	26715.1
			10^{-3}	2.041	3.142	2.004	3.230	2.897	3.215	2.060	2.936	2.001	3.092	2.431	3.082	759.22	26717.6
			10^{-3}	2.050	3.166	2.004	3.236	2.920	3.222	2.064	2.975	2.001	3.105	2.440	3.095	179.04	24639.4
p2-p2	tetra unstr	0.0	10^{-2}	2.051	3.166	2.004	3.236	2.920	3.221	2.064	2.975	2.001	3.105	2.439	3.094	623.79	24635.8
			10^{-3}	2.103	2.704	1.928	2.794	2.100	2.796	2.384	2.719	1.960	2.779	2.102	2.784	342.91	17961.8
			10^{-3}	2.105	2.705	1.928	2.794	2.100	2.795	2.384	2.720	1.960	2.779	2.102	2.783	887.93	17964.1
		0.1	10^{-2}	2.349	2.824	1.928	2.831	2.146	2.833	2.732	2.904	1.960	2.861	2.166	2.866	336.90	21691.1
			10^{-3}	2.350	2.825	1.928	2.831	2.146	2.833	2.727	2.903	1.960	2.862	2.167	2.866	776.48	21693.1
			10^{-3}	2.041	3.142	2.003	3.235	2.897	3.220	2.059	2.936	2.000	3.095	2.432	3.085	355.85	28015.7
	hexa	0.0	10^{-2}	2.041	3.142	2.003	3.235	2.896	3.220	2.060	2.936	2.000	3.095	2.430	3.085	1431.6	28015.9
			10^{-3}	2.050	3.166	2.003	3.242	2.920	3.227	2.064	2.975	2.000	3.108	2.439	3.097	288.86	25931.7
			10^{-3}	2.051	3.166	2.003	3.241	2.919	3.226	2.064	2.975	2.000	3.108	2.438	3.097	1111.8	25928.3
		0.1	10^{-2}	2.103	2.704	1.924	2.802	2.082	2.802	2.384	2.719	1.958	2.769	2.088	2.773	414.66	18813.0
			10^{-3}	2.105	2.705	1.924	2.801	2.082	2.802	2.384	2.720	1.958	2.768	2.089	2.773	1108.6	18824.2
			10^{-3}	2.349	2.824	1.924	2.847	2.128	2.847	2.732	2.904	1.958	2.864	2.152	2.868	497.14	22280.7
p2-p3	tetra unstr	0.0	10^{-2}	2.350	2.825	1.924	2.846	2.128	2.847	2.727	2.903	1.958	2.864	2.153	2.868	1250.4	22275.1
			10^{-3}	2.313	3.204	2.308	2.308	2.312	2.312	2.193	2.830	2.197	2.197	2.200	2.200	897.53	77071.6

$V_m - I_{\text{ion}}$	Mesh type	α	Δt [s]	Fit $N = 10-30$						Fit $N = 20-30$						t_Σ [s]	RSS $_\Sigma$ [MB]
				V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G		
		0.1	10^{-3}	3.332	3.813	1.994	3.884	2.452	3.904	3.260	3.882	1.995	3.956	2.170	3.967	2130.9	79212.1
			10^{-2}	3.348	3.823	1.993	3.890	2.446	3.909	3.272	3.896	1.995	3.966	2.167	3.976	769.90	79286.6
			10^{-3}	3.349	3.823	1.993	3.889	2.446	3.908	3.275	3.896	1.995	3.962	2.167	3.973	1565.6	79288.0
		0.0	10^{-2}	3.195	3.776	1.923	3.842	2.021	3.844	3.460	3.809	1.958	3.837	1.974	3.844	1068.5	46006.7
			10^{-3}	3.197	3.777	1.923	3.838	2.021	3.840	3.462	3.810	1.958	3.826	1.974	3.833	1853.5	46006.8
			10^{-2}	3.450	3.862	1.923	3.884	2.017	3.885	3.777	3.921	1.958	3.904	1.971	3.911	1068.8	54392.0
	tetra unstr	0.1	10^{-3}	3.451	3.863	1.923	3.879	2.017	3.880	3.775	3.921	1.958	3.893	1.971	3.900	1872.5	54389.9
		0.0															

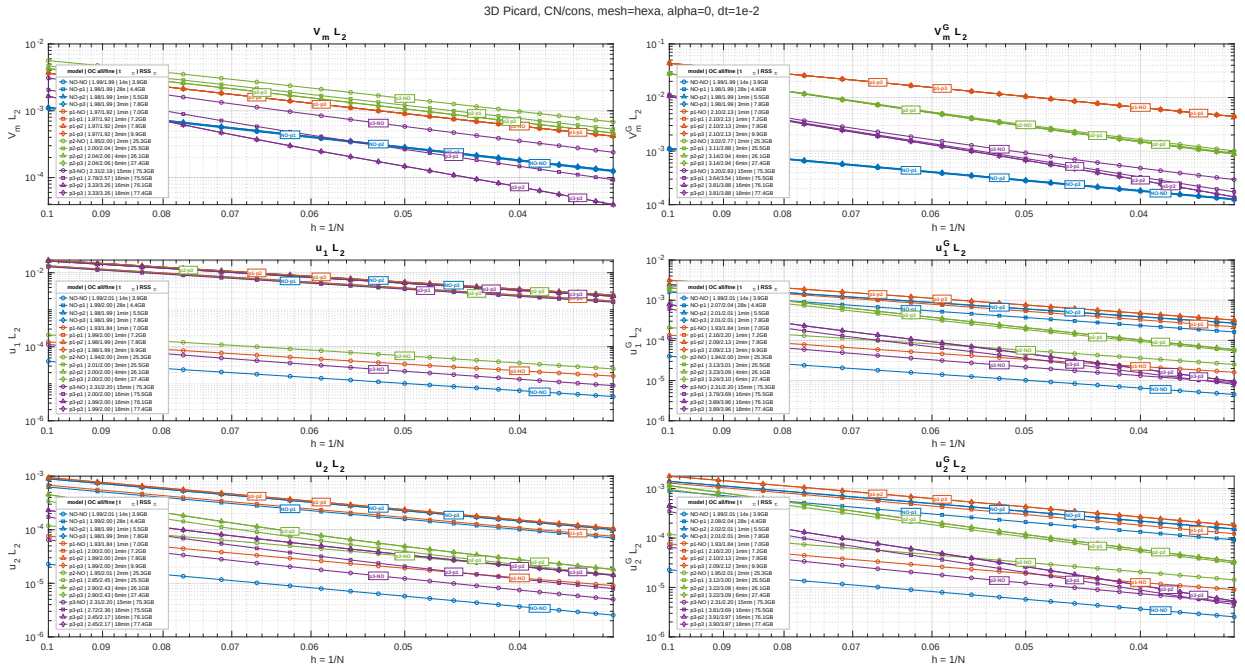


Figure 25: 3D convergence block for Picard, hexa mesh, $\alpha = 0$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the $V_m - I_{\text{ion}}$ model pair.

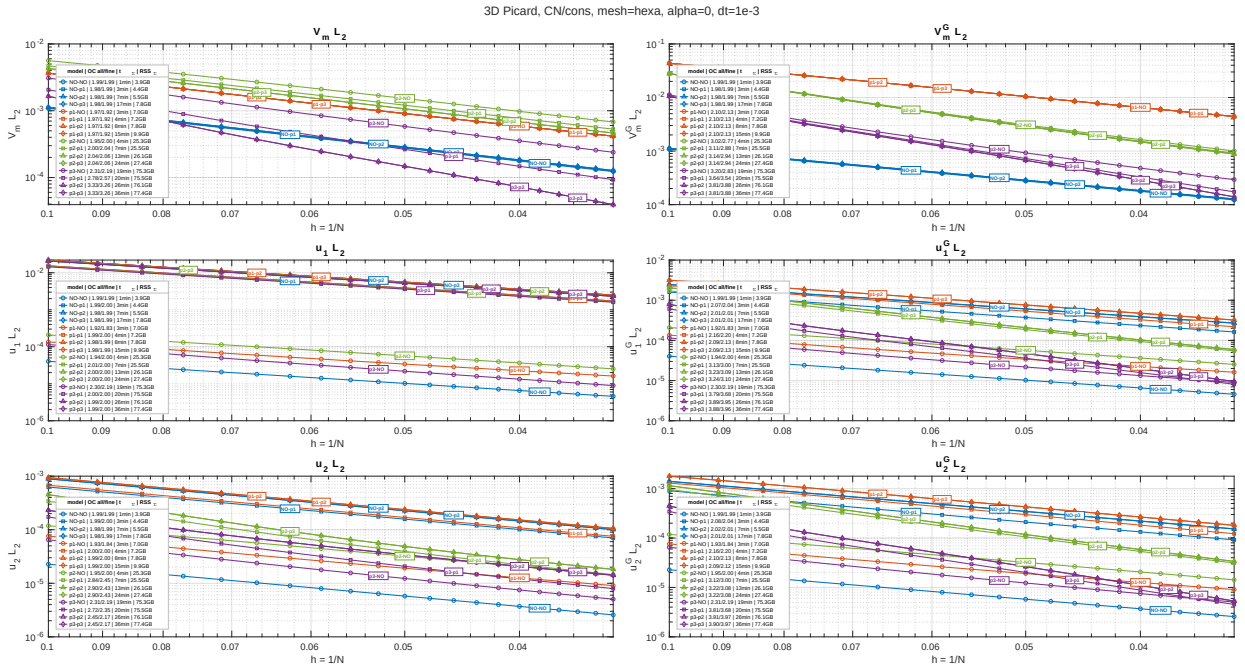


Figure 26: 3D convergence block for Picard, hexa mesh, $\alpha = 0$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m-I_{ion} model pair.

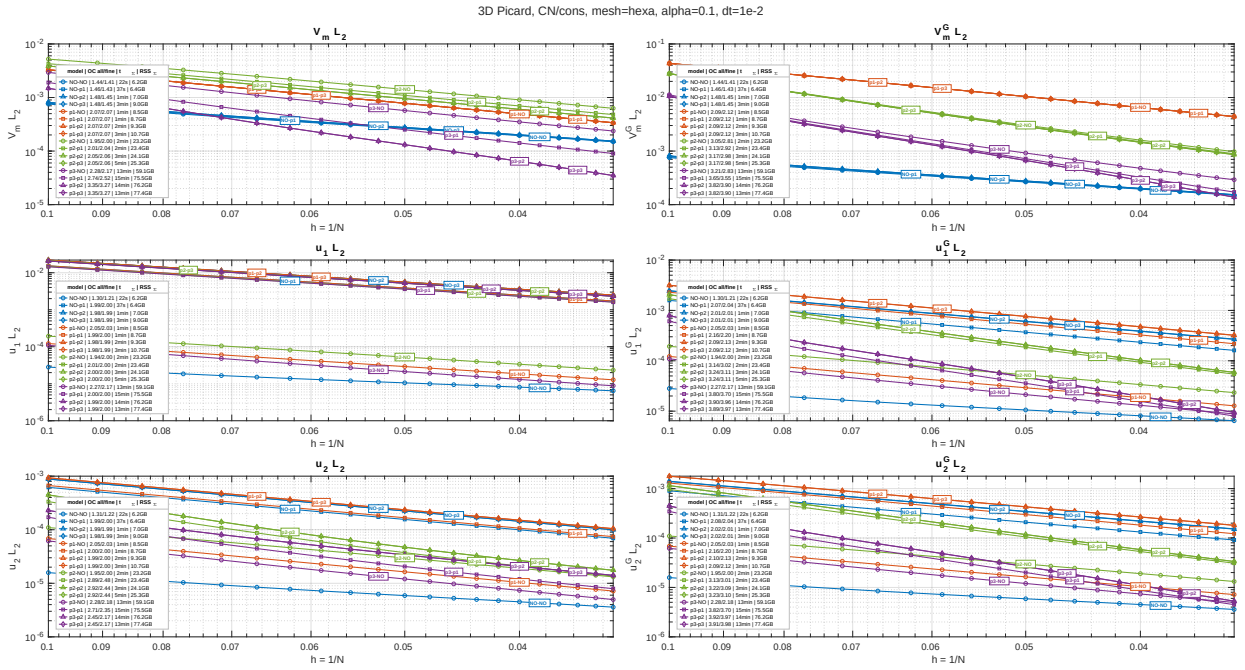


Figure 27: 3D convergence block for Picard, hexa mesh, $\alpha = 0.1$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the $V_m - I_{\text{ion}}$ model pair.

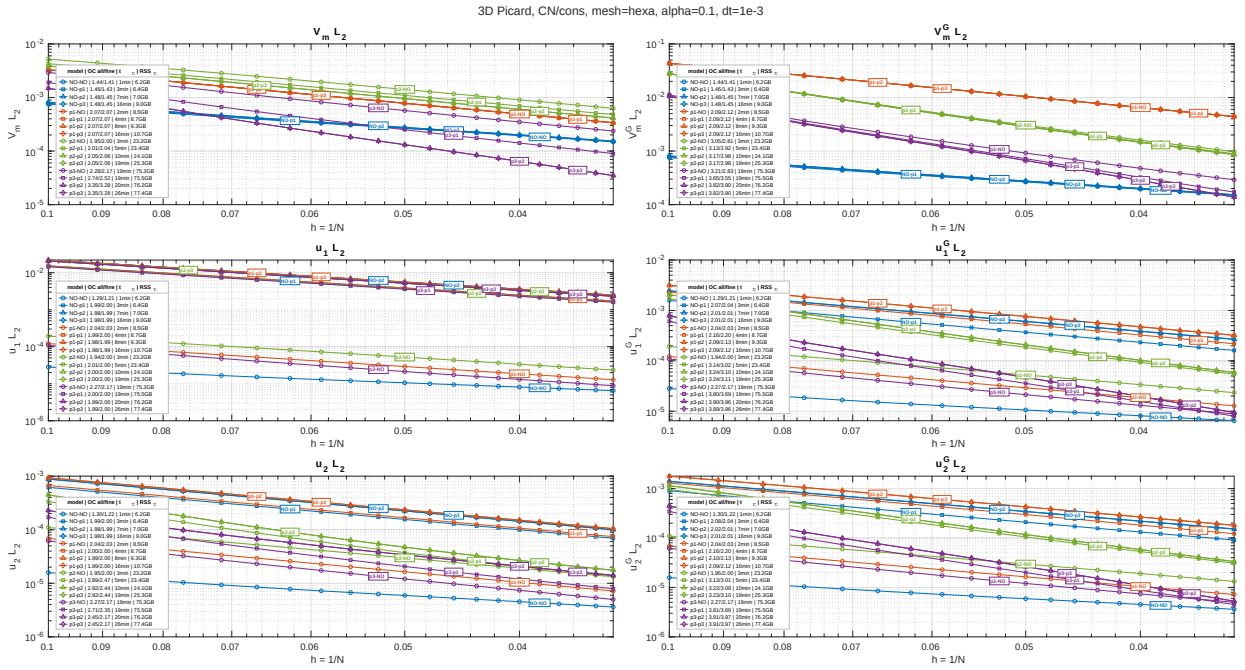


Figure 28: 3D convergence block for Picard, hexa mesh, $\alpha = 0.1$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the $V_m - I_{\text{ion}}$ model pair.

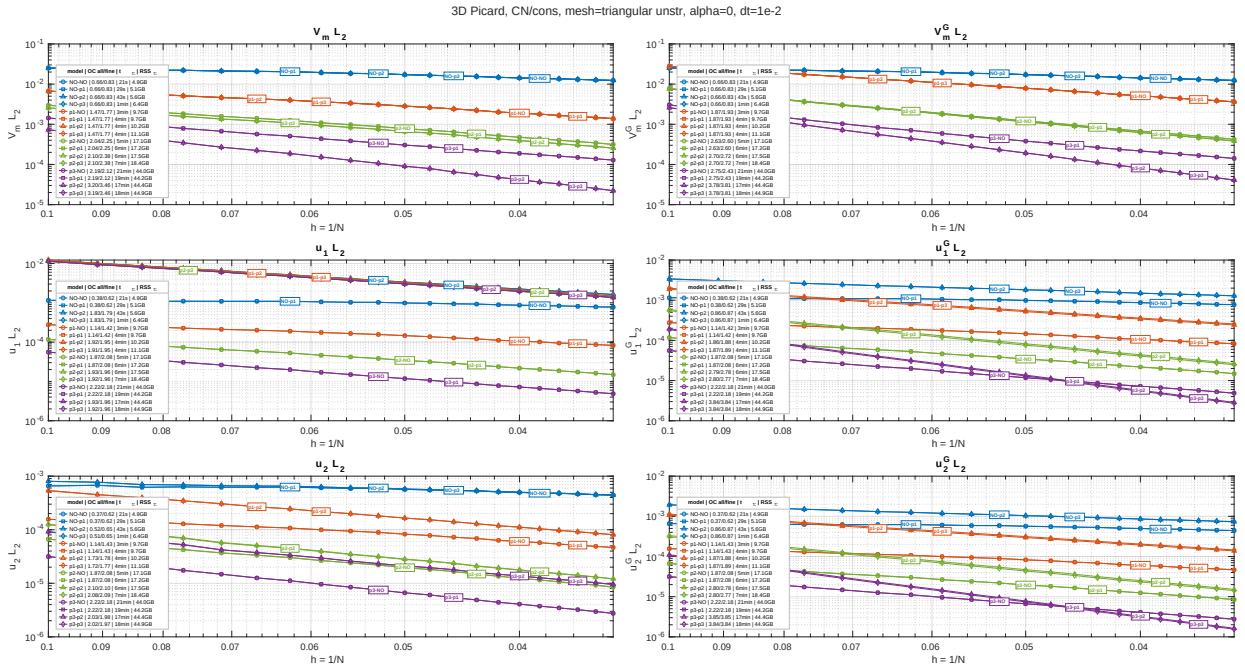


Figure 29: 3D convergence block for Picard, tetra unstr mesh, $\alpha = 0$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

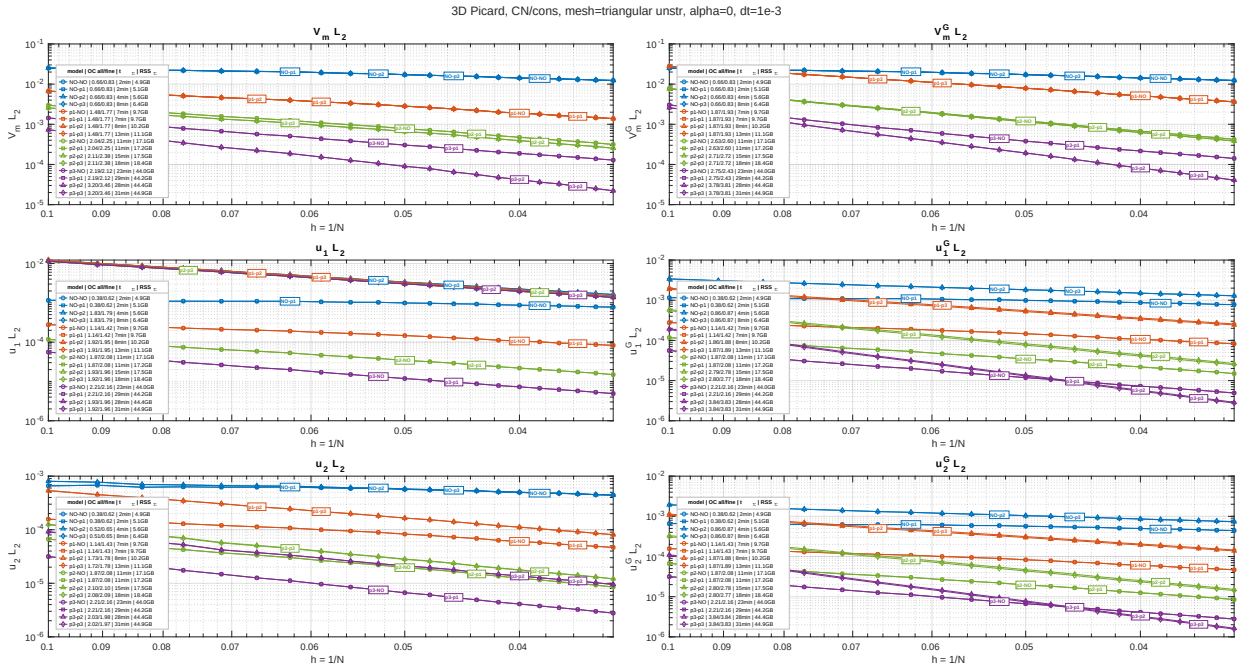


Figure 30: 3D convergence block for Picard, tetra unstr mesh, $\alpha = 0$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

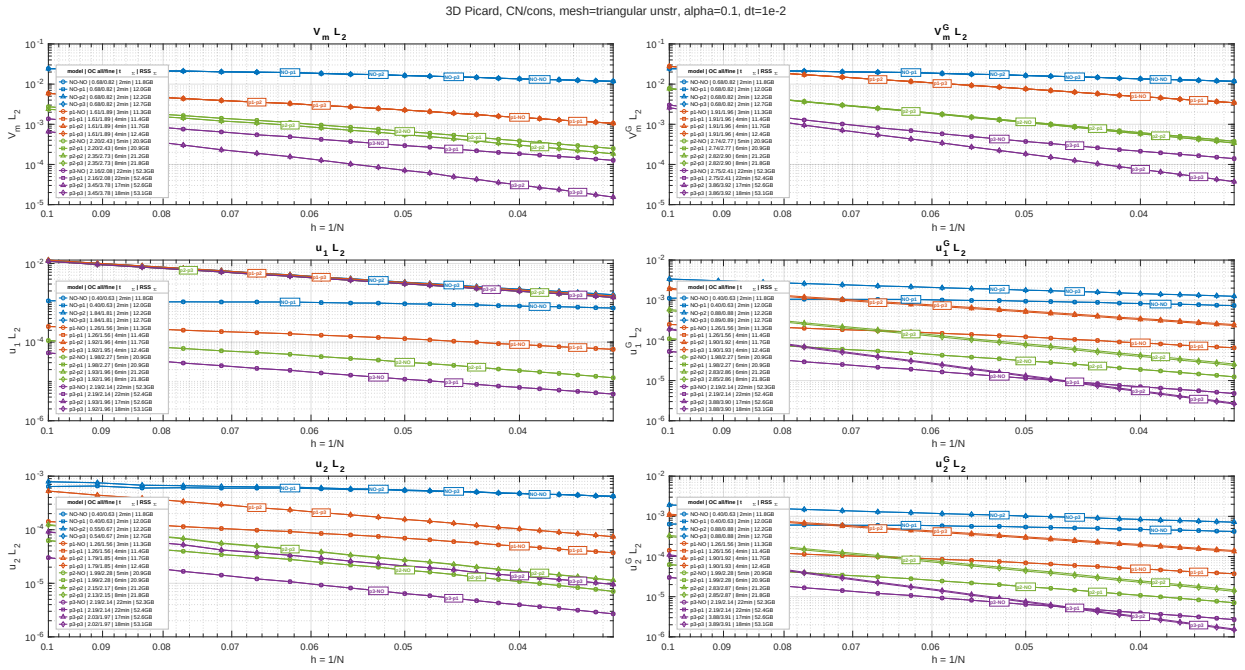


Figure 31: 3D convergence block for Picard, tetra unstr mesh, $\alpha = 0.1$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m – I_{ion} model pair.

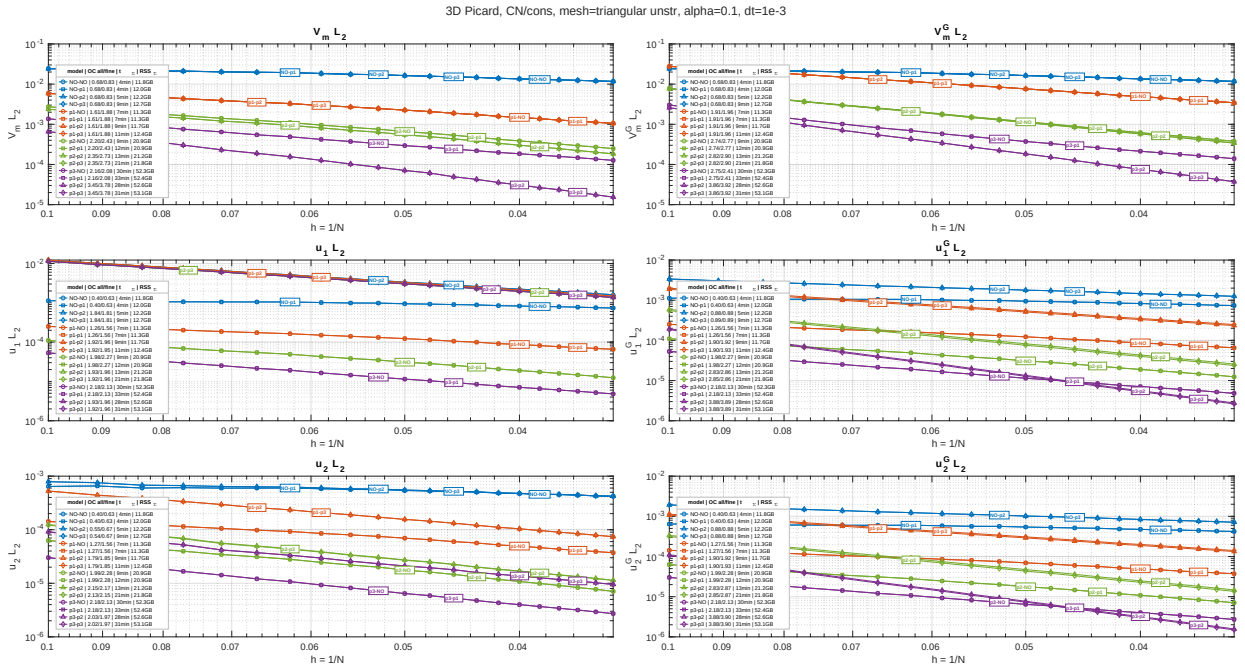


Figure 32: 3D convergence block for Picard, tetra unstr mesh, $\alpha = 0.1$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

11.3.2 diagonalIion, crankNicolson, consistent

Table 6: 3D manufactured results for `diagonalIion`, `crankNicolson`, and the `consistent` mass matrix. The first six convergence columns are log-log fits over meshes $N = 10$ –30; the next six columns are fits over $N = 20$ –30. The time column is the sum of the reported wall-clock total over all mesh runs in the row, and the memory column is the sum of the reported peak resident memory in MB over the same mesh runs.

				Fit $N = 10$ –30						Fit $N = 20$ –30							
$V_m - I_{\text{ion}}$	Mesh type	α	Δt [s]	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	t_Σ [s]	RSS $_\Sigma$ [MB]
NO–NO	hexa	0.0	10^{-2}	1.985	1.985	1.994	1.994	1.994	1.994	1.993	1.993	2.009	2.009	2.009	2.009	17.839	4049.9
			10^{-3}	1.985	1.985	1.985	1.985	1.985	1.985	1.993	1.993	1.993	1.993	1.993	1.993	122.72	4051.0
		0.1	10^{-2}	1.436	1.436	1.297	1.297	1.306	1.306	1.411	1.411	1.214	1.214	1.223	1.223	31.231	6313.3
	10^{-3}		1.438	1.438	1.293	1.293	1.302	1.302	1.413	1.413	1.211	1.211	1.220	1.220	157.38	6313.0	
	tetra unstr	0.0	10^{-2}	0.664	0.664	0.375	0.375	0.373	0.373	0.826	0.826	0.617	0.617	0.616	0.616	38.256	5022.6
			10^{-3}	0.663	0.663	0.377	0.377	0.375	0.375	0.829	0.829	0.617	0.617	0.617	0.617	201.73	5025.8
NO–p1	hexa	0.0	10^{-2}	0.677	0.677	0.398	0.398	0.396	0.396	0.823	0.823	0.629	0.629	0.628	0.628	152.32	12058.9
			10^{-3}	0.677	0.677	0.400	0.400	0.398	0.398	0.828	0.828	0.629	0.629	0.629	0.629	592.24	12057.2
		0.1	10^{-2}	1.984	1.984	1.992	2.072	1.992	2.077	1.992	1.992	1.996	2.038	1.997	2.041	59.598	4506.2
	10^{-3}		1.985	1.985	1.992	2.071	1.992	2.077	1.993	1.993	1.996	2.038	1.996	2.041	299.85	4511.0	
	tetra unstr	0.0	10^{-2}	1.459	1.459	1.992	2.073	1.993	2.079	1.429	1.429	1.996	2.038	1.996	2.041	71.966	6563.7
			10^{-3}	1.461	1.461	1.992	2.073	1.992	2.079	1.431	1.431	1.996	2.038	1.995	2.041	345.03	6562.4
NO–p2	hexa	0.0	10^{-2}	0.664	0.664	0.375	0.375	0.373	0.373	0.826	0.826	0.617	0.617	0.616	0.616	51.433	5256.8
			10^{-3}	0.663	0.663	0.377	0.377	0.375	0.375	0.829	0.829	0.617	0.617	0.617	0.617	254.13	5262.5
		0.1	10^{-2}	0.677	0.677	0.398	0.398	0.396	0.396	0.823	0.823	0.629	0.629	0.628	0.628	147.95	12251.1
	10^{-3}		0.677	0.677	0.400	0.400	0.398	0.398	0.828	0.828	0.629	0.629	0.629	0.629	586.17	12250.4	
	tetra unstr	0.0	10^{-2}	1.983	1.983	1.984	2.014	1.985	2.018	1.992	1.992	1.992	2.007	1.993	2.009	158.85	5951.4
			10^{-3}	1.984	1.984	1.984	2.014	1.984	2.018	1.992	1.992	1.992	2.007	1.993	2.009	979.42	5953.0
NO–p3	hexa	0.0	10^{-2}	1.482	1.482	1.984	2.015	1.985	2.018	1.447	1.447	1.992	2.007	1.993	2.010	155.23	7217.8
			10^{-3}	1.483	1.483	1.984	2.015	1.985	2.018	1.450	1.450	1.992	2.007	1.992	2.009	849.58	7217.6
		0.1	10^{-2}	0.664	0.664	1.832	0.861	0.516	0.859	0.826	0.826	1.792	0.867	0.652	0.866	77.643	5716.7
	10^{-3}		0.663	0.663	1.832	0.862	0.518	0.860	0.829	0.829	1.792	0.867	0.653	0.866	441.41	5718.4	
	tetra unstr	0.0	10^{-2}	0.677	0.677	1.844	0.883	0.546	0.881	0.823	0.823	1.812	0.884	0.668	0.883	175.86	12501.7
			10^{-3}	0.677	0.677	1.843	0.883	0.547	0.882	0.828	0.828	1.812	0.884	0.668	0.883	880.58	12497.6
p1–NO	hexa	0.0	10^{-2}	1.983	1.983	1.983	2.010	1.984	2.014	1.992	1.992	1.992	2.005	1.993	2.008	399.48	8785.1
			10^{-3}	1.984	1.984	1.983	2.010	1.984	2.014	1.992	1.992	1.992	2.005	1.992	2.008	1784.3	8779.5
		0.1	10^{-2}	1.482	1.482	1.984	2.011	1.985	2.015	1.447	1.447	1.992	2.006	1.992	2.008	325.33	9791.0
	10^{-3}		1.483	1.483	1.984	2.011	1.984	2.015	1.450	1.450	1.992	2.005	1.992	2.008	1811.7	9791.1	
	tetra unstr	0.0	10^{-2}	0.664	0.664	1.828	0.863	0.513	0.862	0.826	0.826	1.790	0.868	0.651	0.868	138.65	6716.1
			10^{-3}	0.663	0.663	1.828	0.864	0.514	0.862	0.829	0.829	1.790	0.869	0.652	0.868	946.10	6720.5
p1–p1	hexa	0.0	10^{-2}	0.677	0.677	1.840	0.885	0.542	0.884	0.823	0.823	1.810	0.885	0.667	0.885	256.75	12995.9
			10^{-3}	0.677	0.677	1.840	0.885	0.543	0.884	0.828	0.828	1.810	0.886	0.667	0.885	1222.1	12996.5
		0.1	10^{-2}	1.967	2.095	1.927	1.927	1.930	1.930	1.919	2.125	1.838	1.838	1.845	1.845	76.120	7133.9
	10^{-3}		1.967	2.095	1.924	1.924	1.927	1.927	1.920	2.125	1.833	1.833	1.840	1.840	206.83	7134.2	
	tetra unstr	0.0	10^{-2}	2.067	2.093	2.047	2.047	2.048	2.048	2.066	2.124	2.033	2.033	2.035	2.035	77.548	8693.9
			10^{-3}	2.067	2.093	2.044	2.044	2.044	2.044	2.067	2.124	2.028	2.028	2.029	2.029	219.86	8694.8
p1–p2	hexa	0.0	10^{-2}	1.474	1.866	1.140	1.140	1.142	1.142	1.771	1.928	1.419	1.419	1.428	1.428	261.81	10336.8
			10^{-3}	1.475	1.866	1.142	1.142	1.144	1.144	1.768	1.927	1.422	1.422	1.432	1.432	655.41	10344.2
		0.1	10^{-2}	1.613	1.909	1.262	1.262	1.264	1.264	1.889	1.963	1.556	1.556	1.558	1.558	251.13	11875.1
	10^{-3}		1.612	1.909	1.265	1.265	1.267	1.267	1.884	1.962	1.558	1.558	1.561	1.561	654.84	11878.0	
	tetra unstr	0.0	10^{-2}	1.967	2.095	1.992	2.157	1.999	2.164	1.919	2.125	1.997	2.201	2.001	2.201	104.41	7341.0
			10^{-3}	1.968	2.095	1.992	2.157	1.999	2.164	1.920	2.125	1.997	2.201	2.001	2.200	357.41	7338.5
p1–p3	hexa	0.0	10^{-2}	2.067	2.093	1.993	2.155	2.002	2.162	2.066	2.124	1.997	2.199	2.005	2.199	109.30	8899.4
			10^{-3}	2.067	2.093	1.993	2.155	2.001	2.162	2.067	2.124	1.997	2.199	2.004	2.199	375.47	8901.2
		0.1	10^{-2}	1.474	1.866	1.140	1.140	1.142	1.142	1.771	1.928	1.419	1.419	1.428	1.428	247.77	10396.4
	10^{-3}		1.475	1.866	1.142	1.142	1.144	1.144	1.768	1.927	1.422	1.422	1.432	1.432	610.55	10400.1	
	tetra unstr	0.0	10^{-2}	1.613	1.909	1.262	1.262	1.264	1.264	1.889	1.963	1.556	1.556	1.558	1.558	222.16	11931.7
			10^{-3}	1.612	1.909	1.265	1.265	1.267	1.267	1.884	1.962	1.558	1.558	1.561	1.561	644.92	11931.5
p2–NO	hexa	0.0	10^{-2}	1.968	2.095	1.984	2.094	1.989	2.098	1.920	2.125	1.993	1.993	1.997	1.997	128.63	8084.3
			10^{-3}	1.968	2.095	1.984	2.094	1.989	2.098	1.921	2.125	1.993	1.993	1.996	1.996	128.63	8084.6
		0.1	10^{-2}	2.067	2.093	1.985	2.093	1.991	2.096	2.066	2.124	1.993	1.993	1.998	1.998	126.187	9545.7
	10^{-3}		2.067	2.093	1.985	2.092	1.991	2.096	2.067	2.124	1.993	1.993	1.998	1.998	852.43	9548.3	
	tetra unstr	0.0	10^{-2}	1.475	1.866	1.918	1.862	1.726	1.866	1.771	1.928	1.953	1.877	1.777	1.880	288.75	10853.6
			10^{-3}	1.475	1.866	1.918	1.862	1.726	1.866	1.768	1.927	1.953	1.877	1.777	1.880	810.89	10858.2
p2–p1	hexa	0.0	10^{-2}	1.613	1.909	1.920	1.897	1.790	1.900	1.889	1.963	1.955	1.921	1.848	1.922	250.54	12308.8
			10^{-3}	1.612	1.909	1.920	1.897	1.790	1.900	1.884	1.962	1.955	1.921	1.849	1.923	831.86	12353.3
		0.1	10^{-2}	1.968	2.095	1.984	2.091	1.989	2.094	1.920	2.125	1.993	1.993	1.996	1.996	125.374	10772.3
	10^{-3}		1.968	2.095	1.984	2.091	1.989	2.094	1.921	2.125	1.993	1.993	1.996	1.996	1939.9	10771.9	
	tetra unstr	0.0	10^{-2}	2.067	2.093	1.984	2.089	1.990	2.093	2.066	2.124	1.993	1.993	1.998	1.998	123.363	11158.1
			10^{-3}	2.067	2.093	1.984	2.089	1.990	2.093	2.067	2.124	1.993	1.993	1.998	1.998	123.363	11158.1
p2–p2	hexa	0.0	10^{-2}	1.475	1.866	1.914	1.870	1.723	1.874	1.771	1.928	1.951	1.889	1.775	1.892	341.86	11836.5
			10^{-3}	1.475	1.866	1.914	1.870	1.723	1.873	1.768	1.927	1.951	1.889	1.775	1.892	1182.8	11857.3
		0.1	10^{-2}	1.613	1.909	1.916	1.900	1.786	1.903	1.889	1.963	1.953	1.927	1.846	1.929	311.35	13265.5
	10^{-3}		1.612	1.909	1.916	1.900	1.786	1.903	1.884	1.962	1.953	1.927	1.847	1.929	1142.1	13225.7	
	tetra unstr	0.0	10^{-2}	1.947	3.025	1.944	1.944	1.948	1.948	2.001	2.766	2.003	2.003	2.005	2.005	159.49	25851.3
			10^{-3}	1.947	3.025	1.942	1.942	1.946	1.946	2.001	2.766	2.000	2.000	2.003	2.003	345.81	25850.3
p2–p3	hexa	0.0	10^{-2}	1.946	3.053	1.943	1.943	1.947	1.947	2.000	2.807	2.002	2.002	2.005	2.005	153.99	23778.1
			10^{-3}	1.946	3.053	1.941	1.941	1.946	1.946	2.000	2.807	1.999	1.999	2.002	2.002	333.19	23777.0
		0.1	10^{-2}	2.043	2.626	1.868	1.868	1.869	1.869	2.248	2.602	2.078	2.078	2.084	2.084	410.45	18237.1</

$V_m - I_{\text{ion}}$	Mesh type	α	Δt [s]	Fit $N = 10-30$						Fit $N = 20-30$						t_{Σ} [s]	RSS $_{\Sigma}$ [MB]
				V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G		
p2-p2	tetra unstr	0.1	10^{-3}	2.004	3.107	2.011	3.133	2.844	3.122	2.035	2.884	2.004	3.004	2.447	2.998	590.22	26061.8
			10^{-2}	2.008	3.133	2.011	3.142	2.888	3.131	2.037	2.925	2.004	3.021	2.476	3.015	171.87	23988.3
			10^{-3}	2.008	3.133	2.011	3.142	2.886	3.131	2.037	2.925	2.004	3.021	2.473	3.014	514.55	23988.2
		0.0	10^{-2}	2.043	2.626	1.868	1.868	1.869	1.869	2.248	2.602	2.078	2.078	2.084	2.084	419.78	18303.6
			10^{-3}	2.044	2.627	1.867	1.867	1.868	1.868	2.248	2.603	2.077	2.077	2.084	2.084	1154.9	18312.9
			10^{-2}	2.198	2.743	1.984	1.984	1.987	1.987	2.433	2.768	2.271	2.271	2.279	2.279	402.79	21700.4
	hexa	0.1	10^{-3}	2.198	2.744	1.985	1.985	1.987	1.987	2.429	2.767	2.271	2.271	2.279	2.279	1382.5	21701.0
			10^{-2}	2.041	3.142	2.004	3.230	2.898	3.215	2.059	2.936	2.001	3.092	2.432	3.083	291.55	26710.8
			10^{-3}	2.041	3.142	2.004	3.230	2.897	3.215	2.060	2.936	2.001	3.092	2.431	3.082	1167.8	26712.3
		0.0	10^{-2}	2.050	3.166	2.004	3.236	2.920	3.222	2.064	2.975	2.001	3.105	2.440	3.095	265.41	24629.7
			10^{-3}	2.051	3.166	2.004	3.236	2.920	3.221	2.064	2.975	2.001	3.105	2.439	3.094	1039.5	24632.4
			10^{-2}	2.103	2.704	1.928	2.794	2.100	2.796	2.384	2.719	1.960	2.779	2.102	2.784	421.04	18693.3
p2-p3	tetra unstr	0.1	10^{-3}	2.105	2.705	1.928	2.794	2.100	2.795	2.384	2.720	1.960	2.779	2.102	2.783	1472.4	18701.6
			10^{-2}	2.349	2.824	1.928	2.831	2.146	2.833	2.732	2.904	1.960	2.861	2.166	2.866	432.63	21997.2
			10^{-3}	2.350	2.825	1.928	2.831	2.146	2.833	2.727	2.903	1.960	2.862	2.167	2.866	1684.2	21997.0
		0.0	10^{-2}	2.041	3.142	2.003	3.235	2.897	3.220	2.059	2.936	2.000	3.095	2.432	3.085	524.46	28002.3
			10^{-3}	2.041	3.142	2.003	3.235	2.896	3.220	2.060	2.936	2.000	3.095	2.430	3.085	2138.9	28004.9
			10^{-2}	2.050	3.166	2.003	3.242	2.920	3.227	2.064	2.975	2.000	3.108	2.439	3.097	385.37	25931.2
	hexa	0.1	10^{-3}	2.051	3.166	2.003	3.241	2.919	3.226	2.064	2.975	2.000	3.108	2.438	3.097	1870.2	25931.7
			10^{-2}	2.103	2.704	1.924	2.802	2.082	2.802	2.384	2.719	1.958	2.769	2.088	2.773	463.12	19698.3
			10^{-3}	2.105	2.705	1.924	2.801	2.082	2.802	2.384	2.720	1.958	2.768	2.089	2.773	1914.1	19696.0
		0.0	10^{-2}	2.349	2.824	1.924	2.847	2.128	2.847	2.732	2.904	1.958	2.864	2.152	2.868	486.74	22652.1
			10^{-3}	2.350	2.825	1.924	2.846	2.128	2.847	2.727	2.903	1.958	2.864	2.153	2.868	2017.0	22649.7
			10^{-2}	2.313	3.204	2.308	2.308	2.312	2.312	2.193	2.830	2.197	2.197	2.200	2.200	784.33	77066.1
p3-NO	tetra unstr	0.1	10^{-3}	2.313	3.204	2.303	2.303	2.307	2.307	2.193	2.830	2.189	2.189	2.192	2.192	886.77	77052.6
			10^{-2}	2.278	3.207	2.274	2.274	2.278	2.278	2.169	2.826	2.174	2.174	2.176	2.176	737.90	76981.0
			10^{-3}	2.278	3.207	2.269	2.269	2.273	2.273	2.169	2.827	2.166	2.166	2.168	2.168	879.82	77137.1
		0.0	10^{-2}	2.189	2.753	2.216	2.216	2.217	2.217	2.121	2.431	2.176	2.176	2.179	2.179	1208.8	45193.0
			10^{-3}	2.189	2.753	2.208	2.208	2.209	2.209	2.121	2.431	2.162	2.162	2.165	2.165	2511.9	45194.0
			10^{-2}	2.155	2.754	2.192	2.192	2.193	2.193	2.080	2.410	2.142	2.142	2.144	2.144	1221.0	53420.2
	hexa	0.1	10^{-3}	2.155	2.755	2.184	2.184	2.185	2.185	2.080	2.410	2.128	2.128	2.130	2.130	2695.5	53563.8
			10^{-2}	2.780	3.644	2.003	3.792	2.717	3.810	2.572	3.538	2.000	3.687	2.355	3.691	1136.4	77129.5
			10^{-3}	2.781	3.644	2.003	3.789	2.716	3.807	2.573	3.538	2.000	3.679	2.354	3.683	1524.3	77272.0
		0.0	10^{-2}	2.737	3.653	2.003	3.799	2.707	3.817	2.522	3.546	2.000	3.695	2.347	3.700	826.77	77349.6
			10^{-3}	2.738	3.653	2.003	3.796	2.706	3.814	2.523	3.546	2.000	3.687	2.345	3.692	1268.9	77349.1
			10^{-2}	2.189	2.753	2.216	2.216	2.217	2.217	2.121	2.431	2.176	2.176	2.179	2.179	1530.2	45303.3
p3-p1	tetra unstr	0.1	10^{-3}	2.189	2.753	2.208	2.208	2.209	2.209	2.121	2.431	2.162	2.162	2.165	2.165	3571.4	45305.4
			10^{-2}	2.155	2.754	2.192	2.192	2.193	2.193	2.080	2.410	2.142	2.142	2.144	2.144	1458.6	53640.9
			10^{-3}	2.155	2.755	2.184	2.184	2.185	2.185	2.080	2.410	2.128	2.128	2.130	2.130	4078.1	53641.4
		0.0	10^{-2}	3.331	3.812	1.994	3.893	2.454	3.912	3.257	3.881	1.996	3.958	2.171	3.969	1024.7	77926.4
			10^{-3}	3.332	3.813	1.994	3.892	2.454	3.910	3.260	3.882	1.996	3.955	2.171	3.965	2174.1	77922.8
			10^{-2}	3.348	3.823	1.994	3.897	2.447	3.916	3.272	3.896	1.996	3.964	2.167	3.975	846.01	77996.9
	hexa	0.1	10^{-3}	3.350	3.823	1.994	3.896	2.447	3.914	3.275	3.896	1.996	3.961	2.167	3.971	1884.3	77844.3
			10^{-2}	3.196	3.775	1.927	3.844	2.032	3.845	3.461	3.808	1.961	3.840	1.977	3.846	1658.5	45560.0
			10^{-3}	3.198	3.777	1.927	3.839	2.032	3.841	3.463	3.810	1.961	3.829	1.977	3.835	3649.2	45565.9
		0.0	10^{-2}	3.450	3.862	1.927	3.883	2.028	3.884	3.777	3.920	1.961	3.904	1.974	3.909	1726.4	53889.0
			10^{-3}	3.452	3.863	1.927	3.878	2.028	3.879	3.775	3.921	1.961	3.893	1.974	3.898	4018.1	53890.5
			10^{-2}	3.331	3.812	1.994	3.886	2.452	3.905	3.257	3.881	1.995	3.960	2.170	3.970	1232.6	79222.2
p3-p3	tetra unstr	0.1	10^{-3}	3.332	3.813	1.994	3.884	2.452	3.904	3.260	3.882	1.995	3.956	2.170	3.967	3319.0	79065.9
			10^{-2}	3.348	3.823	1.993	3.890	2.446	3.909	3.272	3.896	1.995	3.966	2.167	3.976	1248.1	79294.2
			10^{-3}	3.349	3.823	1.993	3.889	2.446	3.908	3.275	3.896	1.995	3.962	2.167	3.973	3166.2	79290.2
		0.0	10^{-2}	3.195	3.776	1.923	3.842	2.021	3.844	3.460	3.809	1.958	3.837	1.974	3.844	1679.4	46150.6
			10^{-3}	3.197	3.777	1.923	3.838	2.021	3.840	3.462	3.810	1.958	3.826	1.974	3.833	3682.2	46152.2
			10^{-2}	3.450	3.862	1.923	3.884	2.017	3.885	3.777	3.921	1.958	3.904	1.971	3.911	1530.7	54383.0
	hexa	0.1	10^{-3}	3.451	3.863	1.923	3.879	2.017	3.880	3.775	3.921	1.958	3.893	1.971	3.900	4260.6	54387.8

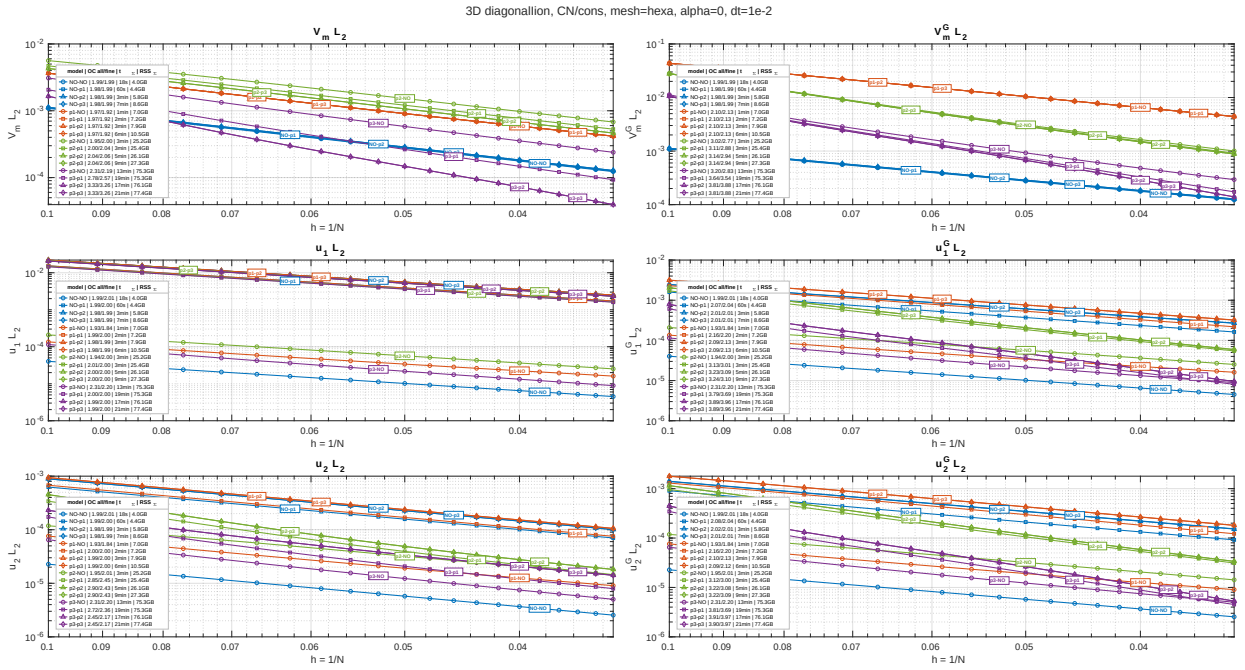


Figure 33: 3D convergence block for diagonalion, hexa mesh, $\alpha = 0$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

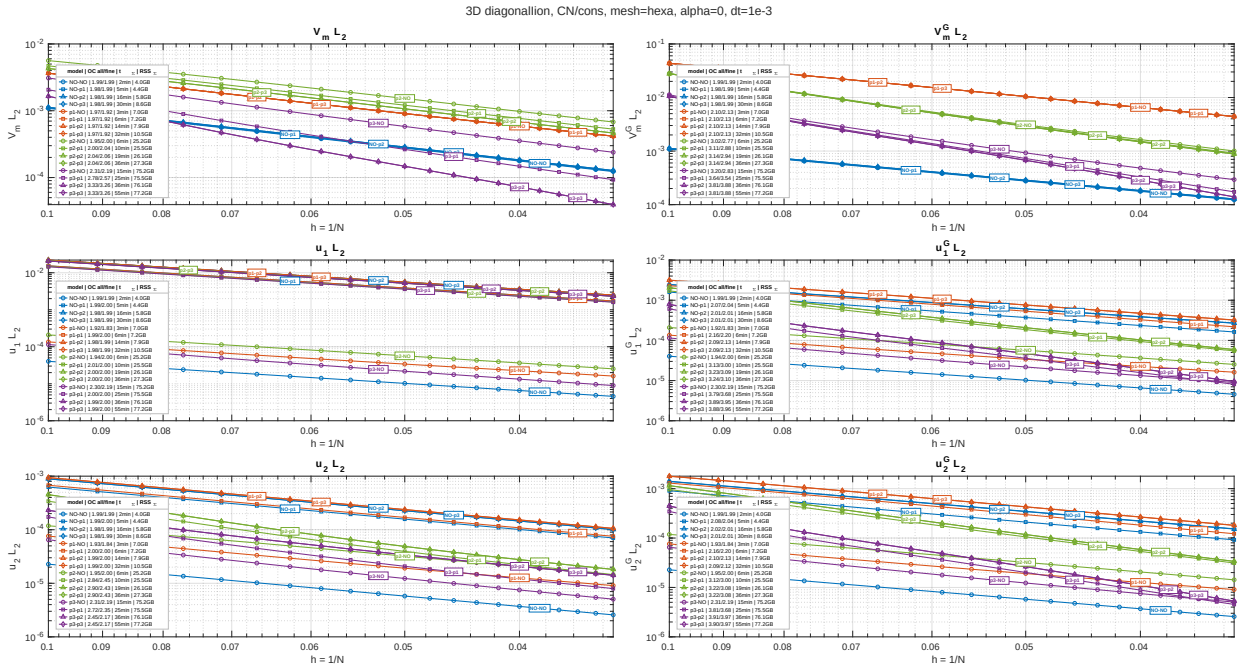


Figure 34: 3D convergence block for diagonalion, hexa mesh, $\alpha = 0$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m-I_{ion} model pair.

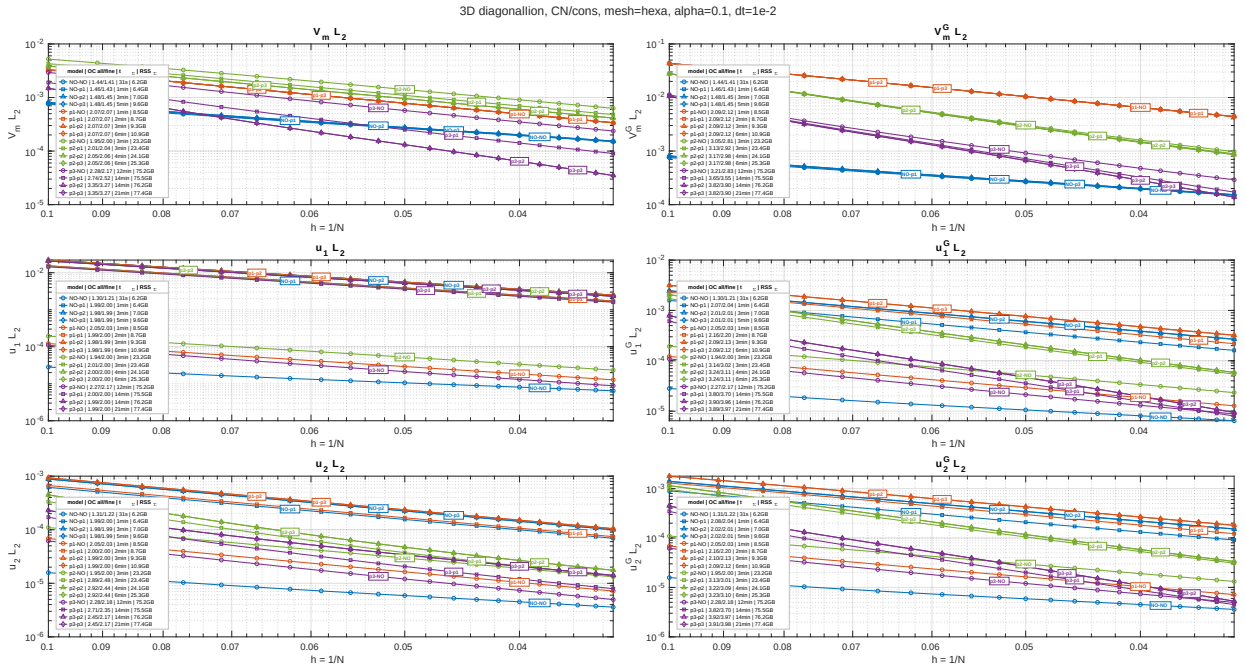


Figure 35: 3D convergence block for `diagonallion`, hexa mesh, $\alpha = 0.1$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the $V_m - I_{\text{ion}}$ model pair.

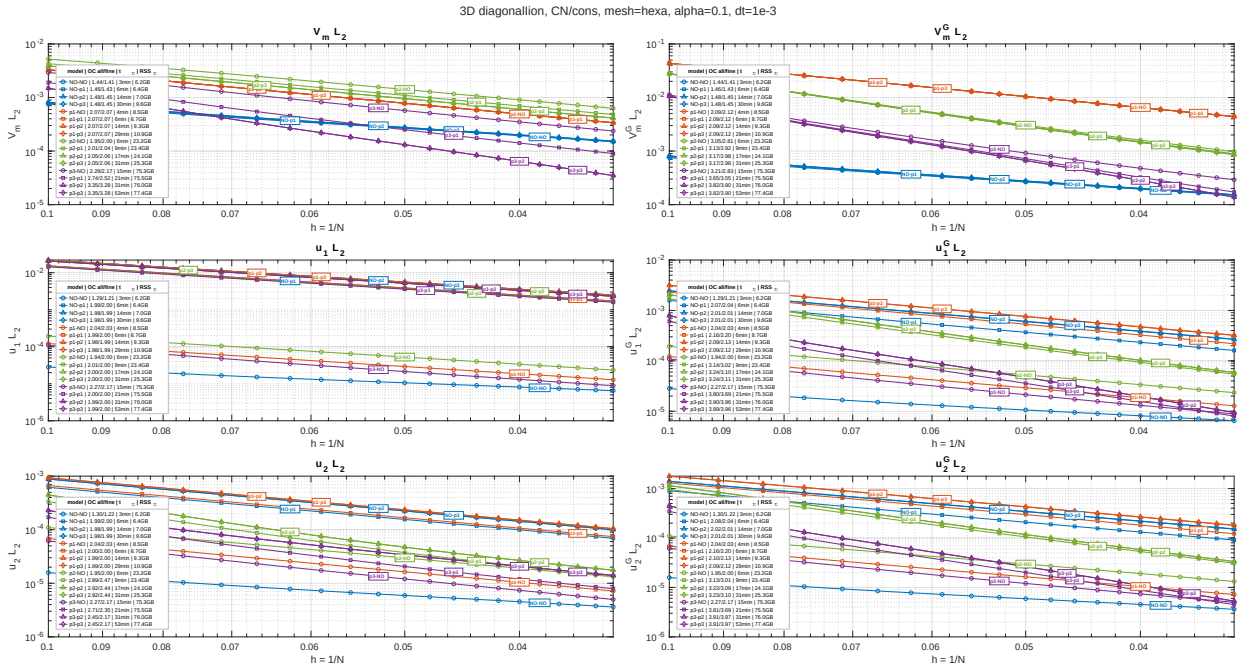


Figure 36: 3D convergence block for diagonalIon, hexa mesh, $\alpha = 0.1$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the $V_m - I_{\text{ion}}$ model pair.

3D diagonallion, CN/cons, mesh=triangular unstr, alpha=0, dt=1e-2

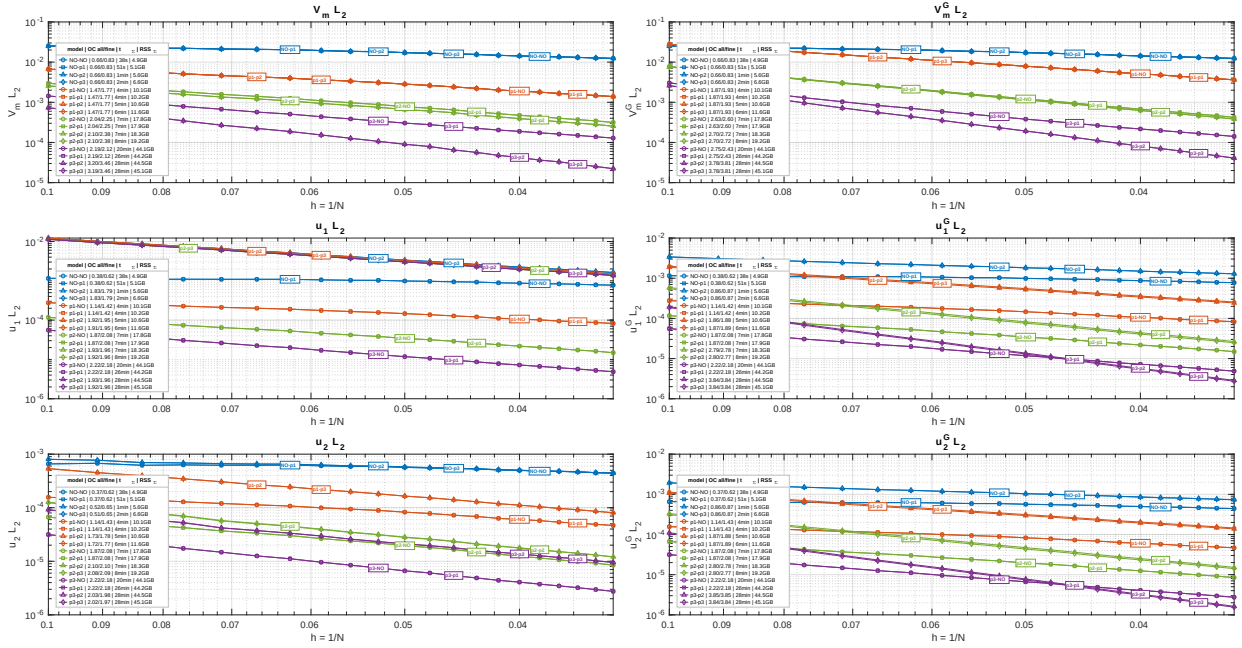


Figure 37: 3D convergence block for diagonalIion, tetra unstr mesh, $\alpha = 0$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

3D diagonallion, CN/cons, mesh=triangular unstr, alpha=0, dt=1e-3

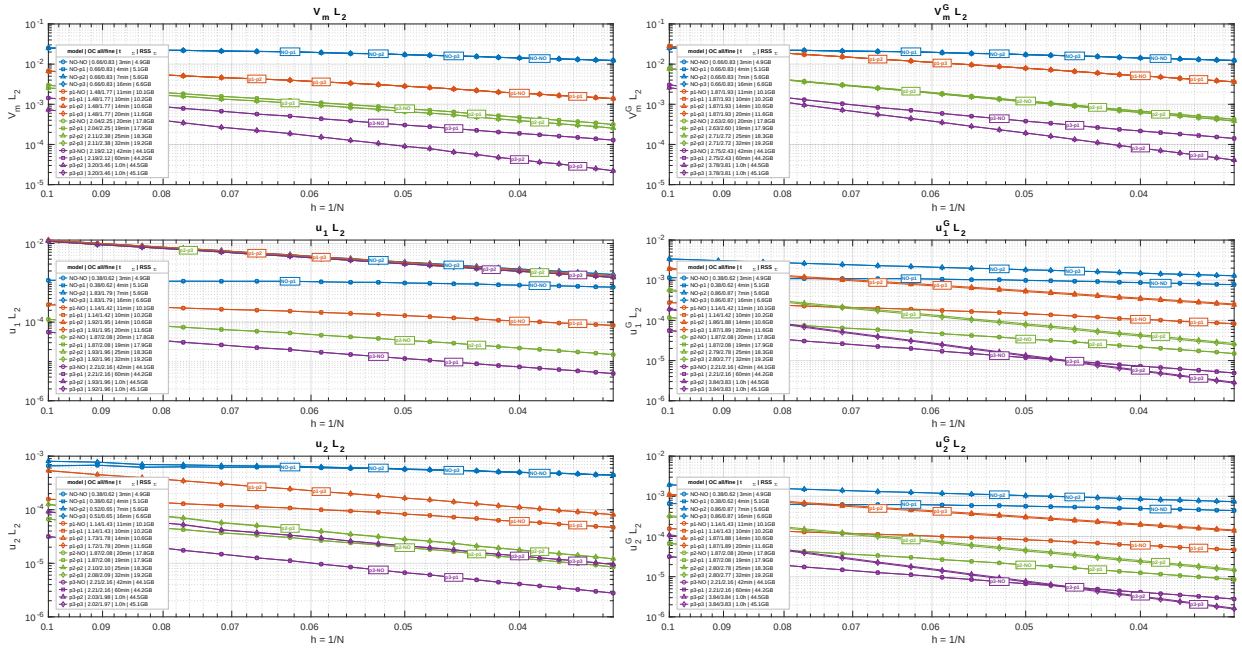


Figure 38: 3D convergence block for diagonalIion, tetra unstr mesh, $\alpha = 0$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

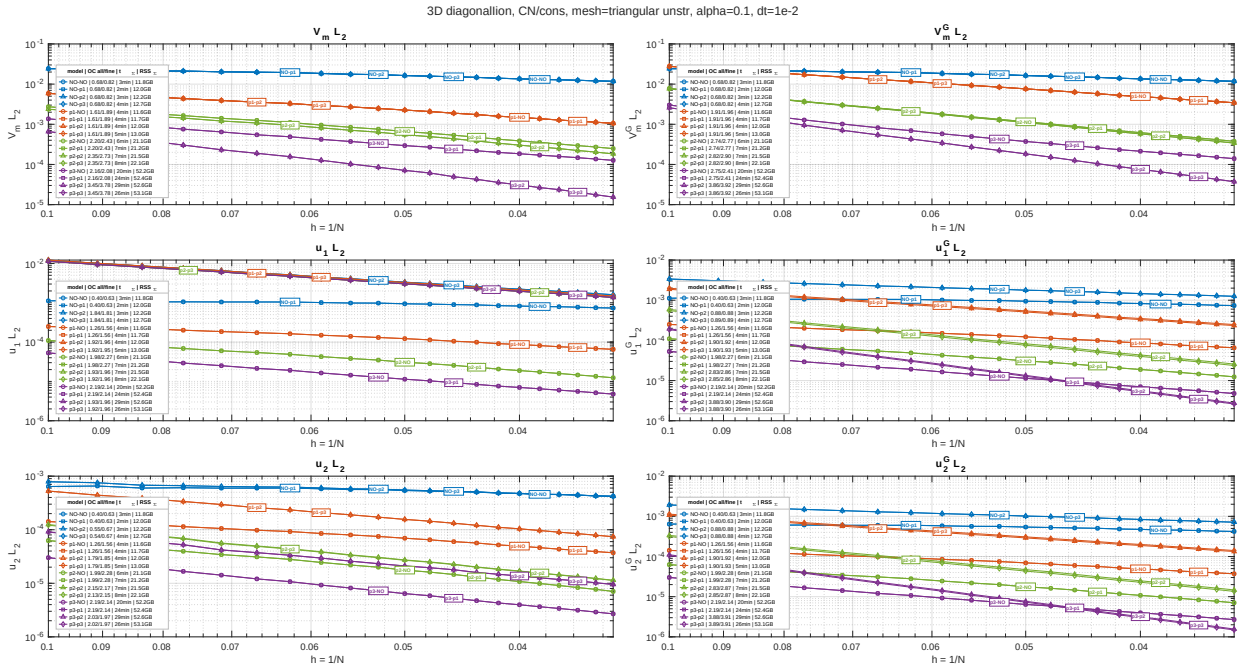


Figure 39: 3D convergence block for `diagonalIion`, tetra unstr mesh, $\alpha = 0.1$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m – I_{ion} model pair.

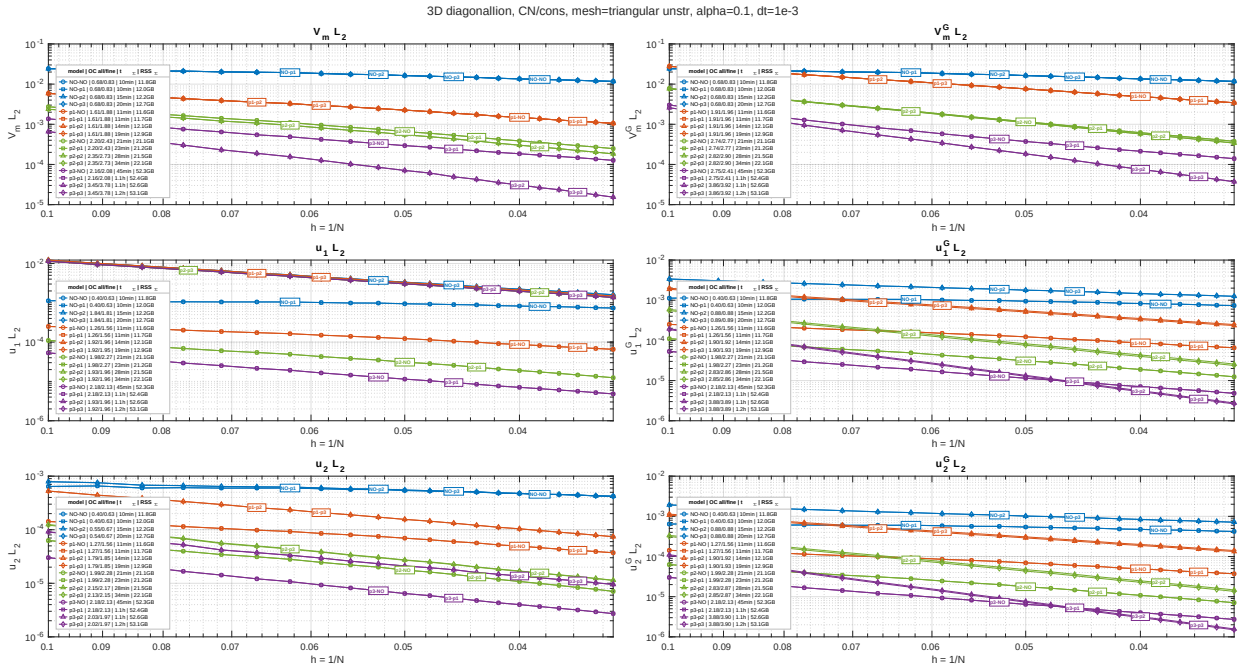


Figure 40: 3D convergence block for `diagonalIion`, tetra unstr mesh, $\alpha = 0.1$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m-I_{ion} model pair.

11.3.3 JFNK, crankNicolson, consistent

Table 7: 3D manufactured results for JFNK, crankNicolson, and the consistent mass matrix. The first six convergence columns are log–log fits over meshes $N = 10$ –30; the next six columns are fits over $N = 20$ –30. The time column is the sum of the reported wall-clock total over all mesh runs in the row, and the memory column is the sum of the reported peak resident memory in MB over the same mesh runs.

$V_m - I_{\text{ion}}$	Mesh type	α	Δt [s]	Fit $N = 10$ –30						Fit $N = 20$ –30						t_{Σ} [s]	RSS $_{\Sigma}$ [MB]
				V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G		
NO–NO	hexa	0.0	10^{-2}	1.985	1.985	1.994	1.994	1.994	1.994	1.993	1.993	2.009	2.009	2.009	2.009	10.903	4081.0
			10^{-3}	1.985	1.985	1.985	1.985	1.985	1.985	1.993	1.993	1.993	1.993	1.993	1.993	72.358	4083.5
		0.1	10^{-2}	1.436	1.436	1.297	1.297	1.306	1.306	1.411	1.411	1.214	1.214	1.223	1.223	26.673	6311.1
			10^{-3}	1.438	1.438	1.293	1.293	1.302	1.302	1.413	1.413	1.211	1.211	1.220	1.220	103.30	6309.8
	tetra unstr	0.0	10^{-2}	0.664	0.664	0.375	0.375	0.373	0.373	0.826	0.826	0.617	0.617	0.616	0.616	46.900	5184.6
			10^{-3}	0.663	0.663	0.377	0.377	0.375	0.375	0.829	0.829	0.617	0.617	0.617	0.617	166.32	5142.7
NO–p1	hexa	0.0	10^{-2}	1.984	1.984	1.992	2.072	1.992	2.077	1.992	1.992	1.996	2.038	1.997	2.041	50.383	4665.5
			10^{-3}	1.985	1.985	1.992	2.071	1.992	2.077	1.993	1.993	1.996	2.038	1.996	2.041	274.71	4669.2
		0.1	10^{-2}	1.459	1.459	1.992	2.073	1.993	2.079	1.429	1.429	1.996	2.038	1.996	2.041	45.835	6563.2
			10^{-3}	1.461	1.461	1.992	2.073	1.992	2.079	1.431	1.431	1.996	2.038	1.995	2.041	286.77	6561.2
	tetra unstr	0.0	10^{-2}	0.664	0.664	0.375	0.375	0.373	0.373	0.826	0.826	0.617	0.617	0.616	0.616	48.848	5486.9
			10^{-3}	0.663	0.663	0.377	0.377	0.375	0.375	0.829	0.829	0.617	0.617	0.617	0.617	172.50	5446.4
NO–p2	hexa	0.0	10^{-2}	1.983	1.983	1.984	2.014	1.985	2.018	1.992	1.992	1.992	2.007	1.993	2.009	130.18	6314.9
			10^{-3}	1.984	1.984	1.984	2.014	1.984	2.018	1.992	1.992	1.992	2.007	1.993	2.009	916.82	6341.8
		0.1	10^{-2}	1.482	1.482	1.984	2.015	1.985	2.018	1.447	1.447	1.992	2.007	1.993	2.010	110.28	7695.8
			10^{-3}	1.483	1.483	1.984	2.014	1.985	2.018	1.450	1.450	1.992	2.007	1.992	2.009	749.00	7704.6
	tetra unstr	0.0	10^{-2}	0.664	0.664	1.832	0.861	0.516	0.859	0.826	0.826	1.792	0.867	0.652	0.866	134.81	6127.4
			10^{-3}	0.663	0.663	1.832	0.862	0.518	0.860	0.829	0.829	1.792	0.867	0.653	0.866	507.51	6090.2
NO–p3	hexa	0.0	10^{-2}	1.983	1.983	1.983	2.010	1.984	2.014	1.992	1.992	1.992	2.005	1.993	2.008	326.06	9615.9
			10^{-3}	1.984	1.984	1.983	2.010	1.984	2.014	1.992	1.992	1.992	2.005	1.992	2.008	2080.8	9677.3
		0.1	10^{-2}	1.482	1.482	1.984	2.011	1.985	2.015	1.447	1.447	1.992	2.006	1.992	2.008	241.31	11005.2
			10^{-3}	1.483	1.483	1.984	2.011	1.984	2.015	1.450	1.450	1.992	2.005	1.992	2.008	1710.3	11009.2
	tetra unstr	0.0	10^{-2}	0.664	0.664	1.828	0.863	0.513	0.862	0.826	0.826	1.790	0.868	0.651	0.868	256.29	7394.5
			10^{-3}	0.663	0.663	1.828	0.864	0.514	0.862	0.829	0.829	1.790	0.869	0.652	0.868	907.09	7355.9
p1–NO	hexa	0.0	10^{-2}	1.983	1.983	1.983	2.010	1.984	2.014	1.992	1.992	1.992	2.005	1.993	2.008	326.06	9615.9
			10^{-3}	1.984	1.984	1.983	2.010	1.984	2.014	1.992	1.992	1.992	2.005	1.992	2.008	2080.8	9677.3
		0.1	10^{-2}	1.482	1.482	1.984	2.011	1.985	2.015	1.447	1.447	1.992	2.006	1.992	2.008	241.31	11005.2
			10^{-3}	1.483	1.483	1.984	2.011	1.984	2.015	1.450	1.450	1.992	2.005	1.992	2.008	1710.3	11009.2
	tetra unstr	0.0	10^{-2}	0.664	0.664	1.828	0.863	0.513	0.862	0.826	0.826	1.790	0.868	0.651	0.868	256.29	7394.5
			10^{-3}	0.663	0.663	1.828	0.864	0.514	0.862	0.829	0.829	1.790	0.869	0.652	0.868	907.09	7355.9
p1–p1	hexa	0.0	10^{-2}	1.967	2.095	1.927	1.927	1.930	1.930	1.919	2.125	1.838	1.838	1.845	1.845	73.508	7146.7
			10^{-3}	1.967	2.095	1.924	1.924	1.927	1.927	1.920	2.125	1.833	1.833	1.840	1.840	156.75	7136.1
		0.1	10^{-2}	2.067	2.093	2.047	2.047	2.048	2.048	2.066	2.124	2.033	2.033	2.035	2.035	66.261	8682.2
			10^{-3}	2.067	2.093	2.044	2.044	2.044	2.044	2.067	2.124	2.028	2.028	2.029	2.029	152.25	8682.2
	tetra unstr	0.0	10^{-2}	1.474	1.866	1.140	1.140	1.142	1.142	1.771	1.928	1.419	1.419	1.428	1.428	201.56	10384.5
			10^{-3}	1.475	1.866	1.142	1.142	1.144	1.144	1.768	1.927	1.422	1.422	1.432	1.432	382.68	10354.9
p1–p2	hexa	0.0	10^{-2}	1.613	1.909	1.262	1.262	1.264	1.264	1.889	1.963	1.556	1.556	1.558	1.558	201.05	11873.0
			10^{-3}	1.612	1.909	1.265	1.265	1.267	1.267	1.884	1.962	1.558	1.558	1.561	1.561	450.48	11878.2
		0.1	10^{-2}	1.967	2.095	1.992	2.157	1.999	2.164	1.919	2.125	1.997	2.201	2.001	2.201	94.746	7340.6
			10^{-3}	1.968	2.095	1.992	2.157	1.999	2.164	1.920	2.125	1.997	2.201	2.001	2.200	321.20	7342.0
	tetra unstr	0.0	10^{-2}	2.067	2.093	1.993	2.155	2.002	2.162	2.066	2.124	1.997	2.199	2.005	2.199	86.296	8892.3
			10^{-3}	2.067	2.093	1.993	2.155	2.001	2.162	2.067	2.124	1.997	2.199	2.004	2.199	312.60	8892.4
p1–p3	hexa	0.0	10^{-2}	1.474	1.866	1.140	1.140	1.142	1.142	1.771	1.928	1.419	1.419	1.428	1.428	207.54	10483.4
			10^{-3}	1.475	1.866	1.142	1.142	1.144	1.144	1.768	1.927	1.422	1.422	1.432	1.432	523.84	10459.1
		0.1	10^{-2}	1.613	1.909	1.262	1.262	1.264	1.264	1.889	1.963	1.556	1.556	1.558	1.558	211.38	11964.7
			10^{-3}	1.612	1.909	1.265	1.265	1.267	1.267	1.884	1.962	1.558	1.558	1.561	1.561	524.88	11970.4
	tetra unstr	0.0	10^{-2}	1.968	2.095	1.984	2.094	1.989	2.098	1.921	2.125	1.993	2.129	1.996	2.128	829.14	8590.9
			10^{-3}	1.968	2.095	1.984	2.094	1.989	2.098	1.921	2.125	1.993	2.129	1.996	2.128	829.14	8590.9
p2–NO	hexa	0.0	10^{-2}	2.067	2.093	1.985	2.093	1.991	2.096	2.066	2.124	1.993	2.128	1.998	2.126	146.84	9578.2
			10^{-3}	2.067	2.093	1.985	2.092	1.991	2.096	2.067	2.124	1.993	2.128	1.998	2.126	807.62	9577.6
		0.1	10^{-2}	1.475	1.866	1.918	1.862	1.726	1.866	1.771	1.928	1.953	1.877	1.777	1.880	282.21	11109.8
			10^{-3}	1.475	1.866	1.918	1.862	1.726	1.866	1.768	1.927	1.953	1.877	1.777	1.880	711.22	11085.8
	tetra unstr	0.0	10^{-2}	1.613	1.909	1.920	1.897	1.790	1.900	1.889	1.963	1.955	1.921	1.848	1.922	322.57	12472.4
			10^{-3}	1.612	1.909	1.920	1.897	1.790	1.900	1.884	1.962	1.955	1.921	1.849	1.923	907.71	12474.6
p2–p1	hexa	0.0	10^{-2}	1.968	2.095	1.984	2.091	1.989	2.094	1.920	2.125	1.993	2.126	1.996	2.125	316.14	12162.0
			10^{-3}	1.968	2.095	1.984	2.091	1.989	2.094	1.921	2.125	1.993	2.126	1.996	2.124	1859.7	12031.6
		0.1	10^{-2}	2.067	2.093	1.984	2.089	1.990	2.093	2.066	2.124	1.993	2.124	1.998	2.123	275.06	12472.4
			10^{-3}	2.067	2.093	1.984	2.089	1.990	2.093	2.067	2.124	1.993	2.124	1.998	2.123	1756.2	12472.6
	tetra unstr	0.0	10^{-2}	1.475	1.866	1.914	1.870	1.723	1.874	1.771	1.928	1.951	1.889	1.775	1.892	354.44	12384.3
			10^{-3}	1.475	1.866	1.914	1.870	1.723	1.873	1.768	1.9						

$V_m - I_{\text{ion}}$	Mesh type	α	Δt [s]	Fit $N = 10-30$						Fit $N = 20-30$						t_{Σ} [s]	RSS $_{\Sigma}$ [MB]
				V_m	V_m^G	u_1	u_1^G	u_2	u_2^G	V_m	V_m^G	u_1	u_1^G	u_2	u_2^G		
p2-p2	tetra unstr	0.1	10^{-3}	2.004	3.107	2.011	3.133	2.844	3.122	2.035	2.884	2.004	3.004	2.447	2.998	475.17	26058.0
			10^{-2}	2.008	3.133	2.011	3.142	2.888	3.131	2.037	2.925	2.004	3.021	2.476	3.015	162.51	21986.0
			10^{-3}	2.008	3.133	2.011	3.142	2.886	3.131	2.037	2.925	2.004	3.021	2.473	3.014	434.84	21987.3
		0.0	10^{-2}	2.043	2.626	1.868	1.868	1.869	1.869	2.248	2.602	2.078	2.078	2.084	2.084	358.27	18397.1
			10^{-3}	2.044	2.627	1.867	1.867	1.868	1.868	2.248	2.603	2.077	2.077	2.084	2.084	923.33	18403.9
			10^{-2}	2.198	2.743	1.984	1.984	1.987	1.987	2.433	2.768	2.271	2.271	2.279	2.279	366.28	21760.9
	hexa	0.1	10^{-3}	2.198	2.744	1.985	1.985	1.987	1.987	2.429	2.767	2.271	2.271	2.279	2.279	1077.7	21759.6
			10^{-2}	2.041	3.142	2.004	3.230	2.898	3.215	2.059	2.936	2.001	3.092	2.432	3.083	259.13	26708.4
			10^{-3}	2.041	3.142	2.004	3.230	2.897	3.215	2.060	2.936	2.001	3.092	2.431	3.082	1125.6	26708.6
		0.0	10^{-2}	2.050	3.166	2.004	3.236	2.920	3.222	2.064	2.975	2.001	3.105	2.440	3.095	232.31	22636.7
			10^{-3}	2.051	3.166	2.004	3.236	2.920	3.221	2.064	2.975	2.001	3.105	2.439	3.094	919.07	22635.8
			10^{-2}	2.103	2.704	1.928	2.794	2.100	2.796	2.384	2.719	1.960	2.779	2.102	2.784	382.90	19041.3
p2-p3	tetra unstr	0.1	10^{-3}	2.105	2.705	1.928	2.794	2.100	2.795	2.384	2.720	1.960	2.779	2.102	2.783	1131.3	19048.7
			10^{-2}	2.349	2.824	1.928	2.831	2.146	2.833	2.732	2.904	1.960	2.861	2.166	2.866	390.35	22132.5
			10^{-3}	2.350	2.825	1.928	2.831	2.146	2.833	2.727	2.903	1.960	2.862	2.167	2.866	1319.3	22134.4
		0.0	10^{-2}	2.041	3.142	2.003	3.235	2.897	3.220	2.059	2.936	2.000	3.095	2.432	3.085	436.66	28002.5
			10^{-3}	2.041	3.142	2.003	3.235	2.896	3.220	2.060	2.936	2.000	3.095	2.430	3.085	2346.5	28002.5
			10^{-2}	2.050	3.166	2.003	3.242	2.920	3.227	2.064	2.975	2.000	3.108	2.439	3.097	357.00	24079.3
	hexa	0.1	10^{-3}	2.051	3.166	2.003	3.241	2.919	3.226	2.064	2.975	2.000	3.108	2.438	3.097	1865.3	24076.1
			10^{-2}	2.103	2.704	1.924	2.802	2.082	2.802	2.384	2.719	1.958	2.769	2.088	2.773	406.97	20303.3
			10^{-3}	2.105	2.705	1.924	2.801	2.082	2.802	2.384	2.720	1.958	2.768	2.089	2.773	1507.3	20304.2
		0.0	10^{-2}	2.349	2.824	1.924	2.847	2.128	2.847	2.732	2.904	1.958	2.864	2.152	2.868	453.30	23026.1
			10^{-3}	2.350	2.825	1.924	2.846	2.128	2.847	2.727	2.903	1.958	2.864	2.153	2.868	1828.5	23037.4
			10^{-2}	2.313	3.204	2.308	2.308	2.312	2.312	2.193	2.830	2.197	2.197	2.200	2.200	872.23	77059.3
p3-NO	tetra unstr	0.1	10^{-3}	2.313	3.204	2.303	2.303	2.307	2.307	2.193	2.830	2.189	2.189	2.192	2.192	816.96	68118.1
			10^{-2}	2.278	3.207	2.274	2.274	2.278	2.278	2.169	2.826	2.174	2.174	2.176	2.176	725.16	72597.5
			10^{-3}	2.278	3.207	2.269	2.269	2.273	2.273	2.169	2.827	2.166	2.166	2.168	2.168	892.18	72598.4
		0.0	10^{-2}	2.189	2.753	2.216	2.216	2.217	2.217	2.121	2.431	2.176	2.176	2.179	2.179	1102.0	45158.7
			10^{-3}	2.189	2.753	2.208	2.208	2.209	2.209	2.121	2.431	2.162	2.162	2.165	2.165	1833.8	45160.4
			10^{-2}	2.155	2.754	2.192	2.192	2.193	2.193	2.080	2.410	2.142	2.142	2.144	2.144	1239.7	53472.5
	hexa	0.1	10^{-3}	2.155	2.755	2.184	2.184	2.185	2.185	2.080	2.410	2.128	2.128	2.130	2.130	2087.5	53471.6
			10^{-2}	2.780	3.644	2.003	3.792	2.717	3.810	2.572	3.538	2.000	3.687	2.355	3.691	1126.1	77279.7
			10^{-3}	2.781	3.644	2.003	3.789	2.716	3.807	2.573	3.538	2.000	3.679	2.354	3.683	1453.9	77276.2
		0.0	10^{-2}	2.737	3.653	2.003	3.799	2.707	3.817	2.522	3.546	2.000	3.695	2.347	3.700	777.00	72810.8
			10^{-3}	2.738	3.653	2.003	3.796	2.706	3.814	2.523	3.546	2.000	3.687	2.345	3.692	1107.5	68550.1
			10^{-2}	2.189	2.753	2.216	2.216	2.217	2.217	2.121	2.431	2.176	2.176	2.179	2.179	1407.6	45295.2
p3-p1	tetra unstr	0.1	10^{-3}	2.189	2.753	2.208	2.208	2.209	2.209	2.121	2.431	2.162	2.162	2.165	2.165	2698.7	45294.4
			10^{-2}	2.155	2.754	2.192	2.192	2.193	2.193	2.080	2.410	2.142	2.142	2.144	2.144	1255.5	53554.9
			10^{-3}	2.155	2.755	2.184	2.184	2.185	2.185	2.080	2.410	2.128	2.128	2.130	2.130	2581.7	53555.5
		0.0	10^{-2}	3.331	3.812	1.994	3.893	2.454	3.912	3.257	3.881	1.996	3.958	2.171	3.969	1014.5	77925.3
			10^{-3}	3.332	3.813	1.994	3.892	2.454	3.910	3.260	3.882	1.996	3.955	2.171	3.965	2161.7	77923.7
			10^{-2}	3.348	3.823	1.994	3.897	2.447	3.916	3.272	3.896	1.996	3.964	2.167	3.975	750.17	69196.2
	hexa	0.1	10^{-3}	3.350	3.823	1.994	3.896	2.447	3.914	3.275	3.896	1.996	3.961	2.167	3.971	1552.4	69194.7
			10^{-2}	3.196	3.775	1.927	3.844	2.032	3.845	3.461	3.808	1.961	3.840	1.977	3.846	1367.1	45574.8
			10^{-3}	3.198	3.777	1.927	3.839	2.032	3.841	3.463	3.810	1.961	3.829	1.977	3.835	2892.2	45571.9
		0.0	10^{-2}	3.450	3.862	1.927	3.883	2.028	3.884	3.777	3.920	1.961	3.904	1.974	3.909	1245.5	53807.3
			10^{-3}	3.452	3.863	1.927	3.878	2.028	3.879	3.775	3.921	1.961	3.893	1.974	3.898	2807.7	53808.9
			10^{-2}	3.331	3.812	1.994	3.886	2.452	3.905	3.257	3.881	1.995	3.960	2.170	3.970	1172.7	79220.7
p3-p3	tetra unstr	0.1	10^{-3}	3.332	3.813	1.994	3.884	2.452	3.904	3.260	3.882	1.995	3.956	2.170	3.967	3441.1	79067.7
			10^{-2}	3.348	3.823	1.993	3.890	2.446	3.909	3.272	3.896	1.995	3.966	2.167	3.976	870.62	70486.9
			10^{-3}	3.349	3.823	1.993	3.889	2.446	3.908	3.275	3.896	1.995	3.962	2.167	3.973	2516.6	70489.4
		0.0	10^{-2}	3.195	3.776	1.923	3.842	2.021	3.844	3.460	3.809	1.958	3.837	1.974	3.844	1258.6	46320.8
			10^{-3}	3.197	3.777	1.923	3.838	2.021	3.840	3.462	3.810	1.958	3.826	1.974	3.833	2931.1	46322.2
			10^{-2}	3.450	3.862	1.923	3.884	2.017	3.885	3.777	3.921	1.958	3.904	1.971	3.911	1397.1	54335.8
	hexa	0.1	10^{-3}	3.451	3.863	1.923	3.879	2.017	3.880	3.775	3.921	1.958	3.893	1.971	3.900	3400.9	54328.9

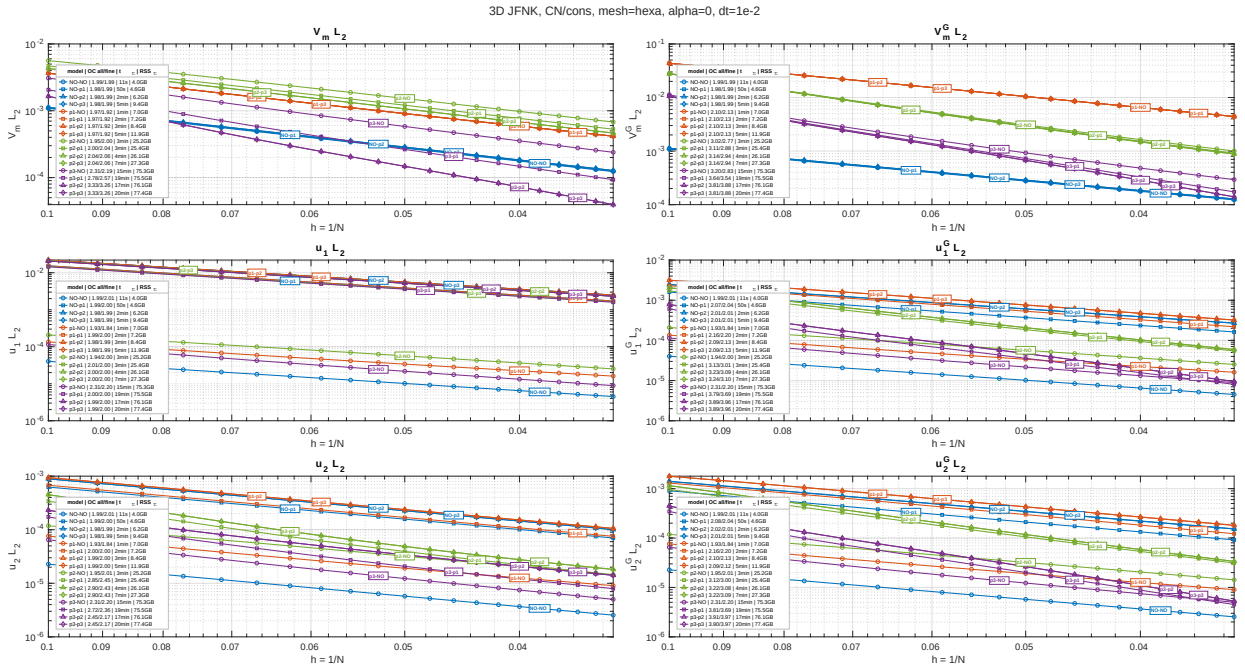


Figure 41: 3D convergence block for JFNK, hexa mesh, $\alpha = 0$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m-I_{ion} model pair.

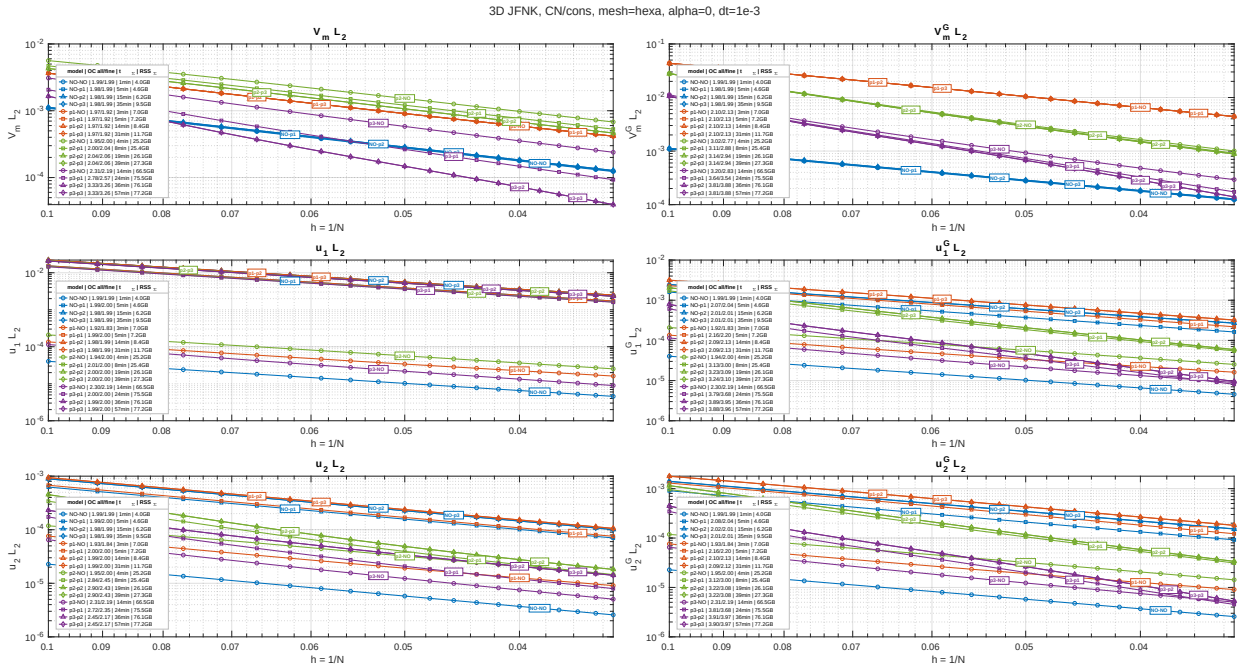


Figure 42: 3D convergence block for JFNK, hexa mesh, $\alpha = 0$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m – I_{ion} model pair.

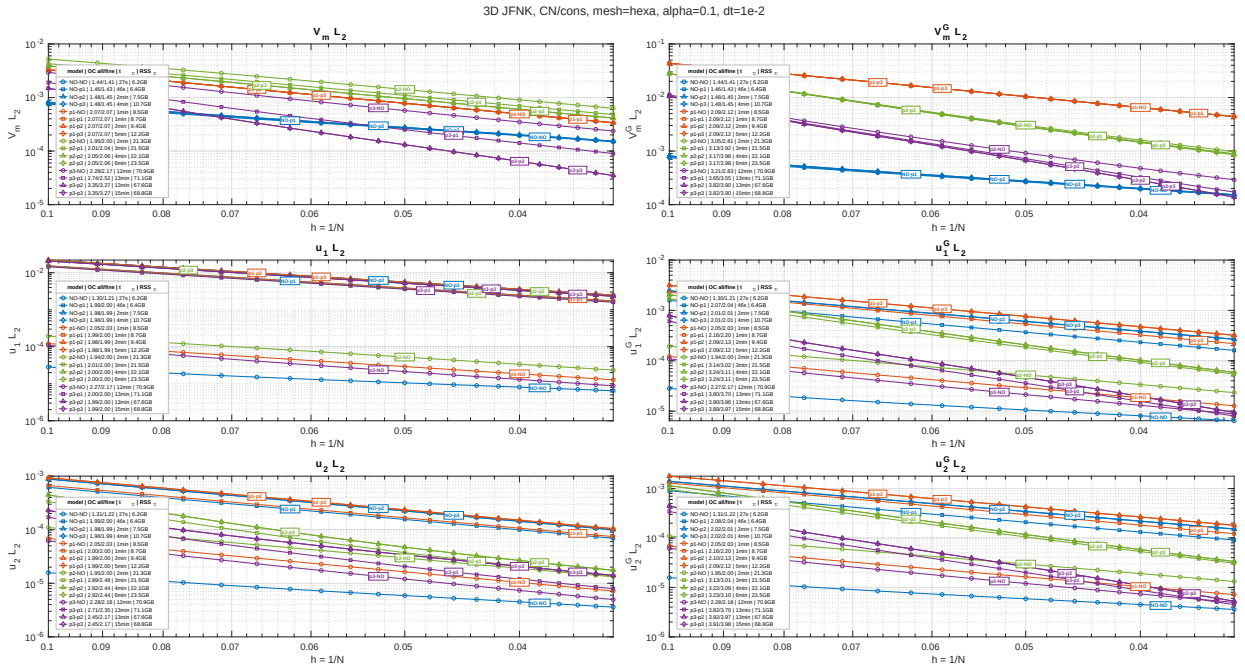


Figure 43: 3D convergence block for JFNK, hexa mesh, $\alpha = 0.1$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the $V_m - I_{\text{ion}}$ model pair.

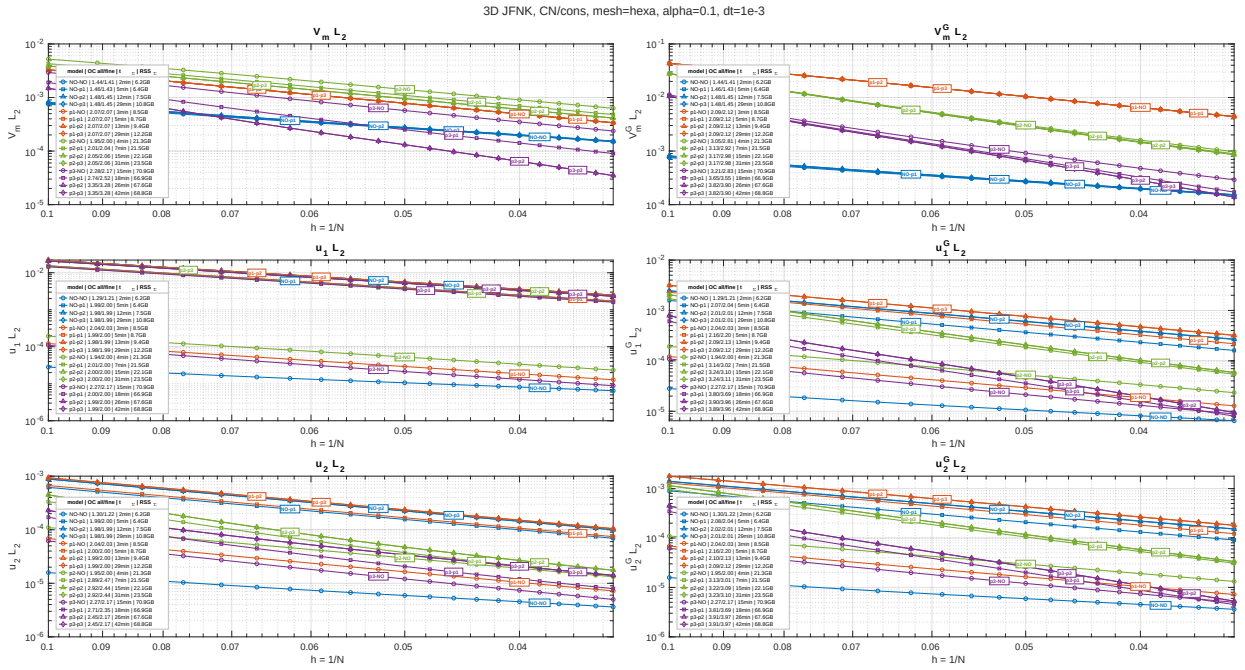


Figure 44: 3D convergence block for JFNK, hexa mesh, $\alpha = 0.1$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the $V_m - I_{\text{ion}}$ model pair.

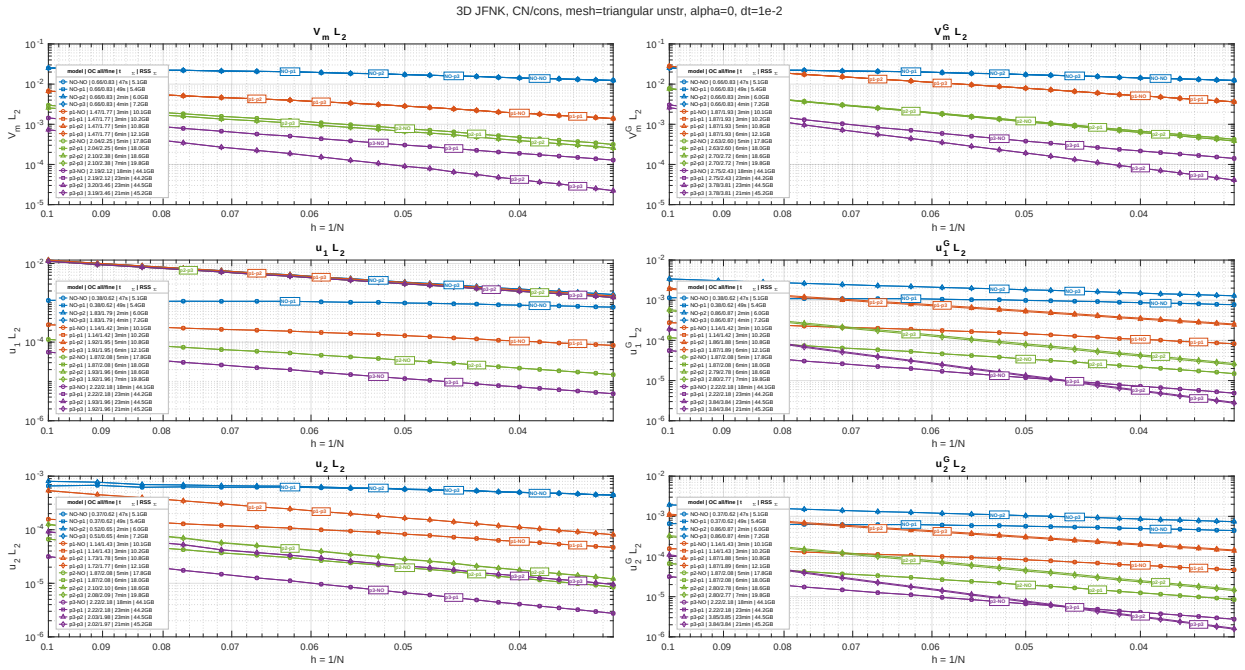
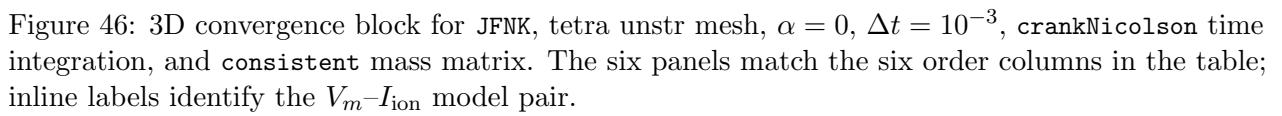


Figure 45: 3D convergence block for JFNK, tetra unstr mesh, $\alpha = 0$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m – I_{ion} model pair.



3D JFNK, CN/cons, mesh=triangular unstr, alpha=0.1, dt=1e-2

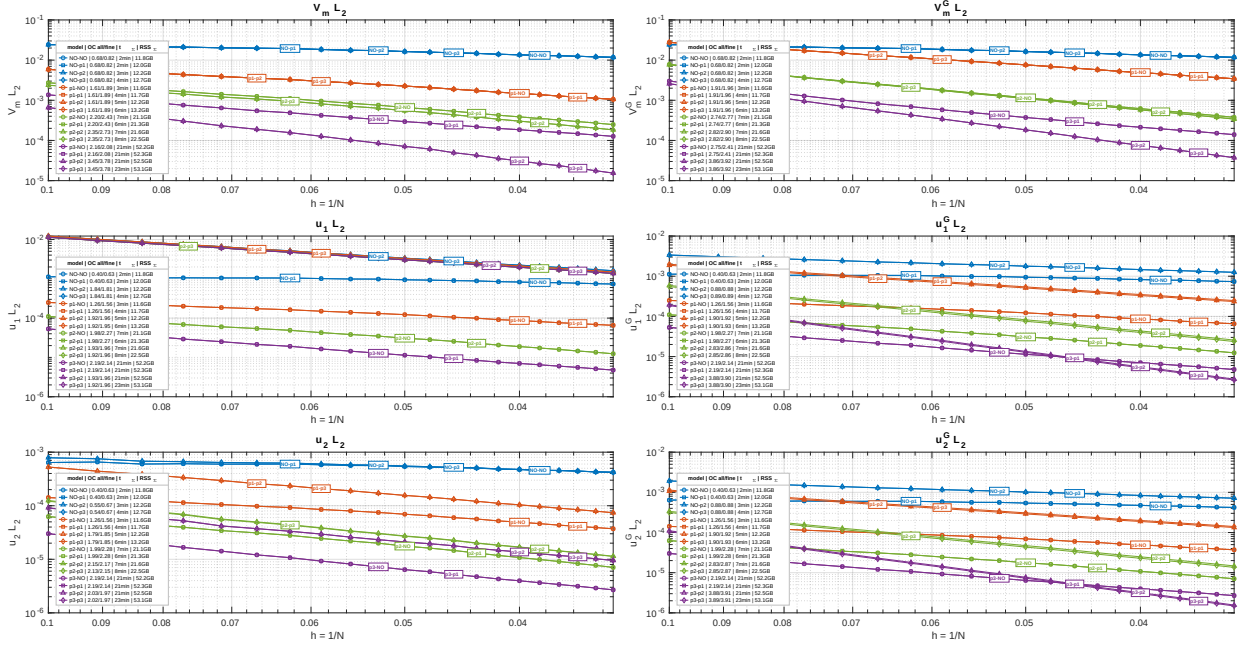


Figure 47: 3D convergence block for JFNK, tetra unstr mesh, $\alpha = 0.1$, $\Delta t = 10^{-2}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

3D JFNK, CN/cons, mesh=triangular unstr, alpha=0.1, dt=1e-3

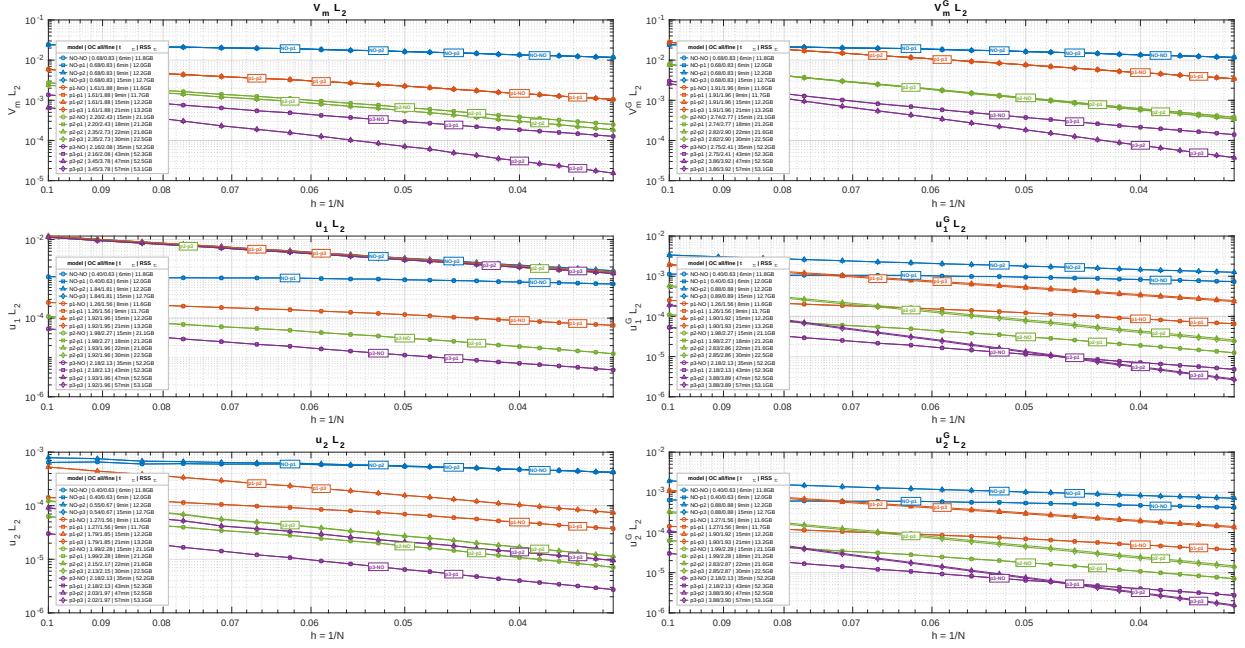


Figure 48: 3D convergence block for JFNK, tetra unstr mesh, $\alpha = 0.1$, $\Delta t = 10^{-3}$, crankNicolson time integration, and consistent mass matrix. The six panels match the six order columns in the table; inline labels identify the V_m - I_{ion} model pair.

3D comments and conclusions. The three nonlinear strategies give indistinguishable convergence orders in the 3D MMS data when the same spatial, temporal, and mass-matrix settings are used. For the structured hexa mesh, the p3/p2 and p3/p3 configurations with $\Delta t = 10^{-3}$ recover approximately fourth-order reconstructed diagnostics, while the cell-centred voltage remains slightly below fourth order over $N = 10$ –30. The unstructured simplex mesh is more sensitive to α and to the available mesh range, but the same high-order configurations remain the clearest verification cases. Runtime and RSS columns should be read as accumulated sweep costs, not single-run costs.

12 Notes

The current result blocks are intentionally restricted to `crankNicolson` and the `consistent` mass matrix, so time-scheme or mass-matrix comparisons should be read from older auxiliary data rather than from these tables. The reconstructed quantities V_m^G , u_1^G , and u_2^G are quadrature-point/reconstructed diagnostics; the cell-centred fields are the values used in the main finite-volume update and in the L_2 convergence tables.